

Documento Requisiti — Ruffino Flow (PRD)

Stato: Documento vivente, riallineato allo stato corrente del checkout (05/09/2026). **Versione:** 5.36 - Fatturazione dal contratto e lettura del contratto PDF (piani 1-3 dietro flag; fixture d'oro dai fogli reali). Prima: 5.32 - Analisi azienda giornaliera di Tars (fotografia deterministica + sintesi del modello, proposte «Chiedi a Tars»). Prima: 5.31 - Tars libero (il modello decide, il dominio verifica; schede Proposte e Registro su /tars; smistamento D7/D8). Prima: 5.30 - Tars v2 è operativo e proattivo in produzione col provider reale, senza tetti di spesa (gate OpenAI §8) e con lo smistamento automatico delle comunicazioni (server/tars/smistamento/). La verità T0 su azioni disponibili, gap e accettazione è in docs/tars/matrice-azioni-tars.md e docs/tars/architettura-tars-v2.md ; la documentazione non deduce né attesta da sola lo stato di flag o provider in un ambiente esterno. §50 resta il registro storico della rimozione, §53 le compatibilità, §54 il progetto corrente. §55-§57 descrivono i tre piani del 3-5 settembre 2026 (contratto strutturato e computo dei limiti, fatturazione dal contratto, lettura del contratto PDF), tutti dietro interruttori fail-closed; §58 il piano 4, pianificato e non implementato. **Riferimento implementativo:** repository infissi-ops-app . Il presente PRD descrive il comportamento atteso del software così come è implementato; ogni divergenza riscontrata nel codice va trattata come bug.

0. Convenzioni del documento

- **MUST / DEVE** — requisito obbligatorio.
 - **SHOULD / DOVREBBE** — requisito fortemente consigliato.
 - **MAY / PUÒ** — opzione facoltativa.
 - Gli identificatori `in_questo_stile` sono valori interni (enum, chiavi di database). Le etichette UI in italiano sono indicate fra virgolette.
 - I numeri di sezione sono stabili; le sotto-sezioni possono crescere.
-

1. Visione del prodotto

Ruffino Flow è lo strumento operativo centrale di **Ruffino Immobiliare S.R.L.** Collega ufficio, laboratorio di produzione e cantiere, accompagnando ogni cliente dalla prima richiesta fino alla garanzia post-vendita. Non è un semplice database: è un assistente proattivo che ricorda le scadenze, blocca i passaggi di stato senza i documenti richiesti, unifica le comunicazioni e mostra a ogni ruolo il lavoro che gli tocca.

Pilastrì:

1. **Una commessa = un percorso tracciato.** Stato, documenti, interventi, anomalie e firme convivono in un unico fascicolo.
2. **Niente dato perso.** Le commesse rifiutate dal cliente vengono **archivate**, non cancellate; in qualsiasi momento si possono ripristinare con stato e file invariati.

3. **Sicurezza by default.** Autenticazione obbligatoria su ogni endpoint; password hashate; gate documentale lato server.
 4. **Operatività prima dell'eleganza.** Pagine come Calendario e Board mostrano in faccia all'utente le informazioni che gli servono (nome, cognome, indirizzo lavoro, telefono cliccabile).
-

2. Architettura tecnica (sintesi)

- **Frontend.** React 19 + Vite 7 + Wouter (routing) + tRPC 11 (client) + React Query (caching) + Tailwind 4 + shadcn/ui + lucide-react + sonner (toast) + jsPDF/jspdf-autotable. Le pagine sono caricate per rotta con `React.lazy`; i vendor sono separati per React, UI, dati e grafici.
 - **Backend.** Node + Express + tRPC 11. Persistenza prevalente in `kv_store` (Postgres JSONB) tramite `persistedStore`, con `save` debounce, `retry` su errori transienti e `recovery` in background. Le Comunicazioni usano una tabella PostgreSQL dedicata.
 - **Autenticazione.** Locale via email/password con JWT firmato (jose, HS256, TTL 7 giorni) + cookie `httpOnly`. Sessione server-side cacheata in memoria con `eviction` periodica.
 - **Sicurezza.** Tutti gli endpoint business sono `protectedProcedure` (utente loggato obbligatorio); le mutazioni su `utenti` e l'intero router `backup / fattureInCloud` sono `adminProcedure` (ruolo di direzione). Header `X-Content-Type-Options`, `X-Frame-Options=SAMEORIGIN`, `Referrer-Policy`, `HSTS` in produzione. Upload con `allowlist mimeType` + validazione dei byte reali. La rotta binaria per i file grandi verifica `same-origin` e sessione prima di leggere il body e ammette un solo upload concorrente per processo. CSRF `same-origin` check su `/api/trpc`. `trust proxy` abilitato (deploy dietro Railway).
 - **Worker e scheduler interni.** Backup notturno Google Drive (00:00 Europe/Rome, `setTimeout` riarmato), sync Fatture in Cloud (ogni 6 h quando abilitato), promemoria personali (giro immediato e poi ogni 15 s), poller IMAP (ogni 5 minuti, più watcher IDLE) e riconciliazione Centro Azioni (ogni 60 s, `debounce` 750 ms, primo giro circa 5 s dopo il bootstrap).
 - **PDF.** jsPDF + jspdf-autotable sia client-side (preventivatori, scheda cliente) sia server-side (scheda cliente nel backup).
 - **Storage file.** Driver `local` o `S3-compatible/R2`. I record conservano `storageKey` + checksum SHA-256; `dataBase64` resta supportato per i record legacy e come fallback in scrittura dei soli file fino a 10 MB. L'upload manuale nel fascicolo commessa accetta fino a 250 MB; allegati importati da comunicazioni/FiC e allegati ticket restano a 10 MB.
 - **Tars v2 con confini deterministici.** Il runtime server esiste (§54), ma `match`, regole, state machine, permessi, importi e scadenze restano deterministici. Il modello non può aggirare servizi di dominio, sede, capability, gate, audit o budget governor.
-

3. Autenticazione e sicurezza

3.1 Login

- Schermata `LoginPage` con email + password.
- Login solo per utenti `attivo === true`.
- Confronto password tramite `verifyPassword` (scrypt) — accetta anche password legacy in chiaro durante una migrazione trasparente.

- **Rate limit per email:** dopo 5 tentativi falliti in 15 minuti l'account è bloccato fino allo scadere della finestra. Login riuscito → reset del contatore.
- Messaggio di errore generico ("Email o password non validi") per non rivelare se l'utente esiste.

3.2 Sessione

- JWT firmato HS256, claim: `sub` (id utente), `email`, `name`, `role`, `ruolo`, `ruoli`. Esp. 7 giorni.
- Cookie `httpOnly`, `secure` su HTTPS, `sameSite` `none` su https / `lax` su http locale.
- Cache server-side `Map<token, { user, expMs }>` con eviction lazy e sweep orario (`setInterval(...).unref()`).
- **Logout** cancella sia il cookie sia l'entry della cache server.

3.3 Hashing password

- Algoritmo: **scrypt** (modulo `crypto` di Node, nessuna dipendenza esterna).
- Formato di archiviazione: `scrypt$<saltHex>$<hashHex>` (salt 16 byte, key 64 byte).
- `verifyPassword` è constant-time tramite `timingSafeEqual`.
- Le password create/aggiornate dalla UI sono sempre hashate.
- Le password nuove devono avere almeno 12 caratteri.
- Le password legacy in chiaro residue nel DB vengono **automaticamente upgrade a hash al primo boot successivo** all'aggiornamento (vedi `utenti.onLoad`).
- Su store utenti vuoto viene creato un solo amministratore da `BOOTSTRAP_ADMIN_*`; in produzione `BOOTSTRAP_ADMIN_PASSWORD` è obbligatoria. Non esistono più password seed fisse nel codice corrente.

3.4 Segreto JWT

- In **produzione** la variabile d'ambiente `JWT_SECRET` è OBBLIGATORIA: in sua assenza il server fa throw all'avvio.
- In dev/test, in mancanza di `JWT_SECRET`, viene usato un fallback hard-coded; **non valido per produzione**.

3.5 Header di sicurezza

Su ogni risposta HTTP:

- `X-Content-Type-Options: nosniff`
- `X-Frame-Options: SAMEORIGIN`
- `Referrer-Policy: strict-origin-when-cross-origin`
- `X-DNS-Prefetch-Control: off`
- `X-Permitted-Cross-Domain-Policies: none`
- In produzione: `Strict-Transport-Security: max-age=15552000; includeSubDomains`
- CSP **non** impostata di default (richiede tuning con Vite + Maps proxy + blob preview) — tracciata come follow-up.

3.6 Esposizione API

- Endpoint tRPC montati su `/api/trpc`.

- Tutti i router business utilizzano `protectedProcedure`. Le sole eccezioni pubbliche sono: `auth.me`, `auth.login`, `auth.logout`, `system.health`.
- `utenti.create`, `utenti.update`, `utenti.delete` richiedono `adminProcedure` (= ruolo `direzione`).

3.7 Protezioni aggiuntive (v4)

- **Hash versionato.** Formato `scrypt$<N>$<saltHex>$<hashHex>` con `N=32768`; verifica retro-compatibile con il legacy `scrypt$salt$hash` (`N=16384`) e con password in chiaro (upgrade automatico al boot).
- **CSRF.** Su `/api/trpc` le richieste con header `Origin` diverso dall'`Host` vengono rifiutate (403). Richieste server-to-server senza `Origin` ammesse.
- **SSRF guard (calendari esterni).** Gli URL iCal inseriti dagli operatori sono validati (solo `https`; vietati `localhost/`, `.local/`, `.internal/`, IP privati RFC1918/link-local/IPv6 raw) sia all'inserimento sia a ogni fetch.
- **Segreti mascherati in UI.** Gli indirizzi iCal privati (bearer secret) e i token Fatture in Cloud sono ritornati mascherati dalle `list/status` API.
- **Isolamento sede.** Ogni query/mutation è vincolata alla sede attiva (`ctx.sedeId`); guardie `assertSedeScope` su tutte le entità (vedi §34).

3.8 Operatività residua a carico del titolare (non risolvibile via codice)

- Rotazione manuale delle vecchie password seed eventualmente ancora attive e revoca dei token GitHub non riconosciuti.
- Decisione e finestra coordinata per pulire lo storico Git (BFG / `git filter-repo` + `force-push`). Il codice corrente non contiene più password seed fisse, ma la cronologia resta immutata.

4. Ruoli, permessi e gating

4.1 Set di ruoli

- `direzione` — ammessi accesso completo + gestione utenti + sezioni gated (Squadre, Garanzie, Fornitori, Utenti, Integrazioni avanzate).
- `amministrazione` — focus su fatturazione, finiture, saldo.
- `commerciale` — gestione clienti e commesse commerciali.
- `tecnico_rilievi` — rilievi e misure.
- `squadra_posa` — esecuzione in cantiere.
- `post_vendita` — ticket, reclami, rifacimenti.
- `ordini` — ordini fornitori.

Ogni utente ha `ruoli: string[]` (1–3 valori). Il campo legacy `ruolo` continua a contenere il ruolo primario per retro-compatibilità.

4.2 Mapping `role` legacy

- `role = "admin"` quando in `ruoli` è presente `direzione`. Altrimenti `user`.

4.3 Gating

- **Lato server.** `protectedProcedure` su tutto il business; `adminProcedure` su mutazioni `utenti` e `system.notifyOwner`.
- **Lato client.** Componente `RequireDirezione` su rotte `Garanzie`, `Squadre`, `Fornitori`, `Utenti`. Le voci di sidebar corrispondenti sono filtrate con il flag `direzioneOnly`.

4.4 Capability di contratto, computo e fattura

Aggiunte in `server/authz/capabilities.ts` dai piani 1 e 2 (§55, §56). La direzione ha per costruzione **tutte** le capability; gli altri ruoli le ricevono così, salvo override individuale:

Capability	Chi ce l'ha	A cosa serve
<code>contratto.read</code>	tutti i ruoli (condivisa)	leggere contratto strutturato e righe: le misure servono a chi rileva e a chi posa
<code>contratto.manage</code>	amministrazione, commerciale, direzione	creare e modificare il contratto strutturato
<code>computo.run</code>	amministrazione, commerciale, direzione	eseguire il computo dei limiti
<code>tariffe.manage</code>	solo direzione	pannello del catalogo DEI in Impostazioni (oggi in sola lettura)
<code>fattura.read</code>	amministrazione, commerciale, direzione	vedere bozze, fatture emesse e stati SdI
<code>fattura.draft</code>	amministrazione, direzione	creare e modificare la bozza, spegnere lo scavalco dei limiti
<code>fattura.emit</code>	amministrazione, direzione	emettere e attivare lo scavalco dei limiti (seconda autorizzazione, §56.4)
<code>fattura.credit_note</code>	amministrazione, direzione	nota di credito totale o parziale

Il client non duplica queste stringhe (`client/src/lib/roles.ts` resta solo helper di ruolo): legge il set effettivo da `trpc.permessi.mie`. Il confine resta il server — ogni handler verifica capability, sede e interruttore.

5. Anagrafica Cliente

5.1 Tipi cliente

`privato` | `azienda` | `condominio` | `ente_pubblico`. Icone in UI: `User`, `Building2`, `Home`, `Landmark`.

5.2 Campi del Cliente

Anagrafica:

- **Tipo** (vedi 5.1; default `privato`) — è il **primo campo del form**: la sua scelta cambia i campi successivi.
- Se `tipo === "privato"` : **Cognome e Nome** (entrambi obbligatori).
- Se `tipo ∈ {azienda, condominio, ente_pubblico}` : un solo campo **Ragione sociale** (obbligatorio), archiviato nel campo `cognome` ; `nome` viene valorizzato a spazio singolo. Stessa convenzione usata dalla sincronizzazione Fatture in Cloud (§40) e dalla migrazione 2026 (§43), così le denominazioni restano indivise.
- **Codice fiscale, partita IVA.**

Doppio indirizzo:

- **Indirizzo di residenza** (`indirizzo` , `citta` , `cap`) — usato dall'amministrazione per la fatturazione. Per i clienti non privati l'etichetta diventa «**Sede legale (fatturazione)**» ovunque: form di creazione/modifica, badge nella scheda cliente e scheda PDF (§42).
- **Indirizzo dei lavori** (`indirizzoLavoro` , `cittaLavoro` , `capLavoro`) — usato dalle commesse per cantiere, calendario, mappe.
- In fase di creazione/modifica il form propone uno **switch "stesso della residenza"** che copia automaticamente i campi residenza → lavoro al salvataggio.
- La scheda cliente espone entrambi gli indirizzi con badge distintivo "Residenza" / "Lavoro".

Contatti: **telefono, email.**

Dati fiscali e amministrativi:

- **Detrazione** (switch). Quando attivata `RICHIEDE tipoDetrazione` .
- **Tipo detrazione**: `ecobonus` | `ristrutturazione` (obbligatorio se `detrazione === true`).
- **Finanziamento** (switch).
- **Pratica edilizia**: `nessuna` | `cil` | `cila` | `scia` .

Referenti (lista multipla):

- Campi per referente: `nome`, `ruolo` (`cliente_finale` | `architetto` | `direttore_lavori` | `amministratore` | `altro`), `telefono`, `email`.

Operativi:

- **Note libere.**
- **Assegnato a** (utente). Default: utente corrente; eredita su nuove commesse linkate al cliente.

5.3 Comportamenti del Cliente

- Lista clienti con ricerca testuale, filtro per tipo, filtro "solo le mie".
- Scheda cliente con tabs: **Commesse, Interventi, Ticket, Garanzie** — ognuno ha un pulsante **+** per creare l'entità collegata.
- **Cascade su update.** Modificando nome/cognome o un campo di contatto/indirizzo del cliente, il server propaga il valore a tutte le commesse linkate (solo dove il valore della commessa coincideva con il valore precedente del cliente, così le override manuali non vengono sovrascritte).
- Eliminazione cliente disponibile (con conferma) ma **SCONSIGLIATA** in presenza di commesse linkate.
- **Scheda PDF** stampabile dall'header (vedi §42) e bottone **WhatsApp** accanto al telefono (vedi §41).
- Ogni cliente appartiene a una sede (`sedeId` , vedi §34).

6. Commesse

6.1 Identificazione

- **Codice automatico** `COM-{ANNO}-{N}` con N progressivo a 3 cifre (es. `COM-2026-001`).
- Lo ZeroPadding è coerente per anno; al cambio anno il contatore può ripartire da 1.

6.2 Campi

- **clientId** (FK), **cliente** (display name denormalizzato, sincronizzato da `cliente.update`).
- **indirizzo**, **citta**, **telefono**, **email** — inizialmente derivati dall' `indirizzoLavoro` del cliente; sovrascrivibili a livello di commessa.
- **stato** — vedi 7.1.
- **priorita**: `bassa` | `media` | `alta` | `urgente`.
- **squadraId** (FK opzionale).
- **dataApertura** (auto).
- **consegnaIndicativa**: enum `"30"` | `"60"` | `"90"` (giorni dal preventivo).
- **dataConsegnaIndicativa**: data libera ISO. **Mutuamente esclusiva** con `consegnaIndicativa`: salvando una delle due l'altra viene azzerata.
- **dataConsegnaConfermata**: data definitiva impostata al passaggio in `produzione`.
- **dataChiusura**: impostata automaticamente al passaggio in `archiviata` (chiusura del flusso).
- **note** libere.
- **importoTotale**: totale pattuito (€, opzionale). **importoIncassato**: derivato — somma del registro `pagamenti` (vedi §37); non modificabile direttamente dalla UI.
- **pagamenti**: registro acconti embedded (vedi §37). Escluso da `commesse.list` come `prodotti`.
- **sedId**: sede proprietaria (vedi §34).
- **prodotti**: array di prodotti della commessa (nome, tipologia/materiale, quantità, dimensioni, note) — *di cosa si tratta*, vedi §44. NON viene incluso nella risposta di `commesse.list` per alleggerire i payload; viene ritornato da `commesse.byId`. La lista riceve invece la sintesi `prodottiSintesi: [{nome, quantita}]`.
- **costi**: registro dei costi fornitore embedded, base del calcolo del margine (vedi §45).
- **costoPosaStimato**: stima manuale del costo di posa (€, opzionale). Scrivibile solo da direzione o amministrazione.
- **squadraId**: squadra di posa assegnata alla commessa (vedi §46).
- **dataApertura**: data di creazione in formato `YYYY-MM-DD`, mostrata come "Creata il" nella scheda e in colonna nella lista.
- **assegnatoA** (FK utente). Modificabile dalla scheda commessa.
- **createdBy** (FK utente).
- **createdAt**, **updatedAt**.
- **archivedAt**: timestamp ISO; `null` finché la commessa non è in soft-archive (vedi §22).

6.3 Consegna indicativa — selezione

La UI offre quattro opzioni nel select **Consegna indicativa**:

1. `+30 giorni`
2. `+60 giorni (default)`

3. +90 giorni

4. **Data da calendario...** → mostra un date picker, popola `dataConsegnaIndicativa` e svuota `consegnaIndicativa`.

L'header della commessa mostra in ordine di priorità:

1. `dataConsegnaConfermata` (etichetta "Data consegna prevista").
2. `dataConsegnaIndicativa` (etichetta "Consegna indicativa", formato `gg/mm/aaaa`).
3. `consegnaIndicativa` (etichetta "Consegna indicativa: +N giorni").

6.4 Trigger Produzione

Quando una commessa entra nello stato `produzione`:

1. Sulla scheda compare un banner ambra "Commessa in produzione" con il pulsante "Aggiorna data consegna".
2. Al click si apre un dialog con date picker per la **Data di Consegna Prevista** definitiva.
3. Il salvataggio popola `dataConsegnaConfermata` (visibile in header, card Kanban, lista in Dashboard).
4. Finché `dataConsegnaConfermata` è vuota, la card del Kanban resta evidenziata con anello ambra e contatore "Consegne da confermare" nel KPI bar.

6.5 Modifica

- Dialog "**Modifica commessa**" consente di modificare contemporaneamente:
 - Anagrafica cliente collegata (nome, cognome, CF, P.IVA, CAP, telefono, email, indirizzo, città) — sincronizzata via `clienti.update` con cascade.
 - Priorità.
 - **Utente assegnato** (SearchSelect sugli utenti, opzione "Non assegnata").
 - Consegna indicativa (preset o data libera).
 - Note.
- Nell'header sono visibili pill: indirizzo, telefono, email, **Assegnata a: Nome** (lookup utente).

6.6 Eliminazione vs Archiviazione

- **Archivia** — operazione consigliata se il cliente non procede. Non distrugge nulla (vedi §22).
- **Elimina** — operazione distruttiva. Conferma esplicita. Dovrebbe essere usata solo per errori di inserimento.

6.7 Lista commesse

- Filtri: search testuale (codice, cliente, città), stato, clienteld, assegnatoA, scope `archived = exclude | only | all` (default `exclude`).
 - Ordinata per `createdAt` desc.
 - Risposta **non include** `prodotti` né `pagamenti` (ottimizzazione bandwidth/render). Include però `prodottiSintesi` (nome + quantità per riga) e `nPagamenti` (conteggio degli acconti), che alimentano rispettivamente la colonna Prodotti e la proposta della rata successiva nella pagina Pagamenti.
-

7. State machine delle commesse

7.1 Stati

Ordine canonico (STATI_COMMESSA):

1. preventivo
2. misure_esecutive
3. aggiornamento_contratto
4. fatture_pagamento
5. da_ordinare
6. produzione
7. ordini_ultimazione (*richiesta secondo acconto*)
8. attesa_posa
9. finiture_saldo
10. interventi_regolazioni
11. archiviata (*chiusura del flusso — distinta dal soft-archive di §24*)

7.2 Transizioni valide

- Transizioni **in avanti** consentite di **una posizione** (es. da_ordinare → produzione).
- Transizioni **all'indietro** consentite di **una posizione**.
- Salti multipli **vietati** lato server (validateTransizione).
- Stato archiviata raggiungibile solo dal predecessore interventi_regolazioni.

7.3 Soft-archive (orthogonal)

- archivedAt è ortogonale a stato . Una commessa può essere soft-archiviata in qualsiasi stato (vedi §22).
- Il soft-archive **non** modifica lo stato corrente.

7.4 Avanzamento via UI

- **Scheda commessa**: pulsante "Avanza" che propone lo stato successivo. Disabilitato se manca un documento richiesto (vedi §9), eccetto bypass con "Procedi comunque".
- **Board Kanban**: ogni card ha frecce **Indietro** / **Avanza**. Le frecce attraversano sempre la stessa state machine (controlli server identici).
- **Timeline ordine**: completare una milestone operativa avanza automaticamente la commessa nella relativa colonna del Board (§35.2). Salvataggio di data/note e riapertura di uno step non cambiano lo stato della commessa.

7.5 Stati e gate documentale (vedi anche §9)

- preventivo → necessita preventivo o contratto .
- misure_esecutive → misure .
- aggiornamento_contratto → contratto .
- fatture_pagamento → fattura .
- da_ordinare → conferma_ordine .

- `produzione` → nessun documento richiesto (gated da `dataConsegnaConfermata`).
- `ordini_ultimazione` → `saldo` o `fattura`.
- `attesa_posa` → `ddt_consegna`.
- `finiture_saldo` → `ddt_posa`.
- `interventi_regolazioni` → `ddt_finale`.
- `archiviata` → nessun documento richiesto.

8. Documenti commessa (Preventivi/Contratti)

8.1 Tipi documento

`preventivo`, `contratto`, `misure`, `fattura`, `conferma_ordine`, `ddt_consegna`, `ddt_posa`, `ddt_finale`, `saldo`, `foto`, `documento_identita`, `visura`, `planimetria`, `certificazione`, `altro`.

Il tipo `ordine` è stato accorpato in `conferma_ordine` il 03/09/2026: erano due voci per lo stesso foglio e il gate già accettava indifferentemente l'una o l'altra, mostrando però due pastiglie di cui una arancione. I documenti già archiviati come `ordine` vengono riportati a `conferma_ordine` al bootstrap.

I primi dieci hanno un ruolo nel doc gate (§9). Gli ultimi quattro sono stati aggiunti il 26/08/2026 perché una commessa raccoglie anche documenti che non fanno avanzare niente — un documento d'identità, una visura, una planimetria — e classificarli tutti come `altro` li rendeva indistinguibili al momento di ritrovarli. L'elenco è unico: `DOC_TIPI` alimenta schema server, dropdown UI e schema dei dati esposti agli operatori.

8.2 Storage e schema

- Persistito in `preventivi_documenti` (kv_store JSONB).
- Per ogni documento: `id`, `sedeId`, `commessaId`, `nome`, `tipo`, `mimeType`, `size`, `storageKey?`, `checksum?`, `dataBase64?`, `note`, `statoAtUpload`, `createdBy`, `createdAt`.
- `storageKey` è la fonte canonica dopo la migrazione; `dataBase64` resta per record legacy/fallback. La lista `byCommessa` non restituisce i byte e usa `hasData`; `byId` rilegge dallo storage e ricostruisce il payload atteso dal client.

8.3 Upload — controlli

- **MimeType allowlist** (stored XSS hardening):
 - `application/pdf`
 - `image/jpeg`, `image/png`, `image/gif`, `image/webp`, `image/heic`, `image/heif`
 - `application/msword`, `application/vnd.openxmlformats-officedocument.wordprocessingml.document`
 - `application/vnd.ms-excel`, `application/vnd.openxmlformats-officedocument.spreadsheetml.sheet`
 - upload manuale commessa: `video/mp4`, `video/quicktime`, `video/webm`
 - **Esclusi esplicitamente** `text/html` e `image/svg+xml`.
- **Dimensione**: validata sui byte reali, senza fidarsi del campo `size` lato client. Cap 250 MB per l'upload manuale commessa; 10 MB per allegati importati da comunicazioni/FiC e allegati ticket.

- L'upload manuale usa un endpoint binario autenticato (`POST /api/commesse/:commessaId/documenti/file`): il browser non crea copie base64/JSON del file e il parser maggiorato non è esposto agli utenti anonimi. Gli endpoint JSON restano al limite di 50 MB.
- Se lo storage fallisce, il fallback `dataBase64` resta ammesso soltanto fino a 10 MB: un file più grande fallisce visibilmente e non viene inserito nel JSONB.
- Il `size` archiviato è quello calcolato dal server.

8.4 Auto-rename e rinomina

- Se `keepNome === false`, il file in upload viene rinominato in `{Tipo} {cliente}.{ext}` (es. "Misure esecutive Mario Rossi.pdf").
- Se `keepNome === true` (usato dai preventivatori), il nome viene preservato e solo deduplicato.
- Dal 26/08/2026 l'auto-rename **non si applica** ai tipi `documento_identita`, `visura`, `planimetria`, `certificazione`, `foto` e `altro`: di quei documenti una commessa ne contiene più d'uno (intestario, coniuge, delegato) e schiacciarli sullo stesso nome produceva (2) e (3) indistinguibili. Conservano il nome originale.
- Disambiguazione automatica: se il nome esiste già per la stessa commessa, viene appeso (2), (3), ecc.
- La scheda commessa espone **Rinomina**: cambia nome libero e tipo di un documento già caricato (`preventiviContratti.update`). Il tipo conta per il doc gate, quindi correggere una classificazione sbagliata non richiede più di ricaricare il file.

8.5 Anteprima e download

- Anteprima inline in `<iframe>` per PDF, in `<img>` con zoom/rotate per immagini o in `<video controls>` per i video supportati. I byte arrivano dall'endpoint autenticato `GET /api/documenti/:id/file`, che supporta richieste HTTP Range e download senza ricodifica base64.
- Download via `<a download>`.
- Invio email via `mailto:` con corpo precompilato (no upload server-side dell'allegato).

8.6 Eliminazione

- Soft delete NON previsto. La cancellazione è definitiva.

8.7 Allegati ticket

- Il router `ticketAllegati` mantiene `storageKey/checksum`, fallback base64, allowlist documenti/immagini e cap 10 MB; l'estensione a 250 MB e ai video riguarda soltanto l'upload manuale del fascicolo commessa.
 - Cancellando un ticket si cancellano in cascata i suoi allegati (`deleteAllegatiByTicket`).
-

9. Doc gate (gate documentale)

9.1 Regola

- Una transizione **in avanti** verifica che esista almeno un documento con uno dei tipi richiesti dallo stato CORRENTE (`REQUIRED_DOC_TIPI_PER_STATO`).
- Conta solo se il documento è stato caricato **mentre la commessa era in quello stato** (campo `statoAtUpload`), così un preventivo non può soddisfare un gate diverso.
- Per i documenti legacy senza `statoAtUpload` , fallback permissivo: il tipo è sufficiente.

9.2 Stati daily reminder

Per gli stati `aggiornamento_contratto` , `fatture_pagamento` , `da_ordinare` viene generata anche una notifica giornaliera (vedi §25.2) anche oltre la soglia di priorità.

9.3 Bypass "Procedi comunque"

- **Server.** `commesse.update` accetta un flag `force: boolean` . Se assente, il gate è bloccante; in caso di mancanza documenti il server lancia un errore con prefisso `DOC_GATE_BLOCKED:` e messaggio human-readable ("Non è stato caricato il file Procedere comunque?").
- **Client (scheda commessa).** Il pulsante "Avanza" non è più disabilitato dal gate. Al click, se il gate è violato, mostra un `ConfirmDialog` non-destructive con label "Procedi comunque". In conferma chiama `update({ stato, force: true })` .
- **Client (Kanban).** Onerror della mutation: se `err.message` inizia per `DOC_GATE_BLOCKED:` , lo stesso `ConfirmDialog` viene aperto e su conferma rilancia con `force: true` . Errori non-gate restano nel banner inline.
- **La state machine resta invariata.** `force` bypassa SOLO il gate documentale, non la shape delle transizioni.

9.4 Indicatore UI

Sotto l'header della commessa è presente una card con stato del gate:

- Verde se tutti i documenti richiesti sono presenti.
- Ambra se mancano: lista di tipi richiesti con badge "Caricato"/"Mancante" + bottone "Carica file" che preimposta il tipo nel dialog upload.

10. Aperture (Rilievo)

- Una **apertura** è un singolo serramento all'interno di una commessa.
- Campi: `commessaId` , `codice` , `descrizione` , `piano` , `locale` , `tipologia` (`finestra` | `porta-finestra` | `porta` | `scorrevole` | `fisso` | `altro`), `larghezza` , `altezza` , `profondita` , `materiale` , `colore` , `vetro` , `noteRilievo` , `criticitaAccesso` , `stato` .
- Stati apertura: `da_rilevare` → `rilevata` → `ordinata` → `consegnata` → `in_posa` → `posata` → `verificata` .
- Le aperture sono visibili nella scheda commessa e nella vista **RilievoDetail** dedicata.

- Una commessa **può** avere zero aperture (es. preventivo iniziale generico).

11. Board Kanban (/kanban)

11.1 Layout

Quattro **fasi**, ognuna con N colonne:

Fase	Colonne
Vendita	Preventivo, Misure Esecutive, Aggiornamento Contratto
Ordine & Produzione	Fatture / Pagamento, Da Ordinare, Produzione
Consegna & Posa	Richiesta Secondo Acconto, Attesa Posa
Chiusura	Finiture / Saldo, Interventi / Regolaz.

Lo stato `archiviata` non compare.

11.2 Card commessa

Mostra: codice, badge priorità, cliente, città, indicatore di consegna (data confermata o indicativa), pulsanti **Indietro** / **Avanza** con etichetta dello stato target. In più (v4):

- **Bordo sinistro 3 px** colorato per priorità (urgente rosso, alta ambra, media blu, bassa grigio).
- **Chip "fermo N gg"** quando la commessa non riceve update da ≥ 5 giorni (ambra) o ≥ 10 (rossa).
- **Blocco prodotti magazzino**: primi 2 prodotti con data consegna corta (✓ verde arrivato, rosso in ritardo) + "+N altri prodotti" (vedi §36).
- **Chip "Da saldare"** (rossa, senza importo) nelle fasi `attesa_posa` / `finiture_saldo` / `interventi_regolazioni` quando il residuo pagamenti è > 0 . Dal 28/08/2026 il Board non trasporta cifre: il chip usa il solo booleano `daSaldare` del server, e gli importi vivono nella scheda e in `/pagamenti`, dietro capability (§37.5).

11.2-bis Colonne con overflow

Ogni colonna mostra al massimo **5 card**; oltre compare il toggle tratteggiato "Mostra altre N" / "Mostra meno" per non forzare scroll infiniti.

11.3 Filtri

- Search per codice/cliente/città.
- Filtro priorità (Tutte/Urgente/Alta/Media/Bassa).
- Toggle "Nascondi vuote" / "Mostra vuote".
- Chip per fase (Tutte le fasi / per singola fase).
- Tutte le fasi possono essere collassate.

11.4 KPI bar

Quattro cards: **Attive**, **Urgenti**, **Alte**, **Consegne da confermare** (commesse in produzione senza dataConsegnaConfermata).

11.5 Doc gate sul Kanban

Le frecce **Avanza** seguono lo stesso doc gate. Errore `DOC_GATE_BLOCKED`: → ConfirmDialog "Procedi comunque". Errori non-gate → banner inline rosso.

11.6 Esclusioni

- Le commesse soft-archivate non appaiono.

12. Calendario / Planning (/planning)

12.1 Viste

- **Mese** (*default*) — griglia 6×7 (la sesta settimana è renderizzata solo se il mese vi sconfina); oggi evidenziato con pill primary; giorni fuori mese e weekend attenuati; chip evento pieni (colore per tipo, testo bianco) con ora + nome cliente; overflow "+N altri" apre la vista giorno.
- **Settimana** — 7 colonne lun-dom; weekend leggermente attenuati.
- **Giorno** — singola colonna. Ordine switcher: Mese · Settimana · Giorno.

12.2 Tipi di intervento

rilievo, posa, assistenza, altro. Colori dedicati per tipo.

12.3 Card intervento (joined info)

Ogni card mostra:

- Ora inizio – ora fine.
- Tipo intervento.
- **Nome + cognome** del cliente (lookup commessa → cliente).
- **Indirizzo** (fallback `intervento.indirizzo` → `commessa.indirizzo` → `cliente.indirizzoLavoro` → `cliente.indirizzo`).
- Note brevi.
- Stato (`pianificato` di default).

12.4 Dialog dettagli appuntamento

In modalità modifica, sopra al form, viene mostrato un blocco di sintesi joined:

- Codice commessa, stato, badge priorità.
- Nome cliente.
- Indirizzo cantiere.

- Telefono (link `tel:`).
- Email (link `mailto:`).
- Squadra assegnata.
- Pulsante "Apri commessa" → naviga alla scheda commessa.

12.5 Auto-fill

Quando si seleziona una commessa in dialog di creazione, l'indirizzo viene auto-popolato dalla commessa (solo se il campo è vuoto, per non sovrascrivere override manuali).

12.6 Drag & drop

- Un intervento può essere trascinato fra giorni nelle viste settimana e mese.
- L'update della data avviene via `interventi.update({ dataPianificata })` .

12.7 Stati intervento

`pianificato`, `in_corso`, `completato`, `sospeso`, `annullato` . L'avvio imposta `dataInizio` ; la chiusura imposta `dataFine` . Le entry `annullato` legacy sono **purgate** automaticamente al boot (cleanup nel `persistedStore.onLoad`).

12.8 Link entità

Un intervento può essere collegato a una delle entità: `commessa` , `ticket` , `reclamo` , `rifacimento` . Solo `commessa` è obbligatoria quando il `linkKind === "commessa"` .

12.9 Eventi Google (overlay read-only)

Gli eventi importati dai calendari Google (vedi §38.2) compaiono in tutte e tre le viste, ordinati per ora insieme agli appuntamenti CRM ma **in sola lettura**: stile distinto (badge GOOGLE + lucchetto, bordo sinistro nel colore della sorgente), nessun drag/edit/delete. Il click apre un dialog dettagli (data, orario/tutto il giorno, luogo, calendario di origine). Una legenda sopra la griglia elenca le sorgenti attive.

12.10 Card intervento (restyle v4)

Chip tipo pieno (POSA/RILIEVO/ASSISTENZA/ALTRO) su sfondo tinta + bordo sinistro 4 px nel colore del tipo; ora in mono grassetto. Nel dialog di modifica, accanto al telefono, è presente il bottone **WhatsApp** con messaggio di conferma appuntamento precompilato (vedi §41).

13. Ticket post-vendita (/ticket e contenuti di /reclami)

13.1 Modello

- Campi: `commessaId?`, `clienteId?`, `contatto?`, `aperturaId?`, `oggetto`, `descrizione`, `categoria`, `priorita`, `stato`, `solleciti[]`, `assegnatoA?`, `dataRisoluzione?`, `esitoIntervento?`, `apertoBy?` .

- Categorie: difetto_prodotto, difetto_posa, regolazione, sostituzione, garanzia, altro.
- Priorità: come per le commesse.
- apertoBy registra l'utente che apre il ticket (usato dai permessi di eliminazione, §13.6).

13.2 Ticket senza commessa

Una chiamata di assistenza arriva spesso **prima** che esista una commessa, e a volte prima ancora che il cliente sia a sistema. Perciò **solo l'oggetto è obbligatorio**; l'intestazione del ticket può essere data, in ordine di precisione, da:

1. **commessa** collegata (commessaId) — caso classico;
2. **cliente** già in anagrafica (clienteId), senza commessa;
3. **contatto** in testo libero (contatto), es. "Sig. Verdi 3401234567".

La UI mostra il primo disponibile, con badge "Senza commessa" quando manca, e un grigio "Senza cliente" se non c'è nulla. Il collegamento **può essere fatto in seguito**: il dialog di modifica espone commessa/cliente/contatto, e agganciando una commessa i campi più deboli vengono azzerati per non lasciare tre risposte in conflitto.

13.3 State machine ticket

aperto → assegnato → in_lavorazione → chiuso. Disponibile **rollback** di una posizione (clear dataRisoluzione se si esce da chiuso). Da aperto il rollback è rifiutato.

Lo stato **risolto è stato ritirato** (§13.7): fra risolto e chiuso non cambiava nulla nella pratica. Il backfill al boot converte i record esistenti; updateStato accetta ancora il valore legacy dai client vecchi e lo piega su chiuso. Stesso collasso applicato ai **reclami** (§14.1).

13.4 Solleciti

Registro solleciti[] sul ticket: { data, nota?, utenteId }. La procedure ticket.sollecita aggiunge una voce; è rifiutata sui ticket chiusi ("riapirlo prima di sollecitare"). In lista compare un badge ambra "**N solleciti · ultimo gg/mm**", così si vede a colpo d'occhio da quanto un ticket è fermo nonostante i solleciti. Il dialog mostra lo storico completo.

13.5 Interventi pianificati dal ticket

Bottone **Pianifica** sul ticket: data, ora inizio/fine, squadra, note. Crea un intervento di tipo assistenza già collegato (ticketId , commessaId , indirizzo ereditato dalla commessa) che appare nel Calendario e sotto il ticket con data, ora, squadra e stato — con "**Senza squadra**" evidenziato in colore warning quando manca.

13.6 Eliminazione

Possono eliminare: la **direzione, chi ha aperto** il ticket (apertoBy), o il **proprietario della commessa** collegata. Se la commessa è stata cancellata si ricade sul solo controllo direzione — diversamente il ticket sarebbe indistruttibile. Cancellando un ticket si cancellano in cascata i suoi allegati.

13.7 Ricerca e presentazione

- **Ricerca** su: nome cliente (da commessa o da `clienteId`), codice e città della commessa, oggetto, descrizione, id `TK-000N`, categoria, esito intervento e **note dei solleciti**.
- Il filtro per stato è **lato client** su tutta la lista di sede: i chip riportano i conteggi reali e la ricerca attraversa anche gli stati non selezionati (cercare un cliente non deve fallire perché il suo ticket è chiuso).
- **Card**: nome cliente in grassetto in testa, poi codice commessa (cliccabile) e `TK-000N`, quindi l'oggetto, le etichette, la descrizione in blocco espandibile (spesso è la nota importata da To Do), gli interventi collegati e infine il footer con meta e azioni. Barra colorata a sinistra: rossa se aperto ad alta priorità, verde se chiuso, neutra altrimenti.
- Empty state distinti fra "nessun ticket" e "nessun risultato per «...»", il secondo con azione **Azzera i filtri**.

13.8 Allegati ticket

Vedi §8.7.

14. Reclami e Rifacimenti (/reclami)

Pagina unificata che gestisce due entità correlate ma distinte.

14.1 Reclamo

- Campi: `commessaId`, `clienteNome`, `descrizione`, `responsabile?`, `stato`, `dataApertura`, `dataRisoluzione?`, `soluzione?`.
- Stati: `aperto`, `in_gestione`, `chiuso` (lo stato `risolto` è ritirato come per i ticket, §13.3; i record esistenti sono convertiti al boot).

14.2 Rifacimento

- Campi: `commessaId`, `clienteNome`, `descrizione`, `elemento`, `fornitoreCoinvolto?`, `ordineRifacimentoId?`, `costoStimato?`, `responsabilita (interna|esterna)`, `responsabile?`, `stato`, `dataApertura`, `dataChiusura?`.
- Stati: `aperto`, `in_gestione`, `in_produzione`, `completato`, `chiuso`.

15. Verbali (/verbale/:interventoId + VerbaleChiusura)

- Tipo: `chiusura_lavori` | `sopralluogo` | `consegna` (default `chiusura_lavori`).
 - Campi: `interventoId`, `commessaId`, `tipo`, `data`, `noteCliente?`, `noteTecnico?`, `firmaClienteData?`, `firmaTecnicoData?`, `firmaCliente`, `firmaTecnico`, `apertureCompletate`, `apertureTotali`, `anomalieRiscontrate`, `stato (bozza|firmato)`.
 - Lo stato passa a `firmato` quando entrambe le firme sono presenti.
-

16. Anomalie

- Campi: `commessaId`, `aperturaId?`, `interventoId?`, `categoria`, `priorita`, `descrizione`, `risoluzione?`, `stato`, `segnalataBy?`, `risoltaBy?`, `risoltaAt?`.
 - Categorie: `materiale_difettoso`, `misura_errata`, `danno_trasporto`, `difetto_posa`, `problema_accessorio`, `non_conformita`, `altro`.
 - Priorità: `bassa`, `media`, `alta`, `critica`.
 - Stati: `aperta`, `in_gestione`, `risolta`, `chiusa`. Endpoint dedicato `resolve` imposta `stato=risolta`, `risoluzione`, `risoltaAt`.
-

17. Garanzie (`/garanzie` , direzione-only)

- Tipi: `prodotto`, `posa`, `accessorio`, `vetro`, `altro`.
 - Campi: `commessaId`, `aperturaId?`, `tipo`, `descrizione`, `fornitore?`, `dataInizio`, `durataMesi`, `dataScadenza`, `stato`, `documentoRif?`, `note?`.
 - `dataScadenza` calcolata server-side da `dataInizio` + `durataMesi`.
 - Stati: `attiva`, `scaduta`, `sospesa`, `revocata`.
 - Statistiche: `totale`, `attive`, `in scadenza` (entro 90 giorni), `scadute`.
-

18. Squadre (`/squadre` , direzione-only)

- Campi: `nome`, `caposquadra?`, `telefono?`, `note?`, `attiva`.
 - Le squadre attive appaiono nei dropdown della scheda commessa, dialog intervento, calendario.
 - L'inattivazione (toggle `attiva`) preserva storico ma rimuove la squadra dai picker.
-

19. Fornitori (`/fornitori` , direzione-only)

19.1 Anagrafica fornitore

- Campi: `ragioneSociale`, `partitaIva`, `indirizzo?`, `citta?`, `telefono?`, `email?`, `categoria`, `referenteCommerciale?`, `scontistica?`, `note?`, `attivo`.
- Categorie: `pvc`, `alluminio`, `vetro`, `ferramenta`, `persiane`, `blindati`, `accessori`, `guarnizioni`, `altro`.

19.2 Ordini fornitore

- Campi: `fornitoreId`, `commessaId`, `codiceOrdine`, `stato`, `dataOrdine`, `dataConsegnaPrevista?`, `dataConsegnaEffettiva?`, `righe[]`, `noteOrdine?`, `noteRicevimento?`, `importoTotale`.
- Stati: `bozza`, `inviato`, `confermato`, `in_transito`, `ricevuto_parziale`, `ricevuto`, `contestato`.

- Riga ordine: `descrizione, codiceArticolo?, quantita, quantitaRicevuta, unitaMisura, prezzoUnitario?, lotto?, conforme?, noteDifetto?`.
- L'update di stato accetta `righeAggornate` per registrare la ricezione parziale.

19.3 Listini fornitore

- Campi: `fornitoreId, nome, versione, dataValidita, nomeFile, tipo (pdf|excel|altro), note?`.
- I listini sono solo metadati: il file effettivo viene gestito altrove (cartella condivisa o sezione documenti separata).

19.4 Analisi delle conferme d'ordine (PDF) — 28/08/2026

Prima slice della Document Intelligence (visione completa in §54.6; piano in `docs/reports/d7-document-intelligence-piano.md`). Dalla scheda ordine (direzione) il pannello «Conferma d'ordine (PDF)» analizza un documento del fascicolo della commessa dell'ordine:

- **Pipeline:** registro parser estendibile; oggi un solo parser, `pdf-testo-nativo` (`unpdf`, testo per pagina). Il run è persistito in `documenti_analisi` con impronta SHA-256 dei byte e versioni di parser/estrattore/confronto: lo stesso file non produce due run (idempotente); la rielaborazione esplicita conserva lo storico.
- **Estrazione deterministica** con evidenza obbligatoria per ogni valore (pagina, frammento, metodo, confidenza, eventuali letture alternative): riferimento al nostro ordine, codici commessa citati, fornitore, numero e data della conferma, date/settimane di consegna, totale documento, riscontro delle righe per codice articolo (best-effort, confidenza bassa). Nessun modello: il testo del documento è un dato inerte, e un prompt injection nel PDF resta un frammento di evidenza.
- **Confronto con l'ordine:** differenze tipizzate e ordinate per gravità — consegna diversa o non dichiarata, totale diverso (tolleranza 50 centesimi), riferimento ordine assente, commessa incoerente, riga non citata, quantità diversa.
- **Nessuna scrittura su dati autorevoli:** l'analisi non tocca commesse, ordini, righe, prezzi, date o stati. L'operatore legge le differenze e aggiorna i dati dalle schede.
- **Scansioni (aggiornato dalla quarta slice, 29/08/2026):** un PDF senza testo nativo passa dal **fallback OCR locale** (v. sotto). Se l'OCR riconosce testo, il run diventa `analizzata` con parser `pdf-ocr`, confidenze dichiarate e — sotto soglia — marcatura «DA VERIFICARE». Se l'OCR non è disponibile, fallisce, va in timeout o non riconosce nulla, il documento resta `scansione_senza_testo` con il MOTIVO esplicito: il contenuto non compreso non viene mai presentato come analizzato (campi e confronto compaiono solo con stato `analizzata`). File corrotti, cifrati o di formato non supportato producono gli stati espliciti `illeggibile` / `non_supportato` con il motivo.

Collegamento assistito documento → ordine (28/08/2026, seconda slice). Da «File e documenti» della scheda commessa, l'azione «Collega a un ordine fornitore» su un PDF apre il dialog dei candidati:

- i candidati sono generati **deterministicamente** su tutti gli ordini della sede, con un punteggio spiegabile per segnali in ordine di forza — codice d'ordine citato (100), codice commessa (60), fornitore (40), codici articolo (15 l'uno, massimo 45), data di consegna coincidente (15), totale coincidente (15) — ognuno con la sua evidenza (pagina e frammento). Il segnale sul totale è calcolato SOLO per chi ha `economia.read`: la sua presenza è un oracolo di uguaglianza sugli importi, e per gli altri ruoli non esiste (v5.10);

- l'esito è uno di quattro stati espliciti: `certa` (un solo ordine citato per codice), `candidata` (plausibile, da confermare), `ambigua` (più ordini equivalenti: MAI un collegamento automatico), `assente`. Anche con `certa` il collegamento nasce SOLO dalla conferma umana;
- il collegamento è un dato separato (`documenti_collegamenti_ordini`) che non altera documento, ordine o commessa; è idempotente, rileva i duplicati per impronta, e porta un audit append-only di conferme, rifiuti e annullamenti (utente, momento, motivo). La correzione è annulla + riconferma; un candidato rifiutato non viene più proposto come certo finché un umano non lo riconferma esplicitamente;
- autorizzazione via capability `commessa.manage_documents` decisa dal motore in ogni `policyMode` (direzione dal ruolo, gli altri sulla commessa posseduta o assegnata, override individuali inclusi): nessun ruolo hardcoded. Sedi isolate: documento e ordine di un'altra sede sono `NOT_FOUND` ;
- un documento collegato può essere analizzato dall'ordine (§19.4) anche se archiviato nel fascicolo di un'altra commessa: la decisione umana prevale sulla posizione del file, che comunque non viene spostato.

Approval gateway delle proposte documentali (29/08/2026, terza slice). Le differenze rilevate dall'analisi possono diventare **proposte di azione**, mai effetti diretti. Il gateway (`server/proposte/gateway.ts`) è una fondazione generale e tipizzata, separata dai router business, la stessa su cui poggerà il futuro agente:

- **registro chiuso dei tipi di azione:** l'unico oggi è `ordine_fornitore.aggiorna_data_consegna` (la data di consegna prevista dell'ordine fornitore). Niente prezzi, quantità, righe, stati o configurazioni;
- ogni proposta porta documento, evidenza (pagina e frammento), valore corrente al momento della generazione, valore proposto, motivazione, versioni dei componenti, autore `sistema`, sede/commessa/ordine, chiave d'idempotenza, scadenza (30 giorni) e cronologia append-only;
- macchina a stati: `proposta` → `approvata` → `applicata` | `fallita`, con `rifiutata`, `annullata`, `scaduta` (tempo) e `obsoleta` (il valore corrente non corrisponde più allo snapshot: serve una nuova revisione, controllata PRIMA di ogni approvazione e applicazione);
- **doppio requisito di capability** per approvare e applicare: `documento.approve_proposals` (dedicata alle proposte) E la capability dell'operazione finale dichiarata dal tipo (`fornitore.manage_ordini`). Default: direzione e ruolo `ordini` ; gli altri con override individuali. Nessun ruolo hardcoded nei router; sedi isolate con `NOT_FOUND` ;
- l'applicazione riesegue autorizzazione, sede e freschezza, mostra l'effetto esatto («10/09/2026 → 24/09/2026, nessun altro campo viene modificato») e chiede una conferma esplicita in due passi. Un errore produce `fallita` col motivo, mai un effetto parziale nascosto;
- l'applicazione NON sposta posa, appuntamenti o stati della commessa: se la nuova consegna cade dopo una posa pianificata, il **Centro Azioni** apre/aggiorna un caso (`consegna_fornitore`, priorità alta o critica se la posa è entro 7 giorni) con l'evidenza documentale e l'azione «Rivedi la pianificazione della posa» — che propone la revisione, senza eseguirla. Il caso si risolve da solo quando il conflitto sparisce;
- UI nella scheda ordine (`/fornitori`): pannello «Proposte dall'analisi» con stato, evidenza, motivazione ed effetto esatto; la generazione parte dal run di analisi («Proponi l'aggiornamento della data di consegna»).

OCR locale per le scansioni (29/08/2026, quarta slice). Tesseract 5 eseguito in locale (`server/documenti/ocr.ts`): nessun servizio cloud, nessuna credenziale, i byte non lasciano la macchina.

- **Sequenza:** prima l'estrazione del testo nativo; solo se assente, le pagine vengono renderizzate in PNG con `pdftoppm` (Tesseract non legge i PDF) e riconosciute una per una in TSV, conservando pagina e confidenza per parola; il testo passa poi dallo stesso estrattore deterministico e dallo stesso confronto del testo nativo;
- **lingue configurabili** via `OCR_LINGUE` (default `ita+eng`, tedesco predisposto): si usa l'intersezione tra richieste e installate, con avvertenza per le mancanti; nessuna lingua utilizzabile = fallimento esplicito;
- **limiti e sicurezza:** 15 MB, 20 pagine, 300 DPI, timeout per pagina (30 s) e complessivo (120 s), una pipeline alla volta; processi avviati con `execFile` e argomenti fissi (mai shell), directory temporanee isolate e SEMPRE ripulite; il testo OCR resta input non fidato e inerte;
- **niente fallback silenziosi:** binario mancante, lingua mancante, timeout, rendering fallito e «nessun testo riconosciuto» sono esiti espliciti con motivo, e il documento resta `scansione_senza_testo`;
- **confidenza:** media per pagina dichiarata nel run; sotto le soglie (media < 80, o una pagina < 60) il run è marcato **da verificare** e la UI lo mostra («Analizzata con OCR — DA VERIFICARE»); una proposta generata da un run OCR a bassa confidenza porta l'avvertenza nella motivazione. Nessun valore estratto viene mai applicato in automatico (vale il gateway della terza slice);
- **idempotenza:** la firma OCR (versione, lingue effettive, DPI) fa parte della chiave dei run: una scansione ferma per OCR assente torna analizzabile quando l'OCR compare o cambia configurazione, senza perdere lo storico;
- **deploy:** `nixpacks.toml` installa via apt `tesseract-ocr` con `ita/eng/deu` e `poppler-utils` (~60-80 MB di immagine). In locale servono `tesseract` e `poppler` (Homebrew); le lingue non installate degradano con avvertenza.

Framework di valutazione (29/08/2026, quinta slice). `server/documenti/eval/` misura la pipeline su fixture sintetiche costruite in codice (PDF nativi e scansioni VERE: testo → rendering → immagine): riferimento esatto, inglese, multipagina, tabella spezzata, valori discordanti, ordine ambiguo, codici ordine e articolo simili, prompt injection, duplicato, file corrotto, scansione pulita/storta/bassa risoluzione/multipagina, timeout OCR.

- **Metriche separate:** correttezza e copertura per campo, precisione del collegamento (con il contatore delle «certa» sbagliate, che deve restare 0), precisione delle differenze, falsi positivi, confidenza OCR, tempo per pagina, percentuale di documenti da rivedere;
- `pnpm eval:documenti` genera il report in `docs/reports/`; i test (`eval.test.ts`) inchiodano SOLO i comportamenti deterministici (nativo perfetto, injection inerte, ambiguità mai «certa», corrotto illeggibile, timeout esplicito) e NON asseriscono soglie OCR;
- **i numeri sintetici non sono accuratezza produttiva:** la misura vera arriverà dai casi reali anonimizzati in `server/documenti/eval/casi-reali/` (cartella in `.gitignore`, mai nel repository), con `atteso.json` accanto a ogni PDF — procedura nel report baseline;
- il framework ha già ripagato: ha scoperto il match senza confini dei riferimenti (ORD-EV-10 riconosciuto dentro ORD-EV-100, FIN-100 dentro FIN-1000), corretto in `trovaRiferimentoTesto` e nel riscontro righe.

Kill switch (29/08/2026, release hardening). Tre interruttori indipendenti via env (`server/platform/interruttori.ts`), **spenti di default in produzione** e accesi in sviluppo/test: `FLAG_DOCUMENT_INTELLIGENCE` (analisi + collegamento), `FLAG_PROPOSTE` (approval gateway), `FLAG_OCR` (fallback OCR, che a flag spento produce `scansione_senza_testo` con motivo e firma `assente`). Il confine è il server: ogni endpoint verifica il proprio interruttore e risponde `PRECONDITION_FAILED` a flag spento, per qualunque ruolo — i test dimostrano che l'API non si aggira. La UI legge `plat-`

`form.interruttori` e nasconde le superfici spente. Rollout progressivo e rollback:
`docs/runbooks/rollout-document-intelligence.md`.

20. Produzione (backend senza pagina; UI rimossa il 29/08/2026)

La pagina UI `/produzione` è stata rimossa (release hardening v5.8): non veniva utilizzata. La vecchia route **reindirizza al Board (`/kanban`)** — la colonna «Produzione» è la superficie operativa dove si seguono le commesse in quello stato, quindi i segnalibri salvati atterrano nel posto giusto invece che su una pagina vuota (`produzioneRedirect` in `client/src/lib/navigation.ts`, pattern `LegacyRedirect`).

Il backend resta intatto ed è **CANDIDATO A BONIFICA FUTURA** (annotato in `server/routers/produzione.ts`): gli store `produzione_distinte` / `produzione_fasi` / `produzione_nc` possono contenere dati reali e il contratto BOM/fasi/NC è una regola di dominio — rimuoverlo richiede decisione registrata, matrice campo→consumer e sorte dei dati. Nulla di ciò che segue dipende dalla pagina: stato `produzione` della commessa, trigger §6.4, gate documentali e logiche di magazzino sono invariati (test: `server/routers/produzionePagina.test.ts`).

Tre sotto-router: `bom` (distinte base), `fasi`, `nc` (non conformità).

20.1 Distinte base (BOM)

- Campi: `commessaId`, `aperturaId`, `stato`, `componenti[]`, `noteValidazione?`, `validataDa?`, `dataValidazione?`.
- Componenti: `tipo` (`profilo|vetro|ferramenta|guarnizione|accessorio`), `descrizione`, `codiceArticolo?`, `fornitoreId?`, `quantita`, `unitaMisura`, `lotto?`, `note?`.
- Stati BOM: `bozza`, `validata`, `in_produzione`, `completata`.
- Validazione consentita solo da `bozza` → `validata`.

20.2 Fasi produzione

- Campi: `commessaId`, `aperturaId?`, `distintaBaseId`, `fase`, `ordine`, `stato`, `operatore?`, `dataInizio?`, `dataFine?`, `checklistItems[]`, `note?`.
- Stati fase: `da_fare`, `in_corso`, `completata`, `non_conforme`.
- Checklist item: `descrizione`, `obbligatorio`, `completato`, `esito?` (`ok|non_conforme`), `note?`. Toggle dedicato `toggleChecklist`.
- `dataInizio` impostata su `in_corso`; `dataFine` su `completata`.

20.3 Non conformità (NC)

- Campi: `commessaId`, `aperturaId?`, `faseProduzioneId?`, `tipo`, `gravita`, `descrizione`, `azioneCorrettiva?`, `stato`, `segnalataDa`, `dataApertura`, `dataChiusura?`.
 - Tipi: `materiale_difettoso`, `errore_taglio`, `errore_assemblaggio`, `vetro_rotto`, `ferramenta_errata`, `altro`.
 - Gravità: `lieve`, `media`, `grave`, `bloccante`.
 - Stati: `aperta`, `in_gestione`, `risolta`, `chiusa`. La chiusura imposta `dataChiusura`.
-

21. Preventivatori (/preventivatori)

Pagina hub con cards per azienda. L'accesso è aperto a tutti gli utenti autenticati.

21.1 Fivizzanese — Persiane (/preventivatori/fivizzanese/persiane)

Calcolo step-by-step:

1. **Selezione modello** (catalogo `shared/listini/fivizzanese.ts`).
2. **Dimensioni di ogni persiana** in millimetri (`larghezza × altezza`), convertite in `areaMq` .
3. **Prezzo base** calcolato in €/m² sul totale delle aree.
4. **Supplementi** configurati dal listino: moltiplicati per m² quando `unita = "mq"` , oppure per pezzo quando `unita = "cad"` .
5. **Posa e smontaggio** opzionali secondo i preset del listino.
6. **Promo** quando applicabile:
 - **Promo "RAL 6005 Opaco"**: prezzo aggiornato a **210 €** (precedentemente 200 €).
 - Quando la promo è attiva, viene aggiunto un **ricarico del 50%** sul totale persiane.
7. **IVA** (10% o 22%) — selettore.
8. Output:
 - Riepilogo a video con righe: totale persiane, supplementi, centinatura, ricarico promo, smontaggio, imponibile, IVA, totale lordo.
 - Esportazione **PDF** con `jspdf-autotable`; salvataggio automatico come documento `preventivo` sulla commessa (`keepNome=true`; nome file "Preventivo {cliente} - Fivizzanese.pdf").

21.2 Punto del Serramento — Persiane (/preventivatori/punto-del-serramento/persiane)

Stesso schema più:

- **Sconto** percentuale modificabile, **max 30%** (clamp server-side e client-side).
- **IVA** (10% o 22%) — selettore.
- Output PDF analogo con righe `lordo, sconto, imponibile, IVA, totale` .

22. Archivio (/archivio)

22.1 Soft-archive

- Endpoint `commesse.archive(id)` : imposta `archivedAt = ISO now` .
- Endpoint `commesse.restore(id)` : imposta `archivedAt = null` .
- Lo stato (`stato`) e i collegamenti (file, aperture, interventi, prodotti) **non vengono toccati**. Restore = la commessa torna esattamente com'era.

22.2 Visibilità

- Le commesse soft-archivate sono escluse da:
 - `commesse.list` (scope default `exclude`).

- `commesse.stats` e `commesse.byPriorita`.
- Board Kanban.
- Dashboard.
- Notifiche proattive (vedi §27).
- Restano accessibili a `/archivio` (scope `only`) e tramite link diretto `/commesse/:id`.

22.3 Pagina Archivio

- Lista con search per codice/cliente/città/indirizzo.
- Per ogni riga: codice, stato originale (badge), priorità, cliente, indirizzo, data apertura, data archiviazione, pulsanti **Ripristina** e **Apri scheda**.
- Conferma esplicita prima del ripristino.

22.4 Pulsanti nella scheda commessa

- **Archivia** (se non archiviata) — conferma non-destructive.
- **Ripristina** (se archiviata).
- Banner di stato "Commessa archiviata" in alto quando applicabile.

23. Classifica venditori — RIMOSSA (16/07/2026)

La pagina `/classifica`, l'endpoint `commesse.classificaVenditori` e la voce di sidebar sono stati rimossi su richiesta della direzione. La sezione resta come riferimento storico (v3); il ranking commerciali non fa più parte del prodotto.

24. Utenti (`/utenti` , direzione-only)

24.1 Funzionalità

- Lista utenti con filtro per ruolo + search testuale.
- Creazione, modifica, eliminazione (tutte `adminProcedure`).
- Multi-ruolo (1–3 ruoli per utente).
- Toggle `attivo/inattivo`.
- Password gestita solo lato server (hashing scrypt, vedi §3.3). Il campo `hasPassword` (bool) è restituito al client al posto del contenuto.

24.2 Gating

- Rotta `/utenti` protetta da `<RequireDirezionale>`.
- Sidebar mostra la voce **Utenti** solo a utenti `direzione`.

24.3 Email

- Univocità email enforced lato server (`utenti.create` rifiuta duplicati case-insensitive).

25. Centro Azioni e notifiche — v3

25.1 Principio

La campanella non misura tutto ciò che il CRM conosce. In modalità `active` mostra soltanto eccezioni personali critiche o alte già esigibili; massimo tre righe nel dropdown e badge visuale `9+`. La coda completa vive in `/notifiche`. Leggere non equivale a gestire: ogni caso possiede stato, responsabile, scadenza o revisione, evidenze e una sola prossima azione.

25.2 Segnali e deduplica

Il motore puro raccoglie aging per priorità, passaggi bottleneck, routing per ruolo, consegna mancante, saldo residuo, garanzia, ticket e intervento senza squadra. I casi sono superfici condivise: il segnale del saldo non contiene importi e il suo fingerprint usa la versione del registro pagamenti, mai il residuo (§37.3, §37.5). Segnali della stessa commessa confluiscono in un solo caso canonico; ticket, garanzie e interventi senza commessa mantengono un caso autonomo. La priorità più alta non viene mai scartata e le altre cause restano come evidenze.

Le condizioni specifiche usano fingerprint dei fatti che le risolvono: data di consegna, residuo, stato/priorità ticket, scadenza garanzia, data e squadra dell'intervento. Un semplice aggiornamento generico non chiude il caso. Commesse archiviate o soft-archivate e record di altre sedi sono escluse.

25.3 Persistenza e ciclo di vita

PostgreSQL usa `azioni_operative` con chiave unica (`sede_id`, `canonical_key`) e `azioni_operative_eventi` append-only. Gli stati sono `da_valutare`, `in_carico`, `rinviata`, `in_attesa`, `risolta`. Sono disponibili presa in carico, assegnazione direzione, rinvio, attesa con controparte/motivo/revisione, chiusura e non rilevante. Un caso scomparso dai segnali viene chiuso automaticamente; un fingerprint nuovo riapre un risolto o sveglia un rinviato.

25.4 Scope e API

La vista personale include casi assegnati all'utente e casi non assegnati destinati a uno dei suoi ruoli. La vista `Tutta la sede` è riservata alla direzione. Ogni lookup applica `sedeId`; un id di altra sede restituisce `NOT_FOUND`. Le API sono `notifiche.summary`, `notifiche.cases.*` e `notifiche.brief`. Liste e apertura pagina non chiamano alcun provider AI.

25.5 Rollout e fallback

`ACTION_CENTER_MODE` accetta `legacy`, `shadow`, `active` e vale `shadow` se assente o non valido. In `shadow` il nuovo motore persiste casi e logga soltanto conteggi aggregati, mentre la campanella conserva `notifiche.list/count` e lo store `notifiche_read`. In `active` la campanella usa il nuovo `summary`. Il fallback `legacy` resta disponibile finché il confronto produzione non è chiuso.

25.6 Promemoria personali

I promemoria sono personali, sede-scoped e visibili soltanto al richiedente. Alla scadenza il worker crea una notifica persistente di tipo `reminder` e il CRM aperto mostra un dialog globale, una scadenza per

volta. La consegna usa SSE quando disponibile e polling ogni 15 secondi come fallback; il polling si sospende in background e riparte al focus.

Il dialog mostra testo e data nel fuso `Europe/Rome` senza troncamenti. Le azioni sono **Fatto**, **Posticipa** (15 minuti, un'ora, domani alle 09:00 o data/ora personalizzata), **Apri commessa** quando collegata e chiusura. Chiudere nasconde il popup ma conserva la notifica; completare, posticipare o annullare risolve il gruppo notifica. Un errore mantiene il dialog aperto e permette il retry. Cambio sede e logout azzerano la cache prima di cambiare principal.

26. Dashboard (/)

26.1 Header personale

"Ciao {nome} — ecco la tua giornata" + data lunga it-IT.

26.2 "Da fare oggi" — feed azioni personalizzato

- **Scope**: direzione vede tutta la sede (etichetta "Tutta la sede"); gli altri solo le proprie commesse (`assegnatoA` , fallback `createdBy`) — etichetta "Le tue attività".
- Fonti, ordinate per urgenza e cap a 8 voci:
 1. Interventi di **oggi senza squadra** (CTA "Apri calendario").
 2. Commesse **urgenti** / ticket urgenti / garanzie scadute.
 3. **Da incassare** — residuo pagamenti nelle fasi finali. L'importo compare solo per chi ha `pagamento.read` (il server lo omette agli altri, che leggono «Da incassare il saldo», §37.5).
 4. **Consegne da confermare** (CTA "Conferma consegna").
 5. Ticket aperti sulle proprie commesse.
 6. Garanzie in scadenza 30 gg (direzione/amministrazione).
- Stato vuoto esplicito: "Niente da fare per ora ..." con icona verde (la card non sparisce).

26.3 KPI principali

Cards Commesse attive, Urgenze, Consegne da confermare, Ticket aperti (+ Interventi settimana). Zero = card "spenta" non cliccabile; >0 = accent bar + navigazione alla lista filtrata. Polling live.

26.4 Calendario settimanale

Slot 7 giorni con eventi per tipo (filtri per calendario) + navigazione settimana.

27. Integrazioni esterne (/integrazioni = Impostazioni)

La pagina Impostazioni ospita l'hub **Gestione** (direzione-only: Fornitori, Squadre, Garanzie, Preventivatori) e le card integrazioni:

27.1 Backup notturno su Google Drive

Vedi §39. Card con stato collegamento, ultimo backup, prossima esecuzione, "Esegui ora", collega/scollega account.

27.2 Fatture in Cloud → Clienti

Vedi §40. Card OAuth con stato collegamento, scadenza credenziali, selettore azienda, switch abilitazione, "Sincronizza ora" e disconnessione. Il token manuale mascherato resta dentro un pannello secondario come fallback di emergenza.

27.3 Mostra Google nel calendario CRM (import)

Vedi §38.2. Gestione sorgenti: nome + indirizzo iCal segreto (mascherato in lista), colore auto-assegnato, toggle mostra/nascondi, stato sync, "Aggiorna", istruzioni passo-passo.

27.4 Pubblica il calendario CRM su Google (export)

Vedi §38.1. Elenco feed copiabili (Tutti/Rilievi/Pose/Assistenza/Altro) + "Rigenera token" con avviso di revoca.

27.5 Microsoft To Do

Card presente ma **mock** (non ancora collegata a Graph API). Storicamente il flusso operativo viveva su liste To Do; la migrazione del 15/07/2026 (vedi §43) le ha importate nel CRM.

27.6 Notifiche owner / Google Maps

Invariati da v3: `system.notifyOwner` (Manus Notification Service, admin-only) e componente `<Map/>` via proxy interno.

27.7 Roadmap: Antenore (Wnd/Oknoplast)

Integrazione richiesta al fornitore (import automatico clienti/preventivi/commesse/ordini creati sul loro CRM). In attesa di risposta tecnica sugli endpoint disponibili; prevista sync unidirezionale Antenore → Ruffino Ops con dedup su id.

27.8 Contabilità → Fatturazione (piano 2, dietro flag)

Con l'interruttore `fatturazione` acceso, la sezione **Contabilità** di Impostazioni (direzione, come tutta la sezione) ospita il pannello **Fatturazione** per sede (`FatturazioneConfigPanel` , §56.8):

- **Permessi di scrittura fatture.** La sola sincronizzazione clienti/fatture (§40) usa scope di **lettura**: emettere richiede una ri-autorizzazione OAuth con gli scope di scrittura su clienti, fatture e note di credito. Il pulsante «Ri-autorizza con permessi di scrittura» compare solo con l'OAuth client FiC configurato; scollegare l'account FiC azzera anche l'esito della verifica.
- **«Verifica permessi»** chiama `/issued_documents/info` e mette in cache aliquote IVA 22/10, numerazioni, conti e metodi di pagamento della sede. Il conto si auto-assegna se è l'unico e, dopo la veri-

fica, entra nel modulo **solo se il campo era vuoto**: un conto scelto a mano non si perde con un modulo sporco.

- **Da compilare a mano**: IBAN (validato col modulo 97), banca, intestatario, metodo di pagamento, numerazione FiC e spese di documentazione (default 150,00 € per sede).

Il pannello del catalogo DEI in sola lettura vive invece nella card «Limiti di spesa» della stessa pagina, dietro `tariffe.manage` (§55.4).

28. Persistenza

28.1 KV store

- Tabella `kv_store`(key text primary key, data jsonb, updated_at timestampz).
- Ogni router business possiede una o più raccolte persistite, tra cui `clienti`, `commesse`, `tickets`, `ticket_allegati`, `preventivi_documenti`, `utenti`, `sedi`, `backup_*`, `fic_config`, `fic_fatture`, `caselle_email`, `whatsapp_*`, e `conoscenza_aziendale`.

La tabella `comunicazioni` è separata dal KV store: insert idempotente per (`casella_id`, `canale`, `message_id`), indici per lista e tombstone per le eliminazioni dal CRM (§51).

Le tabelle PostgreSQL `promemoria` e `promemoria_eventi` conservano scadenze personali e audit append-only. Ogni query e mutation applica `sede_id` e `recipient_user_id`; un record fuori scope restituisce `NOT_FOUND`. Il claim delle scadenze usa locking concorrente e la proiezione notifica è idempotente per id e revisione.

Il refresh token Google del backup è inoltre **specchiato su file** (`data/backup-oauth.json`, mode 600, gitignored) così i riavvii senza `DATABASE_URL` non scollegano Drive; la riga DB, quando presente, ha precedenza.

28.2 Lifecycle

- **Bootstrap**. All'avvio `bootstrapAll()` legge ogni raccolta. Tre stati possibili per la singola key:
 - `firstBoot = true`: nessuna row presente → callback `onLoad` può popolare seed.
 - `firstBoot = false`, `loaded = true`: row presente, dati ripristinati con `reviver Date`.
 - `firstBoot = false`, `loaded = false`: errore transiente; **i save sono bloccati** finché la background recovery non riesce a leggere lo stato dal DB.
- **Save**. Debounciato 200 ms; usa `sql.json()` per evitare doppia encoding. Retry exponential backoff su errori transienti (`EAI_AGAIN`, `ENOTFOUND`, `ECONNREFUSED`, `ECONNRESET`, `ETIMEDOUT`, `ENETUNREACH`, `EHOSTUNREACH`, `EPIPE`).
- **Shutdown**. Handler `SIGTERM` / `SIGINT` chiamano `flushAll()` per non perdere mutazioni in flight, poi chiudono la pool.

28.3 Recovery

- Se `ensureSchema` o `load` falliscono dopo i retry → `backgroundRecover` (max 30 tentativi a intervalli di 5 s) finché ogni raccolta non risulta `loaded = true`. Nessun seed viene applicato in caso di fallimento (per non sovrascrivere dati reali con un seed accidentale).

28.4 Legacy double-encoded payload

- Riconosciuto a load: se `data` è stringa JSON, viene parsata e schedulata una riscrittura corretta.

28.5 Vincoli su Postgres

- Connessione Postgres-js: max 5 connessioni, `idle_timeout=20s`, `connect_timeout=30s` (Railway internal DNS warmup).
- SSL automatico quando l'URL contiene `sslmode=require` o l'host Railway.

29. UI/UX e componenti

29.1 Sistema

- Tailwind + shadcn/ui (Button, Card, Badge, Dialog, AlertDialog, Avatar, Input, Label, Select, Switch, Tabs, Tooltip, DropdownMenu).
- Icone lucide-react.
- Toast: sonner.
- Font Plus Jakarta Sans; palette chiara calda con superfici bianche, inchiostro scuro e giallo come accento. I colori di stato usano token semantici (`success`, `warning`, `danger`, `info`), non hex locali.
- Raggi tra 8 e 14 px, bordi visibili e ombre contenute. Le superfici operative non devono apparire fredde, eccessivamente opache o spigolose.
- Componente `<SearchSelect>` riutilizzato per dropdown ricercabili (utenti, squadre, fornitori, ecc.).
- Componente `<ConfirmDialog>` riutilizzato per: cancellazioni, archiviazioni, "Procedi comunque" del doc gate, ripristino archivio.
- Componente `<FilePreviewDialog>` per preview PDF/immagini.

29.2 Sidebar

- Voci dinamiche; quelle con `direzioneOnly` filtrate per ruolo.
- Resizable (larghezza personalizzabile da utente).
- Avatar utente in basso con menù: nome, email, Esci.

29.3 Mobile

- Layout responsivo. Sidebar collassata in modalità mobile; nasconde scritte tenendo solo icone.
- **Header delle schede** (commessa, cliente): su viewport `< sm` il titolo e la riga di azioni si impilano (`flex-col` → `sm:flex-row`) e i bottoni vanno a capo (`flex-wrap`) invece di uscire dallo schermo.
- **Tabelle di lista** (Commesse, Clienti): le colonne secondarie sono nascoste progressivamente con `hidden {sm|md|lg|xl}:table-cell`, così su telefono restano solo le essenziali — Codice/Cliente/Stato per le commesse, Nome/Telefono per i clienti — con padding ridotto (`px-3`) sulle colonne mantenute. **Non** viene usato un wrapper `overflow-x-auto`: creerebbe un contenitore di scroll che romperebbe gli header `sticky` (regressione già occorsa e corretta in passato).
- Board, Calendario, Dashboard, Pagamenti e Magazzino riflowano nativamente (griglie responsive + `flex-wrap`); nessuno scroll orizzontale di pagina.

- Le tabelle delle sezioni direzione-only a bassa frequenza (Fornitori) restano larghe: sono pensate per l'uso da desktop.

29.4 Empty states

- Tutte le pagine principali hanno empty state esplicito con istruzioni sul prossimo passo.
-

30. Errori e telemetria

30.1 Convenzioni errori

- Gli errori non-trRPC sono ritornati come `Error` con messaggio human-readable in italiano.
- Errori "marker" usano un **prefisso** come `DOC_GATE_BLOCKED:` che il client identifica per offrire UI dedicata.

30.2 Logging

- Tutto il logging della persistenza passa da `console.log/warn/error` con prefisso `[persistence]`.
 - I save e i load mostrano sempre il conteggio degli elementi per raccolta.
-

31. Roadmap aperta (lavori noti)

31.1 Sicurezza

- **Operatività:** ruotare le vecchie credenziali seed eventualmente ancora usate, rifare il login locale `gh` e revocare token GitHub non riconosciuti.
- **Decisione separata:** purge delle password seed dalla cronologia (`BFG/git filter-repo`). Richiede riscrittura SHA, force-push concordato e riallineamento di ogni clone.
- CSP (Content Security Policy) tarata su Vite, blob preview, Maps proxy.

31.2 Ottimizzazione

- **Attivazione object storage:** il layer per-file è completo; restano configurazione R2 su Railway, probe, backup Drive, dry-run sui dati reali e apply (§47). Il dry-run locale senza `DATABASE_URL` non è una verifica della produzione.
- Aggregato dashboard in un unico endpoint per ridurre il fan-out lato client.
- **Flag di piattaforma:** reintrodurre un endpoint direzione con motivazione e audit — `platform.flags` è di sola lettura dal 28/08/2026 e i flag sono congelati ai valori salvati (§53.2).

31.3 UX

- Drag&drop diretto sulle colonne del Kanban (oggi solo bottoni avanza/indietro).
- Confetti hardware-accelerati (opzionale).

31.4 Integrazioni

- **Fatture in Cloud OAuth:** codice completato. Restano configurazione delle variabili Railway, registrazione del redirect e collegamento di ogni sede (§40.3).
- **Antenore (Wnd/Oknoplast):** connettore import clienti/preventivi/ordini — in attesa di specifiche dal fornitore.
- Esportazione CSV/Excel commesse, clienti, anomalie.
- Web Push e avvisi email per i promemoria a CRM chiuso (fuori ambito v4.30).
- UI di restore dal backup Drive.
- Mutation manuale di archiviazione allegati WhatsApp nel fascicolo: il percorso passava dalle proposte dell'agente rimosso (§51.3, limite noto).

31.5 Contratto, limiti e fatturazione (§55-§58)

- **Decisioni parcheggiate del motore limiti** (harvest del 05/09, §55.7), in attesa di direzione o commercialista: **H3** veneziane contate a pezzo nel foglio e a mq nel seed; **H4** cinque fogli su un'edizione precedente del listino DEI; **H5** il foglio somma nei totali solo le opere davvero fatturate, mentre `OpzioniComputo` non sa escludere una singola opera; **H6** doppio prezzo dell'avvolgibile PVC standard nello stesso foglio. Finché non arrivano, i casi che le toccano restano `salta` in fixture. Manca ancora un foglio reale con serramenti **in legno**.
- **Tariffe modificabili dalla UI con validità** (decisione D10): il pannello del catalogo DEI resta in sola lettura, il seed si rigenera da script.
- **Prima fattura reale e prima lettura reale:** i due runbook (§56.11, §57.5) non sono stati eseguiti. Restano aperti col commercialista il segno del totale della nota di credito, la company FIC di prova e le aliquote di detrazione 2025/2027 nel seed.
- **Residui della lettura del contratto** (§57.6): sostantivi di accessorio fuori dalla lista chiusa, materiale composto dedotto senza avvertenza, intestazione del prompt non neutralizzata, editor delle rate duplicato fra dialog e tab, `casi-reali/` dell'eval ancora vuota.
- **PEC, codice destinatario e `ficEntityId`** del cliente restano campi server senza UI nel form cliente.
- **Piano 4 «Fatturazione guidata»** (§58): spec e piano scritti, nulla implementato.

32. Glossario

- **Commessa.** Progetto specifico di vendita+installazione per un cliente.
- **Apertura.** Singolo serramento (finestra, porta, ...) all'interno di una commessa.
- **Stato.** Posizione corrente della commessa nella state machine.
- **Soft-archive.** Stato secondario, ortogonale allo stato del workflow: la commessa è nascosta dalle viste operative ma tutto è preservato.
- **Doc gate.** Vincolo per cui un avanzamento di stato richiede l'upload di documenti previsti per lo stato corrente.
- **Bypass.** Conferma esplicita dell'operatore tramite `force: true` per superare il doc gate.
- **Indirizzo di residenza.** Indirizzo fiscale del cliente (uso amministrativo).
- **Indirizzo di lavoro.** Indirizzo del cantiere (uso operativo per commesse, calendario, mappe).
- **Tipo detrazione.** Modalità fiscale richiesta dal cliente: `ecobonus` o `ristrutturazione`.

- **DEI.** Il listino di riferimento dei prezzi (prodotti, accessori, controtelai, opere) con cui si calcolano i limiti di spesa ammessi dalla detrazione. Nel CRM vive come catalogo seed (`shared/limiti/tariffe-seed.json` , §55.2): il codice DEI di una riga esce sempre dal catalogo, mai da una deduzione.
- **CHECK1 / CHECK2.** I due modi in cui il foglio «CALCOLO NUOVI LIMITI» calcola il tetto di spesa: CHECK1 dai massimali per zona e mq più controtelai e opere; CHECK2 dai prezzi DEI ricalcolati riga per riga. Il limite vincolante è il minore dei due; se una riga non ha voce DEI, CHECK2 non è calcolabile e l'esito è «incompleto» (§55.3).
- **Bene significativo.** Voce di fattura che la normativa tratta a parte nel calcolo dell'IVA agevolata: se il loro totale supera la prestazione, l'eccedenza resta al 22 % (§56.2). Gli accessori non sono beni significativi.
- **Pattuito lordo / imponibile.** Come è stato scritto il prezzo nel contratto: IVA compresa (`lordo`) o al netto (`imponibile`). Il risolutore della fattura usa formule diverse nei due casi.
- **Markup.** La voce «MarkUp servizi di vendita» della fattura: sempre **derivata** dal pattuito e dalle altre voci, mai digitata; se risulta negativa la bozza non è emettibile.
- **Dry-run Sdl.** Invio simulato della fattura elettronica (`FATTURAZIONE_SDI_DRY_RUN` , acceso finché non vale `off`): lo stato resta «Emessa (prova Sdl)». Il numero però lo assegna davvero Fatture in Cloud, che non ha bozze (§56.5).
- **Estrazione (del contratto).** La lettura del PDF del contratto da parte del modello: produce una **proposta** con evidenze verificate sul testo, che una persona rivede prima che tocchi il contratto strutturato (§57).

33. Cronologia significativa

- **v5.36 (05/09/2026) - Fixture d'oro del motore limiti dai fogli reali, due bug corretti, piano 4 pianificato** (Ruling R22 del piano 2). Su `feature/fixture-limiti-reali` (commit 5690958 , `f7-b713b` , `eced152` , `528a59c` , `ea06ec2`), **non ancora integrato su main** . Nuovo `scripts/harvest-fixture-limiti.py` : un solo comando trasforma una copia compilata del foglio «CALCOLO NUOVI LIMITI» in un caso d'oro **anonimo** (legge misure, codici DEI, prezzi di riga, totali e le celle di CHECK1; **non** legge nominativo, indirizzo e comune; il foglio non entra mai nel repository, il caso si chiama come dice `--nome` ; un prodotto ignoto ferma lo script invece di indovinare). `server/computo/___fixtures___/casi-reali.json` passa da 3 a **20 casi: 13 verdi al centesimo, 7 saltati** col motivo scritto nel campo `salta` di ciascuno — divergenze capite e dichiarate, non tolleranza allargata. L'-harvest ha trovato e corretto due bug del motore: **H1**, la maggiorazione dell'avvolgibile abbinato aveva larghezza e altezza scambiate (coefficienti rinominati `avvolgibileExtraLarghezza / avvolgibileExtraAltezza` , stesso valore, dimensione giusta); **H2**, un cassonetto venduto insieme al serramento (blocco B del foglio) pesava nel massimale A invece che in B — nuova chiave `cassonettiB` in `aggregati.ts` , che si somma ad A ovunque conti il prodotto (rilievo, rimozione tapparelle, smaltimento, tiro, posa) e non fa posare due volte la tapparella che ospita. **H7** chiuso: il form dichiara l'oscurante abbinato anche su una riga cassonetto e le avvertenze «oscurante senza voce DEI» non scattano più su un cassonetto abbinato senza tipologia propria. Parcheggiate in attesa di direzione o commercialista **H3-H6** (veneziane a pezzo o a mq, cinque fogli su un'edizione precedente del listino DEI, inclusione «solo fatturato» che `OpzioniComputo` non sa rappresentare, doppio prezzo dell'avvolgibile PVC nello stesso foglio); un settimo caso resta fuori senza decisioni da prendere (riga «serramento + persiana» senza prodotto persiana: CHECK2 non calcolabile, fail-closed per progetto). Manca ancora un foglio reale con serramenti in legno. Stesso giorno: **spec e piano del piano 4 «Fat-**

turazione guidata» (§58, commit 1e5f51d), scritti e approvati ma **non implementati**, e questo PRD portato a 5.36 con le sezioni §55-§58. Suite: 234 file passati e 9 saltati, **2.350 test passati e 50 saltati** (i saltati sono le suite *.pg.test.ts , che girano solo con DATABASE_URL , più i 7 casi d'oro dichiarati).

- **v5.35 (05/09/2026) - Lettura del contratto PDF** su main (piano 3 di 3, 9 task, docs/superpowers/plans/2026-09-04-lettura-contratto.md ; tip d7e0ab5), dietro il nuovo interruttore fail-closed FLAG_CONTRATTO_ESTRAZIONE , che richiede anche FLAG_LIMITI e un provider Tars reale (classe di costo document_intelligence , modello TARS_MODEL_ESTRAZIONE_CONTRATTO). §57. Il modello legge il PDF del contratto firmato e **propone** righe, pattuito, posa, rate e cantiere; nulla viene salvato senza revisione umana. Schema JSON **strict** (estrazione/schema.ts , nullable come union con null) con SCHEMA_JSON_ESTRAZIONE come proiezione dello zod, mai il contrario; input a pagine intere fra marcatori neutralizzati (un documento non può fingere una pagina che non esiste), un solo tentativo, prompt versionato 1.0.0 con l'impronta della configurazione OCR nella chiave di riuso. Il riuso si decide **prima** di estrarre il testo (OCR e lettura visiva costano) e estraiTestoDocumento è chiamato senza lettura visiva: una scansione illeggibile è un errore esplicito, mai una spesa silenziosa. **Mappatura deterministica** (estrazione/mappa.ts , il pezzo più delicato): codici DEI **solo dal catalogo**, ogni valore con {valore, evidenza, daVerificare, nota} e l'evidenza verificata sul testo vero; natura scorrevole/alzante decisa dal sostantivo più vicino, con l'apertura esplicita del serramento che prevale e lo dichiara; materiale per posizione (primo nominato, avvertenza se più d'uno); oscuranti autonomi fusi nel serramento solo a misure uguali (± 10 mm) e pezzi sufficienti, altrimenti righe a sé con avvertenza; accessori solo da etichette note; posa per parole chiave solo su righe senza misure. Arricchimento facoltativo dal layout WnD (riconoscimento su «Riepilogo Costi», poi totali e termini di pagamento) che riscrive numeri con evidenza certa senza cancellare la quota dell'oscurante già fusa. contratto_estrazioni idempotente per documento e versione di prompt; applicaEstrazione scrive **solo** tramite salvaContratto , con stato e timeline in try/catch (un loro errore è un'avvertenza, mai un contratto scomparso); eseguiEstrazioneContratto è fail-closed da solo, non si fida del router. Router estrazioniContratto (stato , esegui , applica , scarta) con sede verificata anche sullo scarto; nessuna capability nuova (riusa contratto.read / contratto.manage). UI: dialog «Leggi il contratto» con evidenze, note del lettore e revisione inline; «Compi-la a mano» sempre disponibile quando la lettura non è configurata. Eval pnpm eval:contratti su tre fixture sintetiche (WnD, Word, scansione) senza rete, chiamata reale solo con EVAL_CONTRATTI_REALI=on e provider realmente disponibile; server/contratti/eval/casi-reali/ resta vuota, quindi l'eval misura parser e mappatura, non l'accuratezza reale del modello. Ruling P3-R1...P3-R42 nel ledger del piano. pnpm check pulito; suite 234 file passati e 9 saltati (243), **2.332 test passati e 43 saltati** (2.375); build con l'unico avviso noto (dist/index.js 3,1 MB). Fuori taglio: nessuno strumento Tars, nessun formato diverso dal PDF, nessuna applicazione automatica.
- **v5.34 (04/09/2026) - Fatturazione dal contratto** su main (piano 2 di 3, 18 task, docs/superpowers/plans/2026-09-04-fatturazione-dal-contratto.md ; 4104e27 porta gli interruttori letturaVisiva e fatturazione insieme), dietro il nuovo interruttore fail-closed FLAG_FATTURAZIONE , che richiede FLAG_LIMITI sullo stesso ambiente (verificato per handler). §56. Dal contratto strutturato e dal computo nasce la bozza, che il CRM emette su Fatture in Cloud, archivia e segue. **6 tabelle** (fatturazione_config , fatture , fattura_righe , fattura_riepilogo_iva , fattura_scadenze , fattura_eventi) con blocco ottimistico su revisione e immutabilità da in_emissione in poi. **Risolutore** puro (G pattuito, B beni significativi, N altri beni, S servizi, M markup derivato; 10 % su 2P e 22 % su B-P quando B > P) verificato al centesimo su tre fatture reali del 2026, con «Riequilibra i beni» ad arrotondamento cumulativo (somma esatta al target, righe mai negative, scarto ≤ 1 centesimo a riga) perché la prassi della commercialista è abbassare i beni, non tagliare i servizi. **Ge-**

neratore: righe bene al 22 % dal contratto, servizi al 10 % dai limiti (arrotondati all'euro, mai per eccesso), markup, coppia storno/riaddebito, **spese di documentazione come bene al 22 %** (default 150,00 € per sede, fuori da entrambi i blocchi dei limiti), diciture con manutenzione straordinaria e pratica edilizia, scadenze **50/40/10** a 0/60/75/90 giorni col resto sull'ultima, righe manuali in bozza (max 20 per operazione). **Limiti verificati per tre blocchi separati** — prodotti+markup contro i massimali, servizi contro le opere proposte, imponibile contro il minore fra CHECK1/CHECK2 — mai come un totale unico; termine di paragone a zero = avviso `limiti_non_verificati`, mai un «ok» di comodo. Lo scavalco richiede una **seconda** autorizzazione con `fattura.emit` e un motivo, controllato nel servizio. **Emissione** idempotente per passo (validazione → cliente FiC → documento → confronto totali → XML → invio → archivio → documento nel fascicolo → timeline) con **lease** compare-and-swap su stato e revisione: due «Emetti» sovrapposti danno `CONFLITTO` al secondo prima di toccare FiC, mai due numeri. Invio Sdl in prova con `FATTURAZIONE_SDI_DRY_RUN` (variabile di tutto il deployment, accesa finché non vale `off`): stato «Emessa (prova Sdl)», ma FiC numera davvero. **Sonda** ogni 15 minuti in un solo processo: legge `ei_status`, recupera l'archivio mancante, riappaia le scadenze scollegate a ogni giro, non ritenta mai l'invio. **Nota di credito** totale o parziale sulla stessa pipeline, specchio esatto dell'origine con intestazione «Accredito su ns. fattura n. X del Y». Sync FiC: un documento il cui id combacia con `fatture.ficDocumentId` nasce collegato (`commessaMatch: "crm"`), senza match automatico né secondo PDF, e non si può ricollegare a mano a un'altra commessa. Capability `fattura.read` (amministrazione, commerciale, direzione) e `fattura.-draft / emit / credit_note` (amministrazione, direzione). UI: tab «Fattura» della commessa, pannello Fatturazione in Impostazioni → Contabilità (scope FiC di scrittura, «Verifica permessi», IBAN col modulo 97), sezione «Fatture emesse dal CRM» in Cassa. Tars: nessuno strumento nuovo, ma il fascicolo mostra una riga per fattura **senza mai un importo** e senza ripetere l'errore Sdl parola per parola. Ruling di piano nel ledger (`Ruling R...`, fino a R40 citati in `handoff.md`); runbook della prima fattura reale in `handoff.md`. `pnpm check` pulito; suite 210 file passati e 7 saltati (217), **2.018 test passati e 32 saltati** (2.050). Fuori ambito v1: fatture libere, acconti, IVA al 4 %, B2B senza contratto.

- **v5.33 (04/09/2026) - Contratto strutturato e computo dei limiti di spesa** su `main` (piano 1 di 3, 16 task, merge `9afaf4c`), dietro il nuovo interruttore fail-closed `FLAG_LIMITI`. §55. Il contratto smette di essere un elenco libero di prodotti e diventa un documento con righe misurate e prezzate (`commessa_contratti`, `commessa_righe`), da cui il motore ricalcola i limiti ammessi dalla detrazione (`computi`, `computo_voci`). **Motore puro** `server/computo/motore.ts` verificato al centesimo su tre commesse reali chiuse nel 2026 col foglio «CALCOLO NUOVI LIMITI» compilato a mano: aggregati per gruppo, ore di tiro e posa, CHECK1 (massimali per zona e mq + controtelai + opere), CHECK2 (prezzi DEI riga per riga), limite = minore dei due, esito «incompleto» quando una riga non ha voce DEI. Le stranezze del foglio si riproducono per scelta (minimo 1 mq sul totale della riga e solo per PVC/alluminio, precedenza degli operatori dello smaltimento, ribalta a pezzo, incollaggio ad anta, soglia del portoncino una volta per riga): sono fatti contabili già accettati, cambiarli è una decisione di direzione. Catalogo seed `shared/limiti/tariffe-seed.json` (342 prodotti, 74 accessori, 22 controtelai, 19 opere) rigenerato da `scripts/estrai-tariffe-limiti.py` — il foglio del listino non entra mai nel repository — e `shared/limiti/comuni-zona.json` (8.104 comuni, Tabella A del DPR 412/93, sigle di provincia del 1993) letto come import statico: nessun caricamento manuale al deploy. **Gate:** `richiedeComputo` blocca **solo** `aggiornamento_contratto` → `fatture_pagamento`; lo scavalco è lo stesso «Procedi comunque» del board e resta nel registro come `gateScavalcato: "documentale" | "computo"`; Tars vede lo stesso gate, lo rivaluta a ogni tappa e senza scavalco si ferma dicendo che manca il **computo**, non un file. UI: tab «Contratto» al posto di «Prodotti», tab «Limiti», banner di stato, badge «da contratto · {pattuitoTipo}» in Pagamenti, pannello Tariffe in sola lettura in Impostazioni. Salvando, `applicaPattuitoDaContratto` allinea pattuito e piano rate senza

toccare le rate già incassate. Capability nuove: `contratto.read` (condivisa da tutti i ruoli), `contratto.manage` e `computo.run` (amministrazione, commerciale, direzione), `tariffe.manage` (solo direzione); il client legge il proprio set da `trpc.permessi.mie`, senza duplicare stringhe. `pnpm check` pulito; suite 170 file, **1.591 test passati e 23 saltati** (le `*.pg.test.ts` girano solo con `DATABASE_URL`: eseguite a parte contro PostgreSQL 16 locale, tutte verdi). Fuori taglio: tariffe modificabili dalla UI con validità (D10), fatturazione (v5.34) e lettura automatica del contratto (v5.35).

- **v5.32 (02/09/2026) - Analisi azienda e sintesi giornaliera di Tars** (fase successiva del piano smistamento; mandato «non sta analizzando l'azienda ... deve proporre»). Modulo `server/tars/analisi/` dietro `FLAG_TARS_ANALISI_AZIENDA` (fail-closed, richiede `FLAG_TARS_PROACTIVE`): fotografia deterministica della sede (commesse attive per stato e ferme da più tempo, casi aperti del Centro Azioni, osservazioni, pattern, smistamento, ticket, interventi della settimana, proposte documentali; senza importi, ogni fatto con riferimenti di entità e link) → sintesi del modello a output JSON strict (`TARS_MODEL_ANALISI`, default `gpt-5.6-sol`, classe di costo `analisi_azienda`): sintesi, punti (rischio/anomalia/andamento/opportunità con priorità), proposte con la frase da dire a Tars per eseguirle, domande alla direzione; verifica deterministica (entità solo dalla fotografia, importi scrubbati, limiti), fallback deterministico senza provider. Una analisi per sede al giorno dalle 06:00 Roma (worker ogni 5 minuti), registro `tars_analisi_azienda`, rigenerazione manuale. Endpoint `tars.analisiAzienda` / `tars.analisiAziendaRigenera` (direzione). UI `/tars` (ridisegnata la sera stessa: selettore Chat / Proposte / Registro in testa, Proposte come coda di decisioni a righe a tutta larghezza — titolo, destinazione in evidenza, chip di sicurezza, Approva/Rifiuta grandi, dettagli a richiesta, filtri — e Registro a colonne): «Analisi di oggi» nel pannello contesto e nello stato vuoto, gruppo «Dall'analisi dell'azienda» nelle Proposte con «Chiedi a Tars» (precompila la chat: nessuna mutazione nasce dall'analisi). Fuori taglio: dati economici, invio via mail, storico fra giorni.
- **v5.31 (02/09/2026) - Tars libero** (mandato direzione: «deve leggere tutto, capire tutto e poter fare tutto; quando serve chiede, quando è sicuro fa da solo; se l'ha fatto Tars viene segnalato; sezione proposte sulla pagina Tars»). Policy in `CLAUDE.md` «Agente AI» e piano `docs/superpowers/plans/2026-09-02-tars-libero.md`. **A:** catalogo = tutto l'autorizzato per capability/sede/flag (niente potatura per superficie/intento), niente classificatori deterministici al posto del modello, ambiguità come hint nel contesto, chiarimenti letti contro i candidati, nessuna autorità derivata dal testo (gli strumenti verificano sede/archiviazione/state machine/gate/versione da soli), prompt v9. **B:** 13 strumenti R1 di scrittura (`strumenti/scrittura.ts`: crea/aggiorna cliente; crea/aggiorna/archivia/ripristina commessa; aggiorna/chiudi ticket; pianifica intervento; collega/classifica/segna gestita comunicazione; risolvi caso) che eseguono la stessa procedura del router con il contesto server dell'utente (stesse capability e `authorizeCoreOperation`), registro azioni 1.10.0 = 44 azioni. **C:** pagina `/tars` con schede Chat / Proposte / Registro; endpoint `tars.proposte` (gateway documentale aperto in sede) e `tars.registroAzioni` (ledger R1: strumento, esito, «Tars per», entità toccate, annullabile). Conferma umana solo per importi, cancellazioni definitive, effetti esterni o su altre sedi. **Smistamento D7:** collegamento automatico anche dal modello quando la commessa indicata con confidenza alta è l'unico candidato commessa (punteggio ≥ 30, rivale a ≥ 20 punti, non archiviata) — le proposte «unica commessa attiva della cliente» erano inutili; `VERSIONE_SMISTAMENTO` 1.2.0 riesamina le aperte. **D8:** `archiviaAllegatoComunicazione` non duplica un file già nel fascicolo (SHA-256; legacy nome+dimensione), per smistamento, strumento R1 e lettore mail. Fuori taglio: conferme pendenti dei turni nella sezione Proposte, Undo dal Registro, analisi azienda/sintesi giornaliera.
- **v5.30 (02/09/2026) - Tars SMISTAMENTO** delle comunicazioni (mandato direzione: «un cervello operativo non deve farsi scappare niente»). Nuovo motore `server/tars/smistamento/` dietro `FLAG_TARS_SMISTAMENTO` (fail-closed, richiede `FLAG_TARS_COMMUNICATIONS` + `FLAG_TARS_PROAC-`

TIVE + PostgreSQL): ogni comunicazione in ingresso viene smistata entro un minuto — candidati deterministici spiegabili (codice commessa, stesso filo già collegato, mittente originale degli inoltri interni, telefoni, cognomi/ragioni sociali con esclusione del personale), analisi col modello a output strutturato (categoria chiusa, urgenza, riepilogo senza importi, risposta attesa, azione suggerita, collegamento SOLO fra i candidati, tipo documento per allegato), effetti deterministici: collegamento automatico SOLO se certo (D1) senza toccare lo stato della comunicazione, archiviazione automatica degli allegati riconosciuti solo su comunicazioni collegate (D2: modello e regole concordi o confidenza alta; immagini solo da WhatsApp sopra 30 KB; `vietaRiassegnazione`, idempotente per source-Ref, reversibile dal fascicolo), triage sulle colonne legacy `categoria / tars_riepilogo / tars_istruzione` (che tornano ad avere un consumatore), proposta a un click per tutto il resto. Registro `tars_smistamento` (esito, proposta, tentativi, errori); worker ogni 60 s per sede (recenti prima; modello entro 90 giorni, oltre solo deterministico; oltre 365 giorni escluso); segnali `comunicazione_decisione / comunicazione_risposta` nel Centro Azioni; sezione `smistamento` del briefing (da decidere, da rispondere, urgenti, contatori); endpoint

`tars.smistamentoStato/PerComunicazione/Proposte/Decidi/Riesamina` (decisione con `commessa.update_operational`, doppia decisione = CONFLICT; approvazione = collegamento manuale «approvato da » + archiviazione). Contratto provider esteso con `formatoJson` (`Responses text.-format json_schema strict`); classe di costo `smistamento` (modello `TARS_MODEL_SMISTAMENTO`, default `gpt-5.6-terra`). UI: banner Tars nel lettore email con Collega/No, liste nella Situazione (Dashboard) e nel pannello contesto di `/tars`. Fuori taglio: analisi azienda su dati reali e sintesi giornaliera, pagina Centro Azioni, UI osservazioni/panorama/miglioramenti. Stesso giorno: **chiarificazione robusta** (la risposta a «Quale intendi» accetta progressivo, codice, ordinale e nome; valvola dopo due risposte non riconosciute; massimo quattro candidati persistiti — prima un quinto candidato faceva perdere il contesto) e **strumento R1 crea_ticket** (post-vendita, commessa dal contesto verificato o esplicita, `ticket.create`, 31 azioni a registro). Suite 136 file / 1395 test.

- **v5.29 (01/09/2026)** - Tars PROATTIVO PIENO su decisione della direzione («non preoccuparti dei budget, eliminali tutti: un cervello operativo non ha bisogno di budget», registrata in `docs/tars/gate-openai.md §8`). **Tetti di spesa eliminati:** `TARS_MAX_COST_PER_RUN_USD / TARS_DAILY_BUDGET_USD / TARS_MONTHLY_BUDGET_USD` senza default (variabile assente = nessun tetto; un valore impostato resta validato fail-closed e applicato; gerarchia valutata solo fra i tetti presenti; tetto di sanità solo su mensile impostato); `TARS_BUDGET_<CLASSE>_USD` assente = nessun tetto di classe, 0 esplicito = kill switch della classe. Contabilità INVARIATA (decoratore prenota→riconcilia su ledger PG, `tars.costi` mostra sempre la spesa reale, residui `null` dove non c'è tetto), circuit breaker errori e limiti operativi del run invariati; card Agente mostra «nessun tetto». **Flag accesi in produzione** (Railway): `FLAG_TARS_PATTERNS`, `FLAG_TARS_IMPROVEMENTS`, `FLAG_TARS_PROPOSALS`, `FLAG_DOCUMENT_INTELLIGENCE`, `FLAG_PROPOSTE`, `TARS_OBSERVER_MODE=active`. **Accesso ampliato:** nuovi tool `L0 cerca_clienti` e `leggi_cliente` (registro azioni v1.9.0, 30 azioni) con archiviati fuori dai quadri operativi salvo richiesta esplicita, economia aggregata solo con capability economiche, cross-sede NOT_FOUND. Fix contestuale: il test dell'audit policy usava date fisse ed è scaduto col calendario (ora relative). Suite 128 file / 1266 test; eval 16/16.
- **v5.28 (01/09/2026)** - Tars operativo T4-T10 completati su `main` locale (nessun flag acceso, nessuna chiamata OpenAI, produzione invariata fino al push autorizzato). **T4 allegati e catena Maccari:** classificatore documentale deterministico (nome/oggetto/testo → DocTipo, fallback `altro`), tool `R1 archivia_allegato_comunicazione` con corrispondenza certa (comunicazione viva collegata alla commessa autorizzata + allegato letto e verificato in conversazione), rilettura della fonte DENTRO la sezione serializzata del servizio canonico con confronto del fingerprint (fonte cambiata = nessuna scrittura), riassegnazione cross-commessa rifiutata (`vietaRiassegnazione`, il flusso manuale del

router mail conserva lo spostamento storico), e transizione condizionale dal set CHIUSO «se appartiene alla commessa / se non trovi problemi»: il classificatore base continua a rifiutare ogni condizione, l'autorità nasce solo quando l'orchestratore verifica le condizioni sull'esito reale dell'archiviazione. Regressione Maccari 6/6 (esecuzione diretta, domanda unica, fonte incoerente, gate invalido, capability mancante, promemoria singolo). **T5 frontiera unica R2/R3**: anteprima hashata nella proiezione delle proposte e verifica opzionale retrocompatibile in `approvaEApplica` (hash diverso = CONFLICT senza applicare), doppio click che riusa, e azioni senza servizio canonico (invio email/WhatsApp, pagamenti) dichiarate INDISPONIBILI con blocco reale nel registro (il validatore rifiuta chi prova a registrarle) ed esposte in `tars.stato`. **T6 osservatore**: consuma i draft già riconciliati dal Centro Azioni (nessun secondo event mesh), regole deterministiche a costo zero con materialità anti-rumore e sintesi sempre senza importi, persistenza additiva `tars_osservazioni` (chiave unica sede/caso/detector/versione, storico append-only, cooldown 6h, riapertura solo a evidenze nuove), fail-closed su `FLAG_TARS_PROACTIVE` e storage autorevole; `TARS_OBSERVER_MODE` `shadow` (default: persiste senza esporre) / `active` (`tars.osservazioni` filtrata per sede e capability a ogni richiesta). **T7 pattern e Panorama**: aggregazioni deterministiche entro la sede su finestre dichiarate — ritardi fornitore, colli di bottiglia, ricorrenze post-vendita, permanenza per fase dal registro transizioni reale, bypass del gate — con campione minimo di commesse distinte, baseline esplicita, correlazione dichiarata (mai causalità) e soppressioni dichiarate; tool `panorama_azienda` direzione-only ed endpoint `tars.panorama` dietro il nuovo `FLAG_TARS_PATTERNS`. **T8 SafeProductCatalog e miglioramenti**: catalogo prodotto versionato di soli metadati autorizzati (guardie strutturali contro sorgenti/env/segreti/percorsi/R4) e proposte di miglioramento INERTI derivate solo da pattern sopra soglia con template chiusi (problema, evidenze, baseline, impatto, soluzione, alternative, rischi, metrica, esperimento, rollout, rollback, test); feedback `utile|non_utile|gia_risolto|troppo_rumore` persistito con effetto solo su cooldown/ranking; «accetta» registra una decisione e non tocca né codice né policy; dietro il nuovo `FLAG_TARS_IMPROVEMENTS`. **T9 flag, classi di costo e osservabilità**: classi logiche

`interactive|document_intelligence|proactive_commissa|pattern_azienda|miglioramento_crm|eval` con colonna additiva nel ledger e tetto giornaliero per classe

(`TARS_BUDGET_<CLASSE>_USD`, default 0 per il background: le pipeline deterministiche non chiamano il modello finché una sintesi non riceve budget esplicito; env invalida blocca solo la sua classe; il globale resta l'unico hard ceiling); `statisticheRun` espone l'ultimo run degradato col motivo sanificato SOLO alla direzione in `tars.stato` (diagnosi diretta del «modello non è al momento disponibile»).

T10 eval e chiusura: `pnpm eval:tars` esteso a 16 casi con cinque metriche nuove a soglia zero (autorità condizionale senza comando, pattern inventati, proposte senza evidenza, chiamate di background senza budget, rumore osservatore senza segnali); matrice test/limiti, piano eval reali e runbook rollout aggiornati. Registro azioni v1.8.0 (28 azioni). Nessun file client modificato nel mandato. Suite 124 file / 1180 test.

- **v5.27 (31/08/2026)** - Upload manuale del fascicolo commessa esteso a 250 MB e ai video MP4/MOV/WebM, con anteprima video e validazione condivisa client/server. Il trasferimento usa una rotta binaria che autentica prima di leggere il body, limita la concorrenza e non crea copie base64/JSON; lettura e download usano una rotta autenticata con supporto HTTP Range. I file oltre 10 MB non ricadono mai nel fallback JSONB quando lo storage fallisce. Allegati importati da comunicazioni/FiC e allegati ticket restano a 10 MB.
- **v5.26 (31/08/2026)** - Tars operativo T3, transizioni commessa: estratta da `commesse.update` una sola state machine canonica con adiacenza, doc gate, rollback cleanup e allineamento timeline; contratto tRPC storico e `force` del solo router invariati. Aggiunti `verifica_transizione_commissa` R0/L0 e `transizione_adiacente_commissa` R1/L2 (comando esplicito derivato dal testo utente e

legato server-side a commessa e target/direzione, entrambe `commessa.update_operational + commessa.change_state`, sede fail-closed, nessun `force`), rilettura/versione pre-effetto, ledger R1, audit prima/dopo e Undo server-side monouso solo se stato/versione e vincoli correnti coincidono. Prompt v6; catalogo 23 tool. La catena allegato Maccari resta successiva; nessun file client, flag, provider o costo modificato e nessuna operazione esterna eseguita.

- **v5.24 (30/08/2026)** - Tars v2 ATTIVATO in produzione su autorizzazione della direzione, con provider FINTO e quindi a costo zero: impostati i sette flag `FLAG_TARS`, `FLAG_TARS_READ_TOOLS`, `FLAG_TARS_REMINDERS`, `FLAG_TARS_MEMORY`, `FLAG_TARS_L2_ACTIONS`, `FLAG_TARS_PROACTIVE`, `FLAG_TARS_COMMUNICATIONS` (un solo redeploy, servizio ripartito e verificato). NON impostati `TARS_PROVIDER` (la chiave residua è ancora su Railway: lo farebbe spendere fuori perimetro), `FLAG_TARS_PROP0SALS` (richiede il rollout DI mai avviato) e i flag DI/OCR. Le azioni di Tars sono reali (promemoria, prese in carico, rinvii, memoria, letture con capability e sede), il ragionamento no: il copione dimostrativo riconosce tre forme di richiesta e per il resto risponde con un messaggio di servizio. La pagina `/tars` è visibile a tutti gli utenti della sede. Budget ai default approvati (0,10/2,00/20,00 USD), governor attivo ma inerte in assenza di provider reale. Spegnimento: rimuovere `FLAG_TARS` e ridistribuire.
- **v5.23 (30/08/2026)** - PR #2 MERGIATA su `main` (merge commit `2096a43`, 34 commit atomici conservati, CI verde) su autorizzazione della direzione, e deploy verificato senza credenziali: le procedure `tars.*` esistono nel codice distribuito (marcatore da «No procedure found» a `UNAUTHORIZED`), i router del CRM rispondono, SPA e `/produzione/*` servite, errori di autenticazione sanificati. **Tutti i flag Tars restano spenti** (fail-closed: in produzione servono variabili esplicite, nessuna imposta) e il provider reale non può nascere (mancano `TARS_PROVIDER=openai` e la chiave dedicata). Tars è implementativamente completo ma NON operativo: attivazione, gate OpenAI, eval reali e rollout restano decisioni della direzione.
- **v5.22 (30/08/2026)** - Revisione indipendente del cost hardening (due revisori sul delta del budget governor) con TUTTI i Critical e Important corretti. **Critical:** (1) il ledger su PostgreSQL non avrebbe MAI funzionato — `COALESCE(... ) FILTER (... )` è SQL invalido e `sql.unsafe()` dentro un template non produce un frammento: nell'unico ambiente in cui il provider reale può nascere ogni prenotazione sarebbe fallita, travestendo il guasto da indisponibilità del modello (ora somme in SQL statico con soli parametri legati, provate da cinque test su un PostgreSQL vero, in CI con servizio dedicato); (2) fail-OPEN sull'uso — se il provider non riporta `usage` plausibile il costo reale sarebbe 0 e la prenotazione verrebbe liberata per una chiamata comunque fatturata (ora `uncertain`, che resta contato). **Important:** stima resa un soffitto vero (2,5 caratteri/token, pessimistico per payload JSON); i limiti interni del run non aprono più il circuito globale né mentono all'utente (`ErroreLimiteRun`); `tars.stato` non espone più budget e diagnosi infrastrutturale a ogni utente autenticato; guardia strutturale estesa a `shared/` e all'override del ledger; doppio click che non produce più due addebiti; test resi mordenti (concorrenza col numero esatto e il picco, DST invernale CET, fabbrica del provider, limiti derivati dalla stima misurata). Aggiunta una guardia di rete GLOBALE alla suite (nessun test può raggiungere un host esterno, provato invocando l'adapter reale) e la matrice test/limiti (`docs/tars/matrice-test-e-limiti.md`). 13 mutation test che mordono; suite 763 test + 5 su PostgreSQL.
- **v5.21 (30/08/2026)** - Cost hardening di Tars v2 su `feature/tars-v2` (nessun deploy, flag spenti, zero chiamate reali): **budget governor** che impedisce per costruzione di superare 0,10 USD per run, 2,00 al giorno e 20,00 al mese. Confine UNICO — l'adapter espone solo `creaProviderRealeGrezzo`, importabile esclusivamente dalla fabbrica `costi/providerGovernato.ts`, con guardia strutturale che fallisce se qualcuno reintroduce una chiamata diretta, sposta il ledger in memoria o resuscita i client LLM legacy (che restano senza consumatori). Contabilità in nanodollari interi con BigInt (mai

floating point) e catalogo tariffe versionato e chiuso (gpt-5.6-terra : 2,00/0,20/12,00 USD per milione, fonte e data registrate; un modello senza tariffa attiva non parte). Ciclo **prenota → chiama → riconcilia** su ledger PostgreSQL con advisory lock globale: stima prudenziale (input a tariffa piena + `max_output_tokens` intero, che per contratto include i reasoning token), riconciliazione al costo reale che libera la quota inutilizzata, stati `reserved/settled/released/expired/uncertain` con politica conservativa (timeout ed esiti incerti restano CONTATI; 4xx e 429 rilasciati), chiave idempotente `runId:passo:tentativo`. Senza `DATABASE_URL` il ledger non è autorevole e il provider reale NON nasce. Limiti tecnici del run espliciti e motivati (8 chiamate modello, 180s, 120k caratteri di contesto). Budget esaurito = messaggio italiano controllato, nessun retry, nessun circuito aperto. Lettura amministrativa `tars.costi` (direzione-only, solo numeri) e diagnosi onesta in `tars.stato`. 38 test dedicati + 6 mutation test che mordono; misurato in test: con 21 strumenti a catalogo la prenotazione per chiamata è ≈0,03 USD, quindi il tetto per-run consente 3-7 chiamate secondo il prompt caching. Documentati il piano dei 60 eval reali (`docs/tars/piano-eval-reali.md`) e la procedura di configurazione OpenAI con progetto, service account, chiave dedicata e hard limit (gate-openai.md §6). Suite 752 test.

- **v5.20 (30/08/2026)** - Revisione INDIPENDENTE dell'intero diff Tars v2 (quattro revisori: sicurezza/authz, runtime/cache, client/convenzioni, qualità dei test) con TUTTI i rilievi Critical/Important corretti: finestra di cronologia che perdeva la domanda corrente (ora ultimi N turni), parser temporale con cinque risoluzioni silenziosamente errate eliminate (weekday vs data esplicita con verifica di coerenza; fasce che qualificano l'ora — «alle 9 di sera» = 21:00; orari a parole rifiutati invece che sostituiti dal default; «il giorno prima»; «prossimo X» di X; «e mezza»), chiave C0 con impronta della cronologia (niente riuso di domande deittiche col referente sbagliato), C1 senza errori cachati, fascicoli C3 invalidati da documenti nuovi e dal rollover di giornata (con «oggi» in Europe/Rome), percorso provider reale strutturalmente corretto (turno assistant con `function_call` nel contratto), versione del registro pagamenti come hash opaco, rate limit per principal su `tars.invia`, guardia `DATABASE_URL` sull'eval CLI, client con query gated sui flag (zero errori console a Tars spento), Select sha-dcn, invalidations post-applicazione e stato azioni onesto. Metrica L3 dell'eval ora MISURATA sul flusso reale. Proposta gate OpenAI documentata su fonti ufficiali correnti (docs/tars/gate-openai.md: raccomandato gpt-5.6-terra, budget e limiti proposti, tetto di spesa software da implementare prima della fase 1). Suite 81 file / 714 test; eval 11/11.
- **v5.19 (30/08/2026)** - Tars v2, slice T8/T9 «eval, shadow, rollout» completate nella parte costruibile su `feature/tars-v2` (nessun deploy, flag spenti): comando `pnpm eval:tars` — 11 casi deterministici col provider finto attraverso il runtime REALE che misurano attrito (L1 esplicito = 0 conferme, proposta L3 = 1 conferma umana), duplicati (0), DST onesto (caso auto-adattivo sulla prossima ultima domenica di ottobre), shaping economico e cross-sede (0 disclosure), isolamento C0 per utente, kill switch (0 effetti da spento), assenza di strumenti di approvazione, degradazione onesta e injection-come-dato — con rapporto versionato in `docs/reports/tars-eval-*.md` che DICHIARA i limiti (sintetico: la selezione strumenti e la resistenza del modello reale si misurano solo con i casi OpenAI dopo il gate) e test vitest sulle soglie critiche che blocca la CI; runbook `docs/runbooks/rollout-tars.md` con fasi 0-4, osservazione, rollback per spegnimento flag (zero migrazioni) e owner. L'ATTIVAZIONE resta interamente della direzione. Decisioni §26.38-40 nella spec. Suite 81 file / 698 test.
- **v5.18 (30/08/2026)** - Tars v2, slice T7 «memoria» completata su `feature/tars-v2` (nessun deploy, flag spenti): memoria v1 su kv `tars_memoria` con tipi CHIUSI e fonte sempre `richiesta_esplicita` (mai ipotesi del modello: regola nel prompt v4 e struttura dello strumento), perimetro `utente o sede` (quest'ultimo solo direzione), `dimentica` che INVALIDA senza cancellare (audit); strumenti `ricorda / dimentica / leggi_memorie` dietro il nuovo kill switch fail-closed `FLAG_TARS_MEMORY`; le memorie valide entrano nei run come messaggio di CONTESTO in coda (mai nel prefisso

stabile: prompt caching C2 intatto) marcate come dati non autorevoli (il CRM corrente prevale, verificato con gli strumenti); il fingerprint delle memorie entra nella chiave C0 (memoria nuova/invalidata = niente riuso di risposte). C5 ricerca semantica DIFFERITA con decisione registrata (§25.36): gli embeddings sono chiamate reali al provider, quindi arrivano solo col gate chiave/budget della direzione. Isolamento cross-utente e cross-sede provato sul contenuto effettivo delle richieste al provider. 8 test + 3 mutation che mordono; suite 80 file / 697 test.

- **v5.17 (30/08/2026)** - Tars v2, slice T6 «documenti e comunicazioni» completata su feature/tars-v2 (nessun deploy, flag spenti): strumento L2 `analizza_conferma_ordine` che riusa l'UNICA fonte `documenti/analisiOrdine.analizzaConfermaPerOrdine` (coerenza fascicolo/collegamento estratta dal router, contratto API invariato, idempotenza per firma, run append-only senza undo dichiarato, errori d'infrastruttura mai grezzi); strumento L0 `leggi_comunicazioni` con metadati ed ESTRATTI brevi (240 caratteri) per commessa/cliente della sede — mai corpi integrali nel contesto del modello (injection e PII ridotte; il contenuto è un DATO, provato con test di injection); NESSUN invio da Tars per decisione registrata (§24.30 della spec): il dominio comunicazioni è un archivio dei canali senza invio programmatico — l'«invio L4» resta un gate della direzione (nuova integrazione esterna); residui legacy `tars_*` su comunicazioni CONGELATI (mai letti/scritti dal nuovo Tars). Profilo `l3-v2`. 6 test + 3 mutation che mordono; regressione D7 33/33; suite 79 file / 689 test.
- **v5.16 (30/08/2026)** - Tars v2, slice T5 «azioni L2 + gateway L3 a conferma unica» completata su feature/tars-v2 (nessun deploy, flag spenti): strumenti L2 `prendi_in_carico_caso / rinvia_caso` sul servizio esistente del Centro Azioni (`transitionActionCase : authz mine/direzione`, fingerprint anti-stale, eventi come audit) con esecuzione DIRETTA su richiesta esplicita (zero conferme, 2 soli turni provati) e rinvio tramite il parser temporale T2; nuovo kill switch fail-closed `FLAG_TARS_L2_ACTIONS`; L3 attraverso il gateway D7 (decisione T0.6): coerenza documento↔ordine e generazione estratte nell'unica fonte `generaDaOrdineEDocumento` (il router `proposte.genera` la richiama dopo la sua authz), strumento `proponi_data_consegna` (direzione + `fornitore.manage_ordini` + interruttori `documentIntelligence/proposte/tarsProposals` TUTTI richiesti) che genera la proposta INERTE con anteprima; il modello NON ha alcuno strumento di approvazione in alcun profilo (L5, sequenza conferma→approva→applica impossibile per costruzione); UNICA conferma umana via nuova `proposte.approvaEApplica` (stessa doppia capability `documento.approve_proposals` + capability finale, approvazione+applicazione in sequenza, idempotente al doppio click, freschezza che marca `obsoleta` una proposta su dati cambiati) con bottone «Approva e applica» nella chat (effetto esatto, assunzioni, evidenze con pagina). Prompt `v3`, profilo `l3-v1`. Decisioni §23.25-29 nella spec; 11 test + 3 mutation che mordono; suite 78 file / 683 test; flusso intero provato nel browser (proposta→un click→solo la data di consegna cambia).
- **v5.15 (30/08/2026)** - Tars v2, slice T4 «briefing, situazioni, proattività shadow» completata su feature/tars-v2 (nessun deploy, flag spenti): briefing DETERMINISTICO a richiesta (zero token, zero scritture di dominio, il modello non partecipa) con promemoria di oggi, casi «mine» del Centro Azioni e segnalazioni di sede; proattività SHADOW senza emissioni — due rilevatori (ordine in ritardo; consegna prevista dopo la data confermata al cliente) le cui segnalazioni si AGGANCIANO ai casi aperti del Centro Azioni per commessa invece di duplicarli (solo un booleano «già seguito») — con telemetria del rumore registrata come run `proattivit -shadow` (contatori: segnalazioni, agganciate, promemoria, casi); sezione segnalazioni dietro `FLAG_TARS_PROACTIVE`, endpoint `tars.briefing` dietro `FLAG_TARS + FLAG_TARS_READ_T00LS`; blocco «Situazione di oggi» nella pagina `/tars`. Nessun worker nuovo (decisioni §22.21-24 nella spec). 7 test + 2 mutation test che mordono; suite 77 file / 673 test.

- v5.14 (30/08/2026)** - Tars v2, slice T3 «fascicoli, cache C3/C4, pannello contestuale» completata su `feature/tars-v2` (nessun deploy, flag spenti): fascicolo sintetico della commessa al «pavimento» di capability `commessa.read` — fatti operativi, gate, ordini SENZA importi, domande aperte deterministiche (gate mancante, consegna prevista dopo la data confermata, ordini in ritardo o senza data) — condivisibile a livello sede per costruzione con test anti-leak sul payload; store persistente `tars_cache_entries` (PG additivo + fallback memoria dichiarato) e registro versioni (`updatedAt` per entità, hash id+versione per le liste: un ordine NUOVO invalida); riuso solo a versioni identiche, «stale» servito e dichiarato solo su errore di ricostruzione, mai per azioni; C0 v2 nell'orchestratore (il riuso di una risposta richiede TTL valido E versioni delle entità osservate ancora correnti; riferimenti non sondabili = riuso negato); strumento `leggi_fascicolo_commissa` e pannello contestuale in `CommessaDetail` via query `tars.fascicolo` (zero run del modello, zero token; con flag spenti il pannello non esiste). Decisioni registrate nella spec §21 (15-20), inclusa la scelta motivata di NON avvolgere in cache C4 le letture in-memory (microsecondi: rischio senza guadagno). 8 test + 3 mutation test che mordono; suite 76 file / 666 test.
- v5.13 (30/08/2026)** - Tars v2, slice T2 «promemoria e azioni personali L1» completata su `feature/tars-v2` (nessun deploy, flag spenti): strumenti `crea/sposta/annulla/completa_promemoria` e agenda `leggi_promemoria` costruiti SOPRA `server/reminders` esistente (decisioni T2 registrate nella spec §20: estensioni additive `listPersonal / get / listEvents`, nessuno scheduler nuovo, consegna via worker esistente con claim atomico); parser temporale deterministico italiano server-side (`server/tars/tempo.ts`) con default aziendali DICHIARATI (mattina 09:00, pomeriggio 15:00, sera 18:00) e due semantiche (calendario Europe/Rome con DST deciso da `reminders/time.ts` — orario inesistente o ambiguo RIFIUTATO con motivo, mai indovinato — e durata esatta per «tra due ore»); idempotenza via `canonicalKey` (doppio invio = «già esistente», zero duplicati) con catena deterministica di ricreazione dopo annullo; regola d'attrito nel prompt v2 e provata nei test: richiesta esplicita personale = ZERO conferme (2 soli turni), al massimo UNA precisazione su ambiguità materiale; esito azione con prima/dopo, auditId su `promemoria_eventi` e Annulla a un click nella UI (router `promemoria.cancel`, senza passare dal modello); kill switch `FLAG_TARS_REMINDERS` a tre strati con mutation test che mordono su ogni strato. Suite 75 file / 658 test; verifica browser desktop e 390x844.
- v5.12 (29/08/2026)** - Merge della PR #1 su `main` (84717e2 , 31 commit conservati) autorizzato dalla direzione e verificato in produzione: nuovo build live (marcatore `platform.interruttori`), 10 router sondati vivi, SPA e fallback `/produzione/*` OK, auth con errori sanificati, kill switch DI/OCR/proposte SPENTI (default fail-closed, nessuna variabile `FLAG_*` impostata). Avviato Tars v2: branch `feature/tars-v2`, T0 in `docs/tars/architettura-tars-v2.md` (§54 aggiornata a progetto attivo). Decisioni T0: orchestratore unico, provider dietro DI con fake deterministico e ZERO chiamate OpenAI reali fino al gate chiave/budget, `llm.ts` superseded, gateway proposte D7 esteso come gateway L3/L4 di Tars, riuso integrale di reminders/eventi/Centro Azioni.
- v5.11 (29/08/2026)** - Chiusura della PR #1: (1) `pnpm storage:check` è ora SOLA LETTURA (configurazione + GET su chiave inesistente); la sonda put/get/delete è lo script separato `storage:probe-write` con flag `--scrivi` obbligatorio, e `server/_core/checklistReadOnly.test.ts` impedisce staticamente e comportamentalmente che la checklist read-only torni a contenere scritture non dichiarate. (2) Prima CI GitHub (`.github/workflows/ci.yml`): install deterministico, typecheck, test mirati DI, suite completa, build, verifica di Tesseract/Poppler e lingue ita/eng/deu sul runner, working tree pulito dopo i test; job eval separato e non bloccante con artifact. (3) Immagine Nixpacks misurata contro la baseline di `main` (f0bb919): l'unico layer di produzione aggiunto è l'apt OCR da 163 MB unpacked (~+7%); nessuna dipendenza npm nuova; le devDependencies nell'immagine sono lo status quo di main (ottimizzazione futura possibile, non applicata). (4) Risposte documentate ai rilievi

Graphify: `annunciaAzioniAutonome / annunciaDecisioneProposta` rimossi con zero consumer già su main (annunci del vecchio Tars), `annunciaAssegnazione` vivo e conservato; `ComponentShowcase` rimosso in 1310001 senza route/import residui; `daSaldare` decisione di policy (§37.5) con test dedicati.

- **v5.10 (29/08/2026)** - Revisione indipendente dell'intero diff (quattro revisori: correttezza server, sicurezza authz, client/convenzioni, qualità) e correzione di TUTTI i rilievi Critical/Important più i minori a basso costo: confini obbligatori su ogni ricerca di riferimento (ORD-10 mai dentro ORD-100, anche per riferimentoOrdine/fornitoreCitato); segnale «totale coincidente» calcolato solo per chi ha `economia.read` (chiuso l'oracolo di uguaglianza sugli importi); kill switch FAIL-CLOSED (accesi solo con `NODE_ENV development/test`) e spostati nella base procedure (`procedureConInterruttore`); `proposte.genera` riverifica la coerenza VIVA documento↔ordine (un collegamento annullato non genera più proposte); idempotenza dei run estesa al contenuto dell'ordine (`ordineFirma`) con storico limitato a 10 run per coppia e guardia anti-doppio-click; date di calendario validate (31/02 rifiutato), importi con punteggiatura e migliaia all'italiana, priorità della consegna sulla spedizione; motivo per-proposta nella UI (audit corretto), importi via helper euro, dialog con descrizione accessibile; predicato del conflitto posa condiviso fra Centro Azioni e avviso post-applicazione; `/produzione/*` coperto dal redirect. Consapevolmente NON cambiati: fingerprint saldo (decisione privacy slice 2), nessun vincolo quattro-occhi proponente/approvatore (doppia capability è il contratto), dedup `parseEuro` in `shared/` (candidato a bonifica).
- **v5.9 (29/08/2026)** - Release hardening: kill switch `FLAG_DOCUMENT_INTELLIGENCE / FLAG_PROPOSTE / FLAG_OCR` (§19.4), disattivati di default in produzione e verificati non aggirabili via API (`server/platform/interruttori.test.ts`); query `platform.interruttori` per la UI; runbook `docs/runbooks/rollout-document-intelligence.md` con rollout progressivo, rollback via flag, checklist post-deploy e nota sulla ri-notifica saldo una tantum.
- **v5.8 (29/08/2026)** - Release hardening: rimossa la pagina UI `/produzione` (inutilizzata) con la card dell'hub Gestione; la vecchia route reindirizza al Board (`/kanban` , colonna Produzione). Backend BOM/fasi/NC conservato e annotato come candidato a bonifica (dati potenzialmente presenti negli store, contratto di dominio §20). Stati commessa, trigger §6.4, gate e magazzino invariati, con test dedicati.
- **v5.7 (29/08/2026)** - Quinta slice della Document Intelligence (§19.4): framework di valutazione (`server/documenti/eval/` , `pnpm eval:documenti`) con 16 fixture sintetiche (nativi, scansioni vere anche storte/a bassa risoluzione, tabella spezzata, ambiguità, codici simili, injection, duplicato, corrotto, timeout) e metriche separate per campo/collegamento/differenze/OCR/tempi/revisione. Nessuna soglia dichiarata sui sintetici; predisposto `casi-reali/` (gitignored) per i documenti veri anonimizzati. L'eval ha scoperto e fatto correggere il match senza confini dei riferimenti (ORD-10 dentro ORD-100, FIN-100 dentro FIN-1000).
- **v5.6 (29/08/2026)** - Quarta slice della Document Intelligence (§19.4): OCR locale con Tesseract 5 come fallback esplicito per i PDF scansionati — rendering pagina per pagina via poppler, TSV con confidenze, lingue configurabili (`OCR_LINGUE` , default `ita+eng` , `deu` predisposto), limiti su dimensione/pagine/tempo/concorrenza, esiti espliciti per binario mancante/lingua mancante/timeout/OCR fallito, marcatura «da verificare» sotto soglia di confidenza, firma OCR nell'idempotenza dei run. Nessun servizio cloud; immagine di deploy +~60-80 MB (apt: `tesseract-ocr ita/eng/deu, poppler-utils`).
- **v5.5 (29/08/2026)** - Terza slice della Document Intelligence (§19.4): approval gateway generale e tipizzato (`server/proposte/`) con registro chiuso delle azioni, stati espliciti (proposta/approvata/rifiutata/applicata/fallita/annullata/scaduta/obsoleta), idempotenza, scadenza e cronologia append-only. Prima azione applicabile: aggiornare la data di consegna prevista dell'ordine fornitore, con dop-

pia capability (`documento.approve_proposals` + `fornitore.manage_ordini`), verifica di freschezza e conferma esplicita in due passi. Il conflitto con la posa diventa un caso del Centro Azioni (`consegna_fornitore`); nessuna ripianificazione automatica.

- **v5.4 (28/08/2026)** - Seconda slice della Document Intelligence (§19.4): collegamento assistito documento→ordine con candidati deterministici a punteggio spiegabile (codice ordine > commessa > fornitore > articoli > date > totali), quattro stati espliciti (certa/candidata/ambigua/assente), conferma umana obbligatoria, rifiuti e annullamenti auditati, idempotenza e duplicati per impronta, capability `commessa.manage_documents` senza ruoli hardcoded. Il collegamento non modifica documento, ordine o commessa.
- **v5.3 (28/08/2026)** - Prima slice della Document Intelligence (§19.4): analisi deterministica delle conferme d'ordine PDF dalla scheda ordine — registro parser, estrazione con evidenze per pagina, confronto con l'ordine per gravità, run idempotenti con impronta e versioni. Nessuna scrittura su dati autorevoli. Limite dichiarato: le scansioni senza testo producono uno stato esplicito e non vengono comprese finché non esiste un OCR.
- **v5.2 (28/08/2026)** - Slice 2 «dati economici dietro capability» (§37.5): registro pagamenti, costi e margine viaggiano solo con `pagamento.read` / `economia.read` (payload sagomati, mai errori sulla parte operativa); scritture acconti dietro `pagamento.record` con override individuale auditato; `/pagamenti` riservata; Board, liste, Dashboard, Centro Azioni e notifiche senza importi (bit `daSaldare` e versione del registro al posto delle cifre); `permessi.mie` per la parità di policy nella UI. La matrice vale identica in ogni `policyMode` (`legacyAllowed: "capability"`).
- **v5.1 (28/08/2026)** - Riconciliazione documentale post-rimozione: §51 e §53 riscritti sul comportamento corrente (nessuna classificazione automatica, nessuna proposta, flag di sola lettura, limite noto sugli allegati WhatsApp); §40.4-40.5 allineati alla riconciliazione senza agente; nuova sezione §54 con la visione del futuro agente, marcata non implementata, inclusa la Document Intelligence decisa dalla direzione (§54.6, decisione D7): comprensione verificabile dei documenti — priorità alle conferme d'ordine PDF — da realizzare dopo la slice authz e prima delle capacità operative avanzate; correzioni minori (poller IMAP a 5 minuti, route legacy). Il PDF `PRD_infissi_ops_v4.pdf` resta il riferimento storico della v4.
- **v5.0 (28/08/2026)** - Tars rimosso per intero su richiesta della direzione: va rifatto da zero. L'infrastruttura comunicazioni (tabella, IMAP, WhatsApp, caselle, matcher, filtri) è stata spostata in `server/comunicazioni/` perché non era l'agente; la Conoscenza aziendale ha un router suo. I dati dell'agente sono stati esportati e cancellati. Smistamento automatico, proposte, classificazione AI dei costi e analisi dei casi non esistono più (§50).
- **v4.33 (26/08/2026)** - Un promemoria con data e ora complete genera direttamente una sola proposta approvabile; Tars chiede soltanto i dati temporali mancanti e non aggiunge una conferma preliminare (§50.11).
- **v4.32 (26/08/2026)** - Le proposte Tars pendenti o fallite possono essere eliminate dalla vista personale con conferma; restano nello store per audit e deduplicazione e continuano a essere visibili agli altri utenti autorizzati (§50.1).
- **v4.31 (26/08/2026)** - FiC diventa fonte autorevole per rate, date e storni; il pattuito resta nel CRM, i movimenti FiC sono idempotenti e auditabili, Tars propone soltanto correzioni dei manuali e i PDF fattura vengono collegati con retry sicuro (§37, §40.4, §50).
- **v4.30 (26/08/2026)** - Tars riconosce le richieste di promemoria personali, chiede sempre data e ora, attende l'approvazione del richiedente e consegna popup e notifica nel CRM aperto con completamento e posticipo (§25.6, §50.11).
- **v4.29 (26/08/2026)** - Il workspace Email amplia il lettore, elimina i troncamenti delle informazioni operative nel dettaglio e attiva automaticamente il focus quando si usa Tars o sono presenti proposte

pendenti (§51.6).

- **v4.28 (25/08/2026)** - Gli allegati WhatsApp in ingresso partecipano allo smistamento Tars e possono essere proposti per l'archiviazione nel fascicolo con gli stessi controlli, storage e deduplica degli allegati Email; dalla chat la sorgente può essere indicata per numero o indirizzo Email e la destinazione per cliente/commessa (§50.2, §51.2-51.7).
- **v4.27 (25/08/2026)** - La sincronizzazione FiC espone stato e orario di avvio per sede, può essere fermata dalla pagina Integrazioni e applica timeout alle richieste e al giro completo (§40.3).
- **v4.26 (25/08/2026)** - La sincronizzazione FiC recupera in modo idempotente i PDF mancanti delle fatture già collegate; il workspace Email guadagna un lettore più ampio, modalità focus e vista singola sotto 1280 px (§40.4, §51.6).
- **v4.25 (25/08/2026)** - Gli esperimenti Tars accettano feedback umano tracciato e correzioni prima dell'approvazione (§50.6, §53.4).
- **v4.24 (25/08/2026)** - Gli allegati Email operativi possono essere proposti da Tars e archiviati nel fascicolo commessa con storage, checksum e deduplica canonici (§51.3, §51.5).
- **v4.23 (25/08/2026)** - Il bootstrap riallinea automaticamente le commesse storiche rimaste indietro rispetto alla Timeline, con riconciliazione idempotente e solo in avanti (§35.2).
- **v4.22 (25/08/2026)** - Le milestone della Timeline ordine avanzano automaticamente la commessa nel Board usando la state machine e il doc gate canonici; date/note e riaperture non spostano la commessa (§7.4, §35.2).
- **v4.21 (25/08/2026)** - Il rollout Tars fallisce chiuso: shadow senza notifiche/push, active bloccato per contesto/planner/semantica incompleti, ACL proposte per ownership, deleghe rivalidate e stream SSE senza finestra replay/live (§53).
- **v4.20 (25/08/2026)** - Introdotte le fondamenta di eventi, notifiche realtime, capability, contesto incrementale, planner persistente, indice ACL-aware, outcome e diagnostica; attivazione subordinata ai gate produzione (§53).
- **v4.19 (25/08/2026)** - La chat Tars può proporre cliente e prima commessa anche senza una comunicazione sorgente; la creazione resta unica, deduplicata e subordinata all'approvazione (§50.2, §50.9).
- **v4.18 (24/08/2026)** - Centro Azioni persistente con deduplica dei segnali, workflow personale, audit, modalità legacy/shadow/active, campanella compatta e analisi Tars asincrona/cachata senza OpenAI sui percorsi di lettura (§25, §50.9).
- **v4.17 (23/08/2026)** - Ricollegamento WhatsApp coexistence verificato in produzione con storico completato, echo live e outbound precedenti preservati; documentata la reinstallazione sicura di WhatsApp Business soltanto dopo backup verificato (§51.9).
- **v4.16 (23/08/2026)** - Il tool `cerca_comunicazioni` espone direzione, autore, controparte, mittente e destinatario; il prompt obbliga Tars a distinguere cliente e ufficio anche nello storico WhatsApp outbound (§50.3, §51.8).
- **v4.15 (23/08/2026)** - Embedded Signup riconosce la casella WhatsApp storica dal numero aziendale salvato nei destinatari e ne riusa l'id interno dopo uno scollegamento, preservando conversazioni, alias e collegamenti (§51.7-51.8).
- **v4.14 (23/08/2026)** - Corretto lo storico WhatsApp outbound usando `history[].threads[].id`; richiesta, progresso e completamento della sincronizzazione sono stati separati e resi visibili in UI. I record outbound legacy senza controparte vengono eliminati una tantum per consentire una reimportazione corretta. I gate dello smistamento Tars producono log solo sulle transizioni di blocco/ripresa (§50.7, §51.7-51.9).
- **v4.13 (22/08/2026)** - `/tars` diventa Command Center con vista Oggi, ranking deterministico, prove, proposte, analisi, chat e registro; il brief non chiama OpenAI e non consuma token all'apertura. La

configurazione WhatsApp espone diagnostica privacy-safe per `smb_message_echoes` e duplicati (§50.9, §51.4-51.7).

- **v4.12 (19/08/2026)** - L'analisi commessa non chiude più in silenzio: proposta, domanda con opzioni oppure `nessuna_azione` motivata che nomina i fatti verificati. Il server rifiuta la chiusura muta su `on_demand`; i trigger automatici restano liberi (§50.8).
- **v4.11 (19/08/2026)** - Collegare una comunicazione a una commessa la porta in Gestite (collegamento manuale e proposte Tars approvate); lo scollegamento la riapre; il match automatico dell'arrivo resta nella coda operativa. Backfill una tantum delle collegate già viste (§51.1).
- **v4.10 (19/08/2026)** - Smistamento Tars ridotto a prompt specializzato e 7 strumenti (-78% token fissi), cache per sede/profilo/modello, `gpt-5.6-luna` opzionale, errori OpenAI sanitizzati e diagnostica token/cache/costi corretta (§50-51).
- **v4.9 (19/08/2026)** - Tars migrato a OpenAI Responses API con `gpt-5.6-sol` per le richieste umane, `gpt-5.6-terra` per gli automatismi, prompt caching per profilo/modello e chiusura forzata dopo il budget strumenti (§50-51).
- **v4.8 (19/08/2026)** - Coda Comunicazioni Tars resa lossless: continuazione automatica oltre 10 mail, trigger concorrenti preservati, retry API non annullabile, recupero periodico e dopo bootstrap, riattivazione da config/budget e stato d'attesa spiegato nella UI (§50-51).
- **v4.7 (19/08/2026)** - Ogni nuova email passa dalla classificazione strutturata di Tars; il filtro locale fornisce solo segnali, le esclusioni richiedono confidenza alta, i dubbi restano visibili e le mail saltate vengono ritentate (§50-51).
- **v4.6 (18/08/2026)** — Richieste di preventivo e opportunità protette da header spam e regole mittente; newsletter inutili ricondotte allo spam; Tars chiede l'assegnatario prima di proporre cliente e commessa e conserva il contesto nel seguito (§50-51).
- **v4.5 (18/08/2026)** — Comunicazioni con triage multilivello, filtro anti-spam/offerte prima dell'AI, regole mittente, azioni multiple e gestione Tars della singola mail con creazione lead approvata (§50-51).
- **v4.4 (18/08/2026)** — Tars con quadro aziendale, lettura controllata di organizzazione/produzione/qualità/documenti, audit periodico dei processi, proposta di miglioramenti misurabili, deduplica per chiave d'azione e Inbox operativa (§50).
- **v4.3 (14/08/2026)** — Inbox Comunicazioni email/WhatsApp (§51); Tars con fascicolo commessa, profili tool, prompt caching e cache per run (§50); OAuth FiC Authorization Code con refresh (§40.3); backup Drive compatibile con `storageKey`; probe e runbook R2 (§47); bootstrap utenti senza password fisse; sistema visuale caldo e code splitting (§52).
- **v4.2 (06/08/2026)** — Storage documenti su object storage con migrazione verificata (§47); marginalità e registro costi fornitore dentro la commessa (§45); post-vendita v2: solleciti, interventi pianificabili dal ticket, ticket senza commessa, ricerca, stato `risolto` ritirato (§13); squadre di posa visibili e assegnabili alla commessa (§46); prodotti dichiarati in creazione e modificabili dopo (§44); data di apertura in scheda e in lista (§9); formato e parsing unici degli importi (§48); correzioni notevoli (§49).
- **v4.1 (23/07/2026)** — Pagina Pagamenti (§37.4) con registrazione rapida degli acconti; acconti modificabili in place (§37.1-37.2); date programmabili sugli step della timeline, pensate per l'Appuntamento Posa (§35.2-bis); Magazzino a 2 tile per riga con 4 prodotti visibili, badge fornitore e filtro fornitore a tendina (§36.3); form cliente con Ragione sociale e Sede legale per i non privati (§5.2); responsive mobile su header schede e tabelle di lista (§29.3).
- **v4.0 (16/07/2026)** — Rimossa la Classifica venditori (§23). Multi-sede con isolamento completo; re-design UI (board v2, calendario mese-default con chip pieni, timeline note post-it, dashboard personalizzata); Magazzino (tile+popup, ordini, lead time, dropdown fornitori); registro acconti; notifiche

v2 con stato lettura; sincronizzazione Google Calendar export+import; backup notturno Google Drive (OAuth drive.file); Fatture in Cloud sync clienti; WhatsApp deep link; scheda cliente PDF; hardening (scrypt versionato, CSRF, SSRF guard, trust proxy, mascheramento segreti); migrazione dati 2026 (§43).

- v3.0 — Riallineamento completo del PRD al codice corrente: dual address, tipoDetrazione, dataConsegnaIndicativa, soft-archive, Archivio, Classifica venditori, doc-gate con bypass, hardening sicurezza (scrypt, JWT fail-hard, mimeType allowlist, rate-limit login, security headers, session eviction, logout server-side), assegnazione utente modificabile, preventivatori Fivizzanese e Punto del Serramento, Planning con joined info.
 - v2.x — Versione precedente (PDF allegato in repo come riferimento storico).
 - v1 — Documento originale.
-

Parte II — Moduli introdotti dopo la v3

34. Multi-sede

34.1 Modello

- Store `sedi`: { `id`, `nome`, `indirizzo?`, `citta?`, `attiva` } (seed prima sede "La Spezia").
- Ogni entità business porta `sedeId` (backfill = 1 per i record pre-esistenti).
- Gli utenti hanno `sediIds`: `number[]` (accesso multi-sede).

34.2 Risoluzione della sede attiva

- Cookie/claim `active_sede` → `ctx.sedeId` su ogni richiesta tRPC.
- **SedeSwitcher** nella sidebar (in alto): cambia la sede attiva; tutte le query si rifanno sul nuovo scope.

34.3 Isolamento

- Ogni `list` filtra per `sedeId`; ogni mutation su entità esistente passa da `assertSedeScope` (404 anche in caso di id valido di altra sede — nessun information leak).
- Entità figlie (documenti, allegati, timeline, magazzino) ereditano lo scope dalla commessa/ticket padre.
- Pagina `/sedi` (direzione-only) per creare/gestire sedi; assegnazione `sediIds` dalla pagina Utenti.

35. Timeline ordine (scheda commessa)

35.1 Struttura

16 step fissi per commessa (store `timeline_steps`, creati lazy alla prima lettura), raggruppati nelle 4 fasi del board: Rilievo Misure, Firma Contratto, Fatturazione, 1° Acconto, Conferma Ordine, Acconto Fornitore, Data Spedizione Prevista, Pagamento Merce Pronta, 2° Acconto, Data Consegna Merce, Appuntamento Posa, Lista Merce Posata, DDT Posa, Finiture, Saldo, Recensione.

«Invio Fattura al Cliente» è stato ritirato il 02/09/2026: emettere la fattura significa già mandarla, e lo step restava aperto per sempre.

«Ordine Merce al Fornitore» è stato fuso in «Conferma Ordine Fornitore» il 03/09/2026: per chi lavora è lo stesso gesto. Sopravvive la conferma perché è il documento che porta il costo imponibile del margine e che il gate di `da_ordinare` richiede; con lei si sposta la milestone verso `produzione`, quindi la commessa avanza quando il fornitore ha risposto, non quando l'ordine è partito.

Le timeline già salvate vengono allineate al bootstrap: i passi fusi si travasano la spunta (data, esecutore e nota) sul passo che resta, i passi ritirati spariscono e la numerazione torna 1..n senza buchi.

35.2 Interazione

- Barra avanzamento ($N/16 + \%$). Fasi collassabili; **la fase con lo step corrente e quelle contenenti note si aprono da sole.**
- Step corrente evidenziato (sfondo primary tenue) con bottone **Completa** one-click. Dialog di modifica per data, utente esecutore (SearchSelect) e note; step completato riapribile.
- Campi step: `stato (da_fare|in_corso|completato)`, `dataCompletamento`, `utente`, `note`, `allegato?`.
- Il primo completamento delle milestone sincronizza lo stato della commessa: **1 Rilievo Misure** → `misure_esecutive`; **2 Firma Contratto** → `aggiornamento_contratto`; **3 Fatturazione** → `fatture_pagamento`; **4 Primo Acconto** → `da_ordinare`; **5 Conferma Ordine** → `produzione`; **8 Merce pronta** → `ordini_ultimazione`; **9 Secondo Acconto** → `attesa_posa`; **13 DDT Posa** → `finiture_saldo`; **15 Saldo** → `interventi_regolazioni`; **16 Recensione** → `archiviata`.
- La sincronizzazione usa la mutation canonica `commesse.update`: permessi, transizioni singole e dogate sono identici al Board. Se manca un file, lo step resta incompleto e la UI offre **Procedi comunque**; confermando ripete entrambe le operazioni con `force: true`.
- Uno step intermedio non sposta il Board. Completare di nuovo una milestone già registrata o riapirla non arretra la commessa; una timeline rimasta indietro rispetto al Board può quindi essere riallineata senza regressioni.
- Dopo il bootstrap, una riconciliazione idempotente esamina anche lo storico: per ogni commessa ricava la milestone completata più avanzata e porta il Board almeno allo stato corrispondente. Il recupero è soltanto in avanti, non riapre stati, non arretra commesse più avanzate e salva unicamente quando trova disallineamenti.

35.2-bis Date programmate (appuntamenti futuri)

Il dialog dello step espone un campo «**Data (appuntamento o completamento)**» e due azioni distinte:

- «**Salva data**» — memorizza data/utente/note **senza** completare lo step. Serve a programmare in anticipo, tipicamente l'**Appuntamento Posa**, ma vale per qualsiasi step futuro.
- «**Segna come completato**» — completa lo step usando la data scelta (non più forzata a oggi).

Uno step non completato con data valorizzata mostra in riga una scritta blu «¹⁷ gg/mm/aaaa • **utente**», visivamente distinta dai metadati grigi degli step già completati.

35.3 Note come cittadino di prima classe

- Le note renderizzano come **post-it ambra** (icona + 12 px, `whitespace-pre-line` : le note multiriga migrate dai To Do conservano a-capo e separatori "— — —").
- L'header di ogni fase mostra un badge ambra "📝 N" con il conteggio note.
- Le date di completamento sono formattate it-IT.

35.4 Collegamento con il Board (bidirezionale)

Completare una milestone della timeline avanza la commessa sul Board tramite la stessa mutation, quindi con gli stessi permessi, la stessa state machine a passo singolo e lo stesso doc gate. Dal 26/08/2026 vale anche il verso opposto: avanzare la commessa sul Board completa ogni milestone il cui stato di riferimento è stato raggiunto o superato.

L'allineamento è **solo in avanti** e idempotente. Arretrare la commessa non riapre gli step: quel lavoro è stato fatto, e riaprirlo cancellerebbe data e autore del completamento. Un errore nell'allineamento non annulla l'avanzamento già salvato.

36. Magazzino (/magazzino)

36.1 Scope

Prodotti fisici in arrivo/da magazzino **per commessa**. Eleggibili solo commesse **da produzione in poi** (incluso; escluse archiviate) — gate applicato sia lato server (create) sia nel filtro pagina.

36.2 Modello (magazzino_prodotti)

```
{ id, sedeId, commessaId, nome, quantita, fornitore?, numeroOrdine?, dataOrdine?, dataConsegna?, arrivato, note?, createdAt, updatedAt }.
```

- **Fornitore** da dropdown fisso aziendale: Wnd, Oknoplast, Alias, Pail, Primed, HenryGlass, Palmieri, Errecci, Fivizzanese, Oskura, Korus, Punto del Serramento, Kopern, Citea, Cerrato, Brianzatende, Seraplastic, St Scale, Sharknet (valori legacy liberi restano selezionabili).
- **Lead time** = giorni `dataOrdine → dataConsegna`, mostrato per prodotto (🕒 N gg) e come **KPI medio** sugli arrivati.
- Cascade: eliminando una commessa si eliminano i suoi prodotti.

36.3 UI

- **KPI**: Prodotti, In arrivo, Arrivati, Lead time medio.
- **Strip "Prossime consegne"**: le 5 consegne pendenti più vicine cross-commessa (rosse se scadute); il click apre il popup della commessa.
- **Ricerca + filtri**: chip di stato (Tutte / In arrivo / In ritardo / Arrivati) affiancati da un **menu a tendina dei fornitori** («Tutti i fornitori» + i 19 dell'elenco) che filtra le commesse contenenti almeno un prodotto di quel fornitore. Il vecchio chip «Con prodotti» è stato sostituito da questo dropdown.
- **Griglia di tile (2 per riga su desktop, 1 su mobile)**: codice, StatoChip, cliente (17 px bold), città, **primi 4 prodotti** colorati per stato (✓ verde arrivato, rosso in ritardo con data corta, grigio in arrivo)

ciascuno con **mini-badge fornitore** accanto alla data, "+N altri prodotti", badge "N/M arrivati". Bordo rosso (ritardi) / verde (tutto arrivato). Ordinamento per urgenza (ritardi → consegna più vicina).

- **Popup dettaglio** (click sul tile, `max-w-6xl`): header con codice/cliente/stato/città + "Apri commessa" + badge; righe prodotto **tutte editabili inline** (quantità, fornitore, n° ordine, date, switch arrivato, nota ghost click-to-edit con blur-save); form "Aggiungi prodotto" su due righe.

36.4 Board

Le card del Kanban mostrano il blocco prodotti (vedi §11.2).

37. Pagamenti — registro acconti

37.1 Modello

Su ogni commessa: `importoTotale` (pattuito) + `pagamenti[]` embedded. Ogni movimento include almeno `{ id, importo, data?, metodo?, note?, origine, stato, createdAt, updatedAt }`; i movimenti FiC conservano inoltre documento, rata, chiave sorgente, stato remoto e date di sync/storno.

- `importoTotale` è un dato contrattuale CRM: nessuna fattura lo propone o lo sovrascrive.
- `importoIncassato` è **sempre ricalcolato dal server** come somma dei soli pagamenti `attivo`; gli `stornato` restano in audit ma valgono zero. Board, dashboard e notifiche usano lo stesso derivato.
- Gli acconti `origine = manuale` sono modificabili e rimovibili; i movimenti `origine = fic` sono immutabili dalle mutation manuali.
- Backfill: record legacy con incassato secco → unico acconto "Importo importato".

37.2 Card "Pagamenti" (scheda commessa)

- Totale pattuito editabile inline (blur-save), barra "% incassato · € N", **Residuo** grande (ambra finché > 0, verde a saldo).
- Registro acconti ordinato per data: data, importo bold, badge metodo, origine `Manuale / FiC`, eventuale `Stornato`, riferimento fattura e nota. Matita e cestino compaiono soltanto sui manuali; gli storni restano visibili con enfasi ridotta. Al salvataggio di un manuale residuo, barra, chip del board, dashboard e notifiche si aggiornano dal derivato server.
- **Chips rapide 50% / 40% / 10%** del totale (il piano di pagamento tipico), cappate al residuo: un click precompila l'importo nel form di registrazione (data default oggi).
- Accent warning sulla card finché c'è residuo.

37.3 Propagazione

- Board: chip "Da saldare" senza importo nelle fasi finali (§11.2).
- Dashboard: voce "Da incassare" nel feed personalizzato (§26.2), con importo solo per chi ha `pagamento.read`.
- Notifiche e Centro Azioni: il testo condiviso non contiene importi («Saldo residuo da incassare»); id e fingerprint usano la **versione del registro** (conteggio movimenti attivi + timestamp ultima modifica, `versioneRegistroPagamenti`), così un incasso parziale ri-notifica e risveglia il caso senza che dall'identificativo si possa ricostruire una cifra.

37.5 Autorizzazioni sui dati economici (28/08/2026)

La matrice confermata dalla direzione (dettagli e decisioni in `docs/reports/slice-2-authz-economia-proposta.md`) è applicata **lato server** in ogni `policyMode`; la UI è solo la seconda protezione:

- il registro `pagamenti[]` richiede `pagamento.read`; `costi[]`, `costoPosaStimato` e il margine richiedono `economia.read`. `commesse.byId` e le risposte delle mutation **omettono** i campi non autorizzati (nessun errore: la parte operativa resta usabile, con `nPagamenti` come conteggio);
- la sintesi della scheda — pattuito, incassato, residuo, piano rate — resta visibile a chi lavora la commessa;
- `commesse.list` e `commesse.byPriorita` non trasmettono cifre agli utenti senza `pagamento.read`: espongono il solo booleano `daSaldare` (e non trasportano mai registro, costi o prodotti);
- registrare, modificare, rimuovere o correggere un acconto richiede `pagamento.record`: amministrazione e direzione dal ruolo, gli altri solo con un **override individuale** (`permessi.updateOverride`, motivato e auditato in `policy_change_events`) — mai assegnato all'intero ruolo commerciale;
- la pagina `/pagamenti` («vista cassa di sede») e `pagamentiRecenti` richiedono `pagamento.read`; la voce di sidebar segue la stessa policy via `permessi.mie`;
- le superfici condivise (Board, feed, casi operativi, notifiche) non contengono importi né valori da cui ricostruirli (§37.3).

37.4 Pagina Pagamenti (`/pagamenti`)

Vista cassa di sede, in sidebar dopo Magazzino. Mostra solo commesse attive (no archiviate).

- **KPI**: Pattuito · Incassato · **Da incassare** · numero commesse senza importo pattuito.
- **Strip «Ultimi incassi»**: gli acconti più recenti della sede (endpoint `commesse.pagamentiRecenti`, che appiattisce i registri ordinandoli per data di registrazione e resta sede-scoped); il click apre la commessa.
- **Righe ordinate per residuo decrescente** (i soldi più grossi in cima): codice, cliente, StatoChip, pattuito, barra percentuale con incassato (nascosta sotto `md`), **Residuo** in ambra oppure «Saldata» in verde.
- **Bottone «Acconto»** su ogni riga con residuo: apre un dialog rapido con chips **50/40/10%** + **«Salda tutto»**, data (default oggi), metodo e nota — registra senza dover aprire la commessa (riusa `addPagamento`).
- **Filtri**: Con residuo (*default*) / Saldate / Senza importo / Tutte, più ricerca per codice o cliente.
- **Copertura costi fissi**: pannello full-width sopra gli ultimi incassi, alimentato esclusivamente dai documenti FiC. Mostra obiettivo di fatturato netto del mese, netto già emesso, importo ancora da fatturare, costi fissi medi, margine di contribuzione, avanzamento e affidabilità. La formula usa i 12 mesi precedenti; tra 3 e 11 mesi usa il periodo disponibile e segnala affidabilità media, sotto i 3 mesi non inventa un risultato. I costi `dubbio` restano esclusi e la CTA apre direttamente la revisione Acquisti in Contabilità.

38. Sincronizzazione Google Calendar

38.1 Export (CRM → Google) — feed iCal per sede

- Ogni sede ha un **token segreto** (store `calendar_tokens`, rigenerabile: la rigenerazione revoca tutte le iscrizioni).
- Endpoint anonimo `GET /api/ics/:token/:feed.ics` (il token è il bearer) con cache 5 min. Feed: `tutti, rilievo, posa, assistenza, altro`.
- ICS RFC 5545 con VTIMEZONE Europe/Rome; titolo = tipo + cliente; location = indirizzo lavoro.
- L'operatore aggiunge il feed in Google Calendar via "Altri calendari → Da URL"; Google lo aggiorna periodicamente (sola lettura lato Google).

38.2 Import (Google → CRM) — overlay in Planning

- Sorgenti per sede (store `external_calendars`): nome, **indirizzo iCal segreto** di Google ("Impostazioni → Integra calendario"), colore da palette, attivo.
- Fetch **server-side** (evita CORS) con cache 10 min, follow redirect, mantiene lo stale su errore; `web-cal://` normalizzato a https; **SSRF guard** (§3.7).
- Parser ICS interno: unfolding, unescape, DATE/DATE-TIME/UTC, TZID→Europe/Rome; **espansore RRULE** limitato alla finestra richiesta (DAILY/WEEKLY/MONTHLY/YEARLY, INTERVAL, COUNT, UNTIL, BYDAY, EXDATE, cap iterazioni).
- Gli eventi appaiono read-only nel Planning (§12.9).

39. Backup notturno su Google Drive

39.1 Contenuto e struttura

Ogni notte alle **00:00 Europe/Rome** (o manualmente) viene generato uno snapshot datato:

- Per documenti e allegati il backup risolve prima `storageKey`, rilegge i byte dal driver attivo e verifica il checksum SHA-256. Se l'oggetto manca o è corrotto il run fallisce in modo visibile: non viene prodotto un backup silenziosamente incompleto.
- I record legacy con `dataBase64` restano supportati. Gli allegati ticket sono inclusi anche quando la commessa o il cliente collegato non sono più presenti.

```
Backup CRM YYYY-MM-DD/
  database/<store>.json          ← dump integrale di ogni raccolta (utenti SENZA password)
  Sede <nome>/
    Utenti.json                  ← utenti della sede (sanificati)
    Clienti/<Cognome Nome (CL-id)>/
      Scheda cliente.pdf         ← generata server-side (gemella della scheda in app)
      cliente.json
      Commesse/<CODICE>/
        commessa.json
        Preventivi e contratti/... ← file caricati, raggruppati per tipo
        Misure/... · Fatture e pagamenti/... · Ordini/... · DDT/... · Foto e altro/...
```

```
Ticket <id>/...      ← allegati ticket
Commesse senza cliente/<CODICE>/...
```

39.2 Collegamento Google (account personale)

- I service account NON possono scrivere su My Drive personale (Google One) → modalità primaria **OAuth utente**: la direzione collega l'account aziendale (una volta) con scope **drive.file** (l'app vede solo i file che crea).
- Flusso: `backup.oauthStartUrl` (adminProcedure, state monouso anti-CSRF) → authorize Google → callback anonima `/api/oauth/gdrive/callback` → refresh token salvato (store + specchio su file, §28.1).
- Al primo backup l'app crea la cartella "**Backup CRM Ruffino**" nel My Drive collegato; l'operatore può spostarla/condividerla ovunque (tracciata per id — attualmente dentro la cartella condivisa aziendale).
- Fallback: senza credenziali il backup viene comunque scritto su disco server (`backups/` , `gitignored`). Modalità service account mantenuta nel codice per un eventuale passaggio a Workspace/Shared Drive.

39.3 API & UI

- Drive v3 via REST puro (JWT/token con crypto nativo — zero dipendenze npm); find-or-create cartelle con cache per run; upload multipart; `supportsAllDrives` .
- Router backup (direzione): `status` , `log` (ultimi 60 run), `runNow` , `updateConfig` , `oauthStartUrl` , `disconnectOAuth` , `checkRoot` .
- Card in Impostazioni: stato ("Drive collegato" + email account), ultimo backup (esito/file/MB), countdown prossimo run, "Esegui ora", collega/scollega, istruzioni una-tantum.
- Single-flight guard (un backup alla volta); log persistito.

40. Fatture in Cloud → Registro economico e clienti

40.1 Funzione

Ogni ora (se abilitato) o su "Sincronizza ora", il CRM legge anno corrente e precedente di **fatture emesse, note di credito emesse, spese e note di credito passive**. Le sole fatture creano i clienti mancanti; i clienti esistenti non vengono modificati.

40.2 Logica di creazione

- Dedup su denominazione normalizzata contro i clienti esistenti.
- Persone: split cognome/nome **validato col codice fiscale** (consonanti del cognome vs primi 3 caratteri del CF; supporta cognomi composti e denominazioni invertite). CF valido salvato sul cliente.
- Aziende (P.IVA presente o ragione sociale riconosciuta): denominazione completa in `cognome` , tipo `azienda` (o `condominio`).
- I clienti creati vanno sulla sede primaria.

40.3 Config & stato

Store `fic_config`: modalità `oauth` o `manual`, access token e refresh token cifrati, scadenza, `companyId`, abilitazione, data collegamento, ultimo esito e contatori privacy-safe dell'ultimo sync. Lo status non restituisce mai segreti in chiaro.

- OAuth Authorization Code: `oauthStartUrl` crea uno state monouso valido 10 minuti; callback `/api/oauth/fic/callback`; scopes `read-only` `entity.clients:r` `issued_documents.invoices:r` `issued_documents.credit_notes:r` `received_documents:r`.
- L'access token viene rinnovato prima della scadenza con refresh deduplicato per sede. Il refresh token aggiornato viene persistito cifrato.
- Se l'account espone una sola azienda, questa viene selezionata automaticamente; altrimenti resta il picker `/user/companies`.
- La disconnessione rimuove i token OAuth. Il token manuale resta disponibile soltanto come fallback di emergenza.
- Router direzione: `status`, `oauthStartUrl`, `disconnectOAuth`, `saveConfig`, `companies`, `syncNow`, `annullaSync`.
- `status` espone `syncInCorso` e `syncAvviataAt` per la sede corrente. La UI aggiorna lo stato mentre il lavoro è attivo e consente di fermarlo anche dopo un refresh della pagina.
- Ogni richiesta HTTP verso FiC ha un timeout di 30 secondi; un giro completo ha un limite di 10 minuti. Cancellazione, timeout ed errori liberano sempre il lock per sede senza consentire due sync concorrenti.

Il requisito OAuth è code-complete. L'attivazione in produzione richiede `FIC_OAUTH_CLIENT_ID`, `FIC_OAUTH_CLIENT_SECRET`, `FIC_OAUTH_REDIRECT_URI` e un `MAIL_ENCRYPTION_KEY` stabile su Railway.

40.4 Fatture e riconciliazione (/economia)

Le fatture sincronizzate sono persistite nello store `fic_fatture`, sede-scoped. La pagina `/economia`, visibile a direzione e amministrazione, separa `Andamento`, `Da riconciliare`, `Costi fissi` e `Acquisti`. `Andamento` si apre con la sintesi dell'anno — fatturato, costi, differenza e cassa attesa — e dichiara quante fatture restano da riconciliare e quanti costi da classificare prima di presentare i totali come definitivi. La panoramica divide Contratti CRM, Vendite FiC e Acquisti FiC: imponibile, IVA e lordo non vengono confusi e l'andamento mensile confronta grandezze dello stesso periodo.

Una fattura può essere collegata manualmente a una commessa soltanto dopo una conferma esplicita. Dal 26/08/2026 il match automatico non è più limitato all'identificativo fiscale: vale un solo segnale in comune fra telefono, email, nome e cognome, indirizzo o identità fiscale del cliente, e il codice commessa citato nell'oggetto della fattura prevale su tutto. La parità fra due commesse non viene sciolta: la fattura resta da riconciliare con i candidati esposti. Dalla stessa data **il pattuito è fonte FiC** quando esiste almeno una fattura collegata (vedi §40.6); resta CRM e modificabile a mano solo in assenza di fatture. Le rate FiC pagate sincronizzano in modo deterministico e idempotente soltanto movimenti `origine = fic`; importo, data, stato e storni seguono FiC. Un movimento stornato resta auditabile e non alimenta `importoIncassato`. Una risposta incompleta non può stornare rate mancanti.

Il sync non modifica mai i pagamenti manuali. Una discordanza fra un manuale compatibile e la rata FiC produce una **segnalazione tipizzata** nell'esito della sincronizzazione (`correggi_manuale`, oppure `scegli_manuale` quando i candidati sono più di uno) e la fattura resta `da_riconciliare`: decide l'operatore, modificando o rimuovendo il pagamento manuale. Esiste anche l'endpoint di correzione gui-

data `commesse.correggiPagamento` (direzione/amministrazione), che rivalida fingerprint del pagamento, rata FiC e link prima di scrivere; dal 28/08/2026 non ha una UI dedicata. La riconciliazione è uno-a-uno in entrambe le direzioni: una rata FiC ha un solo link attivo e lo stesso pagamento manuale non può rappresentare due rate, anche tra fatture diverse. Il risanamento sceglie il link più compatibile con importo e data indipendentemente dall'ordine delle rate; un pagamento FiC già persistito ma rimasto senza link viene recuperato senza crearne un secondo. I movimenti FiC associati ai link storici perdenti vengono stornati prima di superare il link; per un manuale perdente il sync emette invece una segnalazione di storno da applicare a mano, senza spostare il collegamento canonico.

Su una fattura multirata, una nota FiC esplicita ma incompatibile con tutte le rate blocca l'import automatico delle altre rate: la segnalazione indica il manuale da riallineare e il registro resta invariato finché l'operatore non decide. In questo modo l'incassato non contiene contemporaneamente il manuale discordante e nuovi movimenti FiC della stessa fattura. Il blocco riguarda soltanto nuove righe: aggiornamenti, storni e risanamento dei link già presenti continuano.

Prima di applicare `correggiPagamento` vengono riletti la rata FiC corrente, il pagamento CRM e il link attivo: se importo, data, stato, sorgente o destinazione sono cambiati, la richiesta viene rifiutata con `PRECONDITION_FAILED` senza modificare il registro. Nessuna correzione può scrivere su dati diversi da quelli su cui è stata calcolata.

Il documento PDF ufficiale, quando disponibile, viene archiviato nel fascicolo della commessa come file `fattura` dopo aver persistito il collegamento. Le fatture non abbinate restano in coda con i candidati del match esposti, compreso il motivo di scarto o d'incertezza (§40.4): si collegano a mano dopo conferma esplicita, si escludono dalla riconciliazione, oppure — se il cliente non ha commesse — si crea la commessa con «Crea le N commesse mancanti».

Ogni sincronizzazione FiC ripara inoltre le fatture già collegate che non hanno ancora il PDF nel fascicolo. Il recupero considera soltanto collegamenti espliciti (`commessaId`), deduplica per sorgente e id FiC e isola gli errori per singola fattura: un download fallito non interrompe il lotto e viene ritentato alla sincronizzazione successiva. Le sole corrispondenze ipotetiche non generano documenti. Un errore storage non crea nuovi blob base64 e non annulla il collegamento o la riconciliazione economica già completati.

40.5 Snapshot e classificazione dei costi

`fic_fatture` mantiene documenti emessi, tipo, imponibile, IVA, lordo e stato di presenza. `fic_costi` mantiene documenti ricevuti e classificazione `fisso` | `variabile_commissa` | `straordinario` | `dubbio` . Ogni flusso ha uno snapshot indipendente: soltanto una paginazione completa può marcare `presenteInFic = false` ; il record non viene cancellato e conserva audit e collegamenti. Questo impedisce che documenti rimossi da FiC continuino a gonfiare i KPI.

Dal 28/08/2026 nessun modello classifica i costi. Un documento nuovo entra `dubbio` e si classifica nella scheda Acquisti; le regole per fornitore confermate da un operatore (`Ricorda regola` , sempre una scelta esplicita) si applicano deterministicamente ai documenti successivi dello stesso fornitore, anche durante il sync. Un costo `dubbio` resta escluso dal pareggio e visibile nella revisione. Una classificazione manuale non viene mai sovrascritta.

Il fatturato canonico è imponibile fatture meno imponibile note di credito. I costi canonici sono imponibili spese meno imponibile note passive. Solo rate `paid` alimentano incassi/uscite e solo `not_paid` alimentano aperti; altri stati sono esclusi finché non mappati esplicitamente.

40.6 Pattuito e piano rate — fonte FiC (26/08/2026)

Il pattuito (`importoTotale`) e il piano rate (`pianoRate[]`) di una commessa hanno due regimi, distinti da `pattuitoFonte` :

- `fic` — esiste almeno una fattura FiC collegata. Importo e rate sono derivati da quelle fatture, in modo deterministico e idempotente. Le note di credito abbattano il pattuito e non generano rate in attesa. Ogni scrittura manuale su `importoTotale` o sulle rate risponde `PRECONDITION_FAILED` , direzione inclusa: correggere significa correggere la fattura in FiC o scollegarla.
- `manuale` — nessuna fattura collegata. Pattuito e rate li inserisce l'operatore. È il regime normale finché la fattura non viene emessa.

Il passaggio a `fic` avviene al primo collegamento; il ritorno a `manuale` solo quando l'ultima fattura viene scollegata, senza azzerare la cifra già mostrata. Le rate FiC portano `ficDocumentoId` , `ficRataId` e `ficSourceKey` , la stessa chiave stabile del registro pagamenti.

Il piano rate NON è il registro incassi: descrive le scadenze concordate, mentre `pagamenti[]` registra il denaro effettivamente ricevuto. La scheda commessa li mostra separati e dichiara lo scostamento quando la somma delle rate non copre il pattuito.

40.7 Costi fissi confermati (27/08/2026)

La ricorrenza (stesso fornitore e importo per almeno tre mesi consecutivi, tolleranza 50 centesimi) è solo una proposta. Un costo entra nel totale fisso e nel fatturato necessario a coprirlo soltanto dopo conferma nel registro aziendale, con importo, cadenza e validità. Il sync non può confermare né sovrascrivere questa decisione.

40.8 Reset del pattuito (operazione una tantum)

`scripts/reset-pattuiti.ts` azzerà `importoTotale` , `pattuitoFonte` , `pianoRate[]` ed elimina i pagamenti con `origine="manuale"` , conservando i movimenti `origine="fic"` . È distruttivo e senza undo applicativo: si rifiuta di partire in `--apply` senza un backup Drive riuscito nelle ultime 24 ore. Le commesse archiviate restano escluse salvo `--includi-archivate` .

41. WhatsApp (deep link)

- Helper `lib/whatsapp.ts` : normalizzazione numeri italiani → `wa.me/39...` con messaggio precompilato; firma aziendale "Ruffino Group — tel. 0187 872687".
- Bottone verde accanto al telefono in: **scheda cliente** (messaggio generico pratica), **scheda commessa** (messaggio con codice commessa), **dialog appuntamento del Planning** (conferma appuntamento con tipo, data e ora).
- Nessun invio automatico: si apre WhatsApp con il testo pronto, l'operatore preme invia.

42. Scheda cliente PDF

- Bottone "Scheda PDF" nella scheda cliente → A4 via jsPDF/autotable: anagrafica completa (tipo, contatti, CF/P.IVA, doppio indirizzo, detrazione, pratica edilizia, assegnatario), note, e tabelle Commesse / Appuntamenti / Ticket / Garanzie con section header e page-break safe.
- La stessa scheda è generata **server-side** per ogni cliente nel backup notturno (§39.1).

43. Migrazione dati 2026 (eseguita il 15/07/2026)

Operazione una-tantum documentata per riferimento storico:

1. **Clienti**: 101 clienti unici estratti dal riepilogo fatture 2026 di Fatture in Cloud (113 righe); 73 creati, 28 già presenti. Split cognome/nome validato via CF; aziende riconosciute da P.IVA.
2. **Arricchimento**: incrocio con l'export anagrafico completo (846 record) → 72 clienti completati con indirizzo/città/CAP/telefono/email/CF/P.IVA; 1 inversione nome corretta via CF.
3. **Commesse + stati + note**: dalle 7 liste Microsoft To Do di reparto (Misure Esecutive, Aggiornamento Contratto, Ordini, Produzione, Attesa Posa, Finiture a saldo, Interventi/Regolazioni) → 43 commesse create, 23 riusate, 62 stati portati alla fase corretta, 71 note operative scritte negli step timeline corrispondenti (Firma Contratto / Ordine Merce / Data Spedizione / Appuntamento Posa / Finiture).
4. **Dedup**: merge di 5 doppi (typo, nomi invertiti, coniugi cointestatari) con fusione di note timeline e stati; 3 coppie legittime (persona + propria azienda, conviventi) lasciate distinte.
5. Le righe To Do di clienti non-2026 (fatture 2023-25) sono state escluse deliberatamente.

44. Prodotti della commessa

44.1 Scopo

Una commessa deve dire **di cosa si tratta**. Il campo `prodotti[]` esisteva già sul modello ma era riempibile solo dopo, un elemento alla volta, dall'interno della scheda: in lista non compariva nulla.

44.2 Tipologie

Elenco chiuso, per poter raggruppare e filtrare, con "Altro" come valvola di sfogo:

Infissi · Porte interne · Portoncino / Blindato · Zanzariere · Persiane · Avvolgibili / Tapparelle · Cassonetti · Controtelai · Tende da sole · Veneziane · Grate · Vetri · Scale · Altro

`TIPOLOGIE_PRODOTTO` è definito in `server/routers/commesse.ts` e replicato in `client/src/lib/prodotti.ts`: i due elenchi **DEVONO** restare allineati.

44.3 Dichiarazione in creazione

Il dialog **Nuova commessa** — sia nella pagina Commesse sia nella **scheda cliente** — espone un blocco **Prodotti**: righe di tipologia + quantità, aggiungibili e rimuovibili in linea. Le righe finiscono in

`prodotti[]` alla creazione. Le righe senza tipologia vengono scartate; la quantità minima è 1.

44.4 Modifica su commesse esistenti

Il tab **Prodotti** della scheda commessa consente aggiunta, modifica ed eliminazione. Il nome del prodotto è la stessa tendina di tipologie; i prodotti già registrati con **nome libero** (precedenti a questa versione) conservano il proprio valore, che viene proposto in cima alla tendina come opzione a sé — correggere una quantità non deve riscrivere in silenzio cos'è il prodotto. Il campo `tipologia` è etichettato "**Materiale**" (PVC / Alluminio / Legno).

Le mutation `addProdotto` / `updateProdotto` / `removeProdotto` **DEVONO** invalidare sia `commesse.byId` sia `commesse.list`, altrimenti la colonna in lista resta indietro.

44.5 Presentazione

- **Pagina Commesse:** colonna **Prodotti** con badge `7× Infissi`, `3× Zanzariere`; oltre due voci compare `+N` con il dettaglio in tooltip.
- **Scheda cliente:** le card delle commesse mostrano gli stessi badge.

45. Marginalità stimata CRM

45.1 Formula della stima legacy

```
marginale lordo = importoTotale - Σ costi[] - costoPosaStimato
marginale %      = marginale lordo / importoTotale
```

Il calcolo vive in `server/_core/margine.ts` come funzione pura (`calcolaMargine`) ed è esplicitamente una **stima CRM**, non il totale effettivo aziendale. I totali effettivi e il punto di pareggio usano solo `FIC`.

Il flag `datiIncompleti` è vero quando manca il pattuito **oppure** non è registrato alcun costo: senza costi il margine risulterebbe un finto 100 %.

45.2 Registro costi (`costi[]`)

I costi si registrano **dentro la commessa**, non dagli ordini fornitore: `{ id, importo, fornitore, descrizione, data, numeroOrdine, note }`. Le mutation `addCosto` / `updateCosto` / `removeCosto` sono riservate a **direzione o amministrazione**.

Scelta deliberata: un solo posto dove scrivere un costo, nessun doppio conteggio. Il modulo Fornitori conserva i propri ordini per la parte logistica (righe, ricevimento merce) ma **non alimenta più il margine** — evitando anche la trappola dell'ordine lasciato in "bozza" che silenziosamente non contava.

`importaCostiDaOrdini` esegue un import **una tantum** degli ordini già a sistema (esclusi `bozza` e `contestato`); è idempotente su `numeroOrdine` e il bottone compare solo finché il registro è vuoto.

45.3 Card "Economia" (scheda commessa)

Visibile **solo** a direzione e amministrazione (la query stessa è gated lato server). Mostra pattuito, costi fornitore con conteggio, costo posa modificabile in linea con blur-save, e margine in € e %.

Colori per fascia: ≥ 30 % verde, **15–30** % ambra, < 15 % rosso, grigio quando i dati sono incompleti. Sotto, il registro dei costi con modifica in riga ed eliminazione.

45.4 Pagina /marginalita

Riservata alla **direzione** (voce di sidebar con flag `direzioneOnly`). Esclude le commesse archiviate.

- **KPI**: margine totale, margine medio %, commesse con dati, dati incompleti.
- **Tabella** ordinabile per % (peggiori prima), margine € o pattuito; ricerca e filtro per stato; le righe con dati incompleti sempre in fondo.
- **Aggregati**: costi per fornitore, margine per venditore, margine per mese di apertura — calcolati solo sulle righe complete.

46. Squadre di posa

46.1 Visibilità

La pagina **Squadre di posa** è in sidebar ed è **leggibile da tutti i ruoli** (`squadre.list` è `protected-Procedure`): serve a chiunque debba sapere chi è in cantiere. Creazione, modifica ed eliminazione restano `adminProcedure`, e la pagina nasconde i comandi a chi non è direzione invece di lasciarlo sbattere contro un FORBIDDEN.

Ogni card squadra elenca le **commesse attive assegnate**, quelle già in fase di posa per prime, ciascuna con link alla commessa.

46.2 Assegnazione alla commessa

Il campo `squadraId` esisteva sul modello ma non era né mostrato né impostabile: le squadre si assegnavano solo al singolo intervento, quindi guardando una commessa non si sapeva chi la stesse posando.

La card **Squadra di posa** nella scheda commessa assegna la squadra con una tendina cercabile. Diventa **ambra con "Da assegnare"** quando la commessa raggiunge `attesa_posa` senza squadra; altrimenti mostra caposquadra e telefono.

Creando un intervento dalla commessa, la squadra della commessa è **pre-selezionata**.

46.3 Board

Le card nelle fasi di posa (`attesa_posa`, `finiture_saldo`, `interventi_regolazioni`) portano un chip con il nome della squadra, oppure **"Squadra da assegnare"** in colore warning quando manca.

47. Storage dei documenti

47.1 Problema

Prima di questa versione ogni documento viveva in base64 dentro la JSONB della propria raccolta (`preventivi_documenti`, `ticket_allegati`). Poiché `persistedStore` riscrive l'intero blob a ogni `save()`, **ogni upload riscriveva tutti i byte di tutti i file**.

47.2 Layer

`server/_core/fileStorage.ts` espone `putFile` / `getFile` / `deleteFileQuiet` con due driver, selezionati da `STORAGE_DRIVER`:

- **local** — file sotto `./data/files/<key>`; adatto allo sviluppo o a un deploy con volume montato;
- **s3** — qualunque endpoint S3-compatibile (Cloudflare R2, AWS S3, MinIO) via REST + **SigV4 firmato con `node:crypto`**, senza dipendenze npm aggiuntive.

Variabili: `S3_ENDPOINT`, `S3_BUCKET`, `S3_ACCESS_KEY_ID`, `S3_SECRET_ACCESS_KEY`, `S3_REGION`.

47.3 Modello del documento

Il record conserva i soli metadati più `storageKey` e `checksum` (sha256). La lettura è **retro-compatibile**: i record che portano ancora `dataBase64` funzionano immutati; `byId` continua a ricostruire il base64 per i consumer storici, mentre la scheda commessa usa la rotta binaria autenticata.

Se il driver fallisce in scrittura, soltanto i file fino a 10 MB possono ricadere sul base64 inline. Oltre 10 MB il caricamento fallisce visibilmente senza creare il record: i file grandi non devono entrare nel JSONB.

47.4 Guardia sul filesystem effimero

Su Railway senza volume il filesystem è effimero: un record otterrebbe `storageKey` e i byte morirebbero al deploy successivo. `putFile` **rifiuta** quando il driver è `local`, l'ambiente è Railway e `STORAGE_ALLOW_EPHEMERAL` non è `1`: i file fino a 10 MB ricadono sull'inline, quelli più grandi falliscono senza persistere metadati orfani.

47.5 Migrazione

`scripts/migrate-documents-to-storage.ts` (e le procedure direzione `fileStorage.status` / `fileStorage.migrate`):

- **dry-run di default**, `--apply` per eseguire;
- per ogni documento: scrive → **rilegge** → confronta lo sha256 → **solo allora** rimuove il `dataBase64`;
- **idempotente e riprendibile**: salta chi ha già `storageKey`;
- **rifiuta di partire** senza un backup Drive riuscito nelle ultime 24 h (controlla `backup_log`);
- **rifiuta** il driver `local` su Railway senza opt-in esplicito.

`pnpm storage:check` e la procedura direzione `fileStorage.probe` verificano la configurazione con un ciclo reale `put` → `get` → `checksum` → `delete` senza esporre access key o secret. Il runbook Cloudflare

R2 è in `docs/storage-r2.md`.

47.6 Cascade

L'eliminazione di un documento, di un allegato, di un ticket e — nuova — di una **commessa** rimuove anche i byte dallo storage. Prima l'eliminazione di una commessa lasciava i documenti orfani nella raccolta.

47.7 Backup dopo la migrazione

Il backup Drive non legge più soltanto `dataBase64`: usa `storageKey` come fonte canonica, verifica `checksum` e conserva il fallback legacy. Questo requisito è coperto da test per lettura inline, oggetto presente, oggetto mancante e checksum non valido.

48. Formato e parsing degli importi

48.1 Formato

`formatEuro` in `client/src/lib/euro.ts` è l'**unico** formatter: separatore di migliaia col punto, decimali con la virgola, **sempre due cifre decimali** anche quando sono `,00` → `1.234,56`.

`useGrouping: true` è obbligatorio: la regola italiana di `Intl` raggruppa solo da cinque cifre (`minimumGroupingDigits: 2`), per cui `5000` usciva "5000" accanto a "10.000" nella stessa colonna.

Ogni superficie che mostra denaro **DEVE** usare `formatEuro` / `formatEuroSimbolo`: Pagamenti, card acconti, card Economia, Marginalità, feed Dashboard, chip del board, Fornitori, rifacimenti. I preventivatori mantengono il proprio `Intl.NumberFormat` (stampano nei PDF col simbolo in coda) ma con lo stesso flag di raggruppamento.

48.2 Parsing

`parseEuro` interpreta il separatore dal contesto:

- punto e virgola → l'ultimo dei due è il decimale (`1.500,50` = 1500.5);
- solo virgola → decimale (`1500,50`);
- solo punto → **decimale** se seguito da 1–2 cifre (`1500.50`), **migliaia** se seguito da esattamente 3 cifre o se ce n'è più d'uno (`1.500` , `1.234.567`).

Varianti: `parseEuroPositivo` (incassi, > 0) e `parseEuroNonNegativo` (costi, ≥ 0).

Il parser precedente faceva `replace(/\.\/g, "").replace(",", ".")`: corretto per la notazione italiana, **catastrofico** per chi digita il punto decimale — `1500.50` veniva registrato come **150050**, cento volte tanto, senza alcun avviso. Il punto è ciò che quasi tutti digitano sul tastierino numerico. **Nessun `parseFloat` a mano sugli importi.**

49. Correzioni notevoli (v4.2)

Registrate perché ognuna nasconde una regola da non violare di nuovo.

Difetto	Regola che ne deriva
1500.50 salvato come 150050	il parsing degli importi passa solo da <code>parseEuro*</code> (§48.2)
"Da incassare" calcolato come <code>max(0, Σpattuito - Σincassato)</code> : una commessa incassata in eccesso cancellava il debito di un'altra (4.000 invece di 5.000)	i residui si sommano per commessa , non sugli aggregati; l'eccedenza ha un proprio KPI, un filtro e la dicitura "+€ X in più" sulla riga (prima leggevano "Saldata")
<code>importoIncassato</code> scrivibile via <code>commesse.update</code>	i campi derivati non stanno negli schemi di input
Ticket con commessa cancellata impossibile da eliminare	<code>requireOwnershipOrDirezione(null)</code> lancia NOT_FOUND prima di valutare il ruolo: prevedere sempre il ramo "genitore mancante"
<code>apertoBy</code> sempre <code>null</code> alla creazione	un campo di autorizzazione che non viene mai popolato non autorizza nessuno
Notifiche mute sui ticket senza commessa	ogni sorgente di notifica deve prevedere il caso in cui l'entità collegata manchi
Costo posa che tornava indietro al blur	una scrittura deve invalidare tutte le query che mostrano quel dato, non solo la più ovvia
Colonna Prodotti ferma dopo una modifica dal tab	idem: <code>byId</code> e <code>list</code>
Rata suggerita sempre "1° acconto"	se una <code>list</code> filtra dei campi, ciò che serve alla UI va esposto in forma sintetica (<code>nPagamenti</code>)

50. Registro storico — agente operativo rimosso il 28/08/2026

Questa sezione conserva fedelmente la rimozione dell'agente allora esistente. **Non descrive il presente**: dal 29/08 è nato Tars v2 e il runtime corrente è descritto in §54 e nella matrice T0. Il testo seguente resta per dire cosa fu tolto, cosa sopravvisse e quali decisioni erano aperte in quel momento.

Cosa è stato tolto. Loop agentico, proposte con approvazione, chat, Command Center `/tars`, smistamento automatico di email e fatture, classificazione AI dei costi FiC, audit processi ed esperimenti, planner, motore di contesto, ricerca semantica, autonomia per capability, evals, registro esecuzioni e budget. In tutto ~27.000 righe.

Cosa è rimasto, perché non era l'agente e viveva nella stessa cartella per ragioni storiche: la tabella `comunicazioni` con Inbox e conversazioni, IMAP, l'integrazione WhatsApp, le caselle, il matcher deterministico cliente/commessa (usato anche dalle fatture FiC) e le regole filtro mittente. Tutto spostato in `server/comunicazioni/`. È rimasta anche la **Conoscenza aziendale** (`/conoscenza`), che è una scheda scritta da persone e non il cervello di nessuno: ora ha il suo router.

Cosa NON funziona più, ed è voluto:

- le comunicazioni in arrivo non vengono più classificate né collegate automaticamente: entrano col match deterministico e restano da lavorare;
- le fatture FiC senza commessa non generano più proposte: si collegano a mano, oppure si crea la commessa col bottone dedicato (§40.4);
- i costi FiC non vengono più classificati dal modello: si classificano in Acquisti, che è comunque diventata la fonte del costo fisso (§40.5);
- il Centro Azioni non ha più l'analisi automatica del caso;
- la diagnostica non espone più piani e workflow.

I dati. Gli store dell'agente sono stati esportati prima della rimozione — 498 proposte, 999 esecuzioni, 95 esiti, 6 chat, config e snapshot di processo, 1.610 record in tutto — e poi cancellati da `kv_store`. Le colonne `tars_*` della tabella `comunicazioni` sono rimaste: costano nulla e il prossimo agente probabilmente le rivuole. Le capability `tars.*` restano in `authz/capabilities.ts` perché `tars.manage_policy` governa i permessi stessi e rinominarla vorrebbe dire migrare le regole salvate.

Domande aperte per il prossimo. Nessuna è stata decisa:

1. Cosa deve fare, in concreto, prima di essere considerato utile? Il vecchio faceva diciassette tipi di proposta e nessuno di questi era stato scelto: erano cresciuti uno alla volta.
2. Propone e basta, o esegue? Il vecchio aveva entrambe le modalità e l'autonomia è rimasta spenta per sempre, il che dice qualcosa.
3. Dove vive il punto di ingresso: una pagina sua, oppure dentro le schede in cui il lavoro succede davvero?
4. Quanto può costare al mese, e chi se ne accorge se sfora?
5. Cosa succede quando sbaglia — chi lo scopre, e come si torna indietro?

51. Comunicazioni (Email e WhatsApp)

51.1 Modello e ingestione

Email e WhatsApp confluiscono nella tabella `comunicazioni`. La chiave (`casella_id`, `canale`, `message_id`) rende idempotente la sincronizzazione. Oltre a canale, mittente, contenuto, allegati, cliente/commessa, stato e data, ogni riga persiste categoria, score, motivazione e fonte della classificazione, più le colonne `tars_*` conservate come compatibilità per il futuro agente (senza consumatore dal 28/08/2026).

Gli stati sono `nuova`, `vista`, `gestita`. L'eliminazione dal CRM usa `deleted_at`: il messaggio resta nella casella sorgente e il tombstone impedisce che venga importato di nuovo.

Il collegamento esplicito a una commessa DEVE portare la comunicazione in `gestita`: vale per il collegamento manuale dell'operatore e per l'approvazione di `collega_comunicazione` e `crea_lead`. Il match automatico dell'ingestione NON marca `gestita` — una richiesta nuova su una commessa aperta resta lavoro da leggere. Scollegare riapre la pratica riportandola a `vista`, tranne per le categorie escluse, che sono fuori dalla coda per classificazione e non per collegamento. Una comunicazione già `gestita` non regredisce.

51.2 Classificazione e filtro anti-rumore

Le categorie sono `operativa`, `nuovo_lead`, `amministrativa`, `fornitore`, `da_classificare`, `offerta_marketing` e `spam`. `spam` e `offerta_marketing` sono escluse dalla coda e dai conteggi operativi dopo la classificazione, ma restano consultabili nella vista Escluse.

Dal 28/08/2026 **non esiste alcuna classificazione automatica**. Ogni Email e ogni WhatsApp in ingresso nasce `da_classificare`: il filtro locale deterministico — header del server mail (`X-Spam=*`, `List-Unsubscribe`, `Precedence`), mittente, linguaggio, allegati, regole persistenti e match CRM — calcola soltanto un punteggio e un motivo preliminari, registrati sulla riga (`Controllo preliminare: ...`). La scelta della categoria è dell'operatore, e una classificazione manuale non viene mai sovrascritta da un automatismo. Fanno eccezione i soli record che entrano già dichiarati analizzati in fase di importazione (storico, echo outbound): per quelli vale l'esito del filtro deterministico.

La direzione può memorizzare una regola esatta per mittente; ogni regola è sede-scoped e revocabile. Le regole alimentano punteggio e motivo preliminari e la vista Escluse, non decidono da sole la categoria di un messaggio nuovo.

51.3 Matching deterministico e gestione manuale

Il matching deterministico dell'ingestione prova riferimenti a commessa/cliente (codice, telefono, email, nome, alias WhatsApp) e registra confidenza e motivazione. Un collegamento manuale dell'operatore prevale sempre sul match automatico e porta la comunicazione in `gestita` (§51.1); il match automatico dell'arrivo non marca gestita. Corpo, nomi file e contenuti restano fonti esterne non fidate: sono dati da leggere, mai istruzioni.

Dal 28/08/2026 non esistono proposte automatiche: niente creazione lead assistita, niente archiviazione allegati suggerita, niente gestione proposta. Il lavoro sulle comunicazioni è dell'operatore, con gli strumenti manuali di §51.5.

Un allegato Email si archivia nel fascicolo con `mail.email.archiviaAllegato`: il server rilegge i byte dalla casella IMAP e crea un documento normale del fascicolo con storage e checksum standard. La chiave `sedeId:comunicazioneId:allegatoIndex` rende l'operazione idempotente; il file risultante DEVE essere apribile e scaricabile come un upload manuale.

Limite noto (dal 28/08/2026): per WhatsApp non esiste un percorso di archiviazione attivo. L'helper server sa scaricare i media da Meta tramite `mediaId`, ma l'unico punto d'ingresso era la proposta dell'agente rimosso; la reintroduzione di una mutation manuale equivalente è lavoro tracciato (§31.4).

51.4 Route e compatibilità

Le route canoniche sono `/messaggi/email` e `/messaggi/whatsapp`. `/comunicazioni` DEVE restare un redirect legacy con `replace` verso `/messaggi/email`, conservando solo `view` consentito e `messaggio` numerico positivo; parametri non riconosciuti non devono essere propagati. Le route storiche `/tars` e `/inbox` non esistono più: sono state rimosse il 28/08/2026 insieme all'agente e rispondono con la pagina non trovata.

51.5 API per canale e scope

`mail.email.list`, `mail.email.byId`, `mail.email.stats` e `mail.email.segnaTutteViste` DEVONO forzare il canale Email e la sede attiva. `mail.email.archiviaAllegato` è ammessa solo per un'email della stessa sede, già collegata alla commessa indicata: legge l'allegato dalla casella sorgente e lo archivia nel fascicolo della commessa. Il router storico `mail.comunicazioni.*` resta compatibile per azioni condivise e consumatori esistenti.

`mail.email.archiviaAllegato` richiede che l'email sia già collegata alla commessa indicata («Collega prima l'email alla commessa selezionata»), valida MIME e limite reale di 10 MB prima di scrivere e archivia il documento con tipo `altro`, riclassificabile e rinominabile dalla scheda commessa (§8.4). Un errore di storage non lascia collegamenti parziali. Per WhatsApp non esiste una mutation equivalente (§51.3, limite noto).

`mail.whatsapp.conversazioni` e `mail.whatsapp.thread` sono API di sola lettura e applicano sempre `sedeId`; una conversazione o un thread fuori sede deve rispondere `NOT_FOUND`. `mail.whatsapp.rinominaConversazione` persiste soltanto l'alias locale di una chat non collegata a un cliente CRM. Non esistono API di invio WhatsApp o Email in questa fase.

51.6 Workspace Email

Email offre code Da gestire, Nuovi lead, Gestite ed Escluse, ricerca e lettore operativo con classificazione, collegamento, allegati e corpo. Nel dettaglio il corpo completo precede gli allegati; in lista l'antepri-ma usa due righe e badge testuali per allegati, collegamento e stato. Eliminare dal CRM non tocca la casella IMAP: la riga diventa un tombstone per evitare una re-importazione.

Su desktop da 1280 px la lista ha una larghezza stabile e il lettore occupa tutto lo spazio residuo. Il comando `Espandi email` nasconde temporaneamente la lista senza cambiare messaggio, filtri o URL; `Mostra elenco email` ripristina la vista affiancata. Sotto 1280 px elenco e lettore non vengono compressi in due colonne: si mostra una vista alla volta con ritorno esplicito all'elenco. Corpo e allegati usano contenitori distinti: il testo resta entro una misura leggibile. Mittente, indirizzi, collegamenti CRM e nomi degli allegati sono sempre accessibili nel dettaglio tramite testo a capo, senza ellissi distruttive. Nessuna modalità introduce scroll orizzontale globale.

51.7 Workspace WhatsApp

WhatsApp raggruppa i messaggi in una sola conversazione per account e numero normalizzato, identificata nel client da `wa:<casellaId>:<numero-normalizzato>`. La chiave non espone `sedeId`, che resta un vincolo obbligatorio lato server. Il nome visualizzato ha ordine vincolante: cliente CRM collegato, alias scelto dall'operatore, profilo Meta, numero normalizzato. L'alias è persistito per sede, account e numero e non può sovrascrivere il cliente CRM.

Il thread è cronologico e paginato; media e allegati sono metadati ispezionabili. Il workspace è esplicitamente di sola lettura: nessun invio di messaggi o media, nessuna modifica alla fonte WhatsApp, e — dal 28/08/2026 — nessun percorso di archiviazione degli allegati nel fascicolo (§51.3, limite noto). Su desktop la lista e il dettaglio usano colonne con `min-width: 0`; su mobile si mostra una vista alla volta. Nessun testo, numero o allegato deve introdurre scroll orizzontale di pagina.

Scollegare il numero dal CRM non elimina le comunicazioni già importate. Un nuovo Embedded Signup dello stesso numero aziendale DEVE recuperare la casella storica tramite i destinatari dei messaggi e riutilizzarne l'id interno, così che messaggi nuovi, storico, alias e collegamenti continuino nella stessa conversazione invece di creare una seconda chat.

La configurazione WhatsApp DEVE mostrare una diagnostica webhook priva di dati cliente: ultimo evento/campo, ultimo `smb_message_echoes`, quantità di eventi echo e rapporto tra messaggi echo ricevuti e registrati. Testi, numeri, nomi e message id non entrano nella diagnostica. Un echo consegnato ma già presente deve risultare `duplicato`; l'assenza di `ultimoEchoAt` indica invece che Meta non ha ancora consegnato quel campo al CRM. Il payload previsto usa `changes[].field = smb_message_echoes` e `value.message_echoes[]`.

51.8 Sincronizzazione storico coexistence

Dopo l'onboarding il server richiede automaticamente contatti e storico entro la finestra Meta di 24 ore. L'accettazione delle chiamate `smb_app_data` imposta `storicoRichiestoAt`, ma NON equivale al completamento. I webhook `history` aggiornano `storicoUltimoEventoAt` e `storicoProgresso`; soltanto un blocco con progresso 100 imposta `storicoCompletatoAt` e il campo legacy `storicoSincronizzato`.

Per ogni `history[].threads[]`, `thread.id` è la controparte canonica della conversazione sia per i messaggi in ingresso sia per quelli in uscita. Gli echo live continuano a usare `to/recipient_id`. Un messaggio senza controparte normalizzabile viene rifiutato con log privacy-safe e non crea conversazioni vuote. Una migrazione PostgreSQL una tantum elimina esclusivamente gli outbound WhatsApp legacy con `mittente` vuoto, liberando i `message_id` per una nuova importazione dopo il re-onboarding.

La UI mostra stati distinti `non richiesto`, `richiesto/in attesa`, `progresso` e `completato`, aggiornandosi automaticamente durante la consegna. Non deve presentare una richiesta accettata come storico già sincronizzato.

51.9 Verifica esterna

Senza `DATABASE_URL` lo sviluppo locale usa il fallback in memoria: test, typecheck e build non dimostrano query PostgreSQL, dati o integrazioni Railway. Prima di pubblicare devono essere verificate su Railway le route e i redirect, lo scope tra sedi, la casella Email e i suoi allegati, la configurazione WhatsApp e l'assenza di controlli di invio.

La verifica WhatsApp è completa soltanto quando la configurazione risulta attiva, il webhook riceve `history`, lo storico raggiunge il completamento, un echo live viene registrato e almeno un outbound precedente al collegamento è visibile nel thread corretto. Il 23/08/2026 questi criteri sono stati verificati in produzione: 1/1 configurazioni attive, storico completato, 195 messaggi totali, echo live 1/1 e outbound del 18/08 correttamente etichettato `Tu`.

Se il percorso coexistence corretto genera un QR o codice nuovo ma WhatsApp Business lo rifiuta sul telefono, aggiornare e riavviare prima l'app. Come ultima misura è consentita la reinstallazione soltanto dopo aver verificato dall'app un backup chat riuscito e ripristinabile. Dopo il ripristino si ripete Embedded Signup scegliendo `Collega l'app WhatsApp Business` e condividendo lo storico. Non eliminare il numero/WABA su Meta e non cancellare le comunicazioni CRM come tentativo di risoluzione.

51-bis. Chat aziendale (/chat)

51-bis.1 Scopo

Comunicazione interna fra le persone dell'ufficio, distinta da Email e WhatsApp che parlano con i clienti. Persistita in tabelle PostgreSQL dedicate (`chat_canali` , `chat_messaggi` , `chat_letture`), con fall-back in memoria senza `DATABASE_URL` .

51-bis.2 Canali

- `generale` : uno per sede, non si lascia. Nato come registro delle azioni dell'agente (rimosso il 28/08/2026); oggi è il canale comune della sede.
- `diretto` : fra due persone. La chiave è la coppia ordinata di id, quindi la conversazione è la stessa nei due versi. L'id 0 è riservato al mittente di sistema («Sistema»; i canali creati prima del 28/08/2026 possono conservare il vecchio nome «Tars» finché non si decide una migrazione).
- `commessa` : previsto nel modello, non ancora esposto.

51-bis.3 Notifiche di assegnazione

Assegnare una commessa, un cliente, un ticket o un intervento a un'altra persona produce un messaggio nella sua conversazione diretta col mittente di sistema. Il consumer è separato dal proiettore delle notifiche: la campanella dipende da `notificationMode` , il messaggio in chat no. Entrambi però passano dal bus eventi: con `eventBusMode=off` per la sede (default, §53.1) l'evento di assegnazione non viene pubblicato e non arriva né notifica né messaggio. Assegnarsi qualcosa da soli non produce messaggio.

51-bis.4 Vincoli

Scope sede su ogni lettura e scrittura; un canale di un'altra sede risponde `NOT_FOUND` . I messaggi di sistema hanno autore nullo e non sono scrivibili dal client. Il segnalibro di lettura non arretra. Limiti attuali: refresh a polling di 5 secondi, nessun allegato, nessuna modifica o cancellazione dei messaggi, nessun push a CRM chiuso.

52. Design system, caricamento e analytics

52.1 Identità visuale

Il tema usa Plus Jakarta Sans, fondo grigio caldo, card bianche, testo inchiostro e giallo saturo come accento. Successo, warning, errore e informazione restano cromaticamente distinti. I componenti operativi usano bordi visibili, raggio moderato e ombre leggere; sono vietati colori locali che ricreano una palette fredda o opaca.

52.2 Responsive e tabelle

Clienti, Commesse e Comunicazioni non devono richiedere scroll orizzontale globale. Gli header sticky devono occupare spazio nel flusso e non coprire la prima riga. Le colonne secondarie si nascondono

progressivamente; controlli e titoli rifluiscono senza sovrapporsi.

52.3 Code splitting

Ogni pagina è importata con `React.lazy` e caricata dentro un `Suspense` stabile. Il build separa i vendor React, UI, dati e grafici per caching indipendente. Il runtime Manus, JSX location e il debug collector sono ammessi soltanto sul dev server e non devono gonfiare `index.html` di produzione.

52.4 Umami

Lo script Umami viene installato soltanto in produzione, con endpoint HTTP(S) valido e website id presenti. Ha un id univoco per evitare duplicati, usa `async / defer` e si rimuove in caso di errore di caricamento. In sviluppo non deve generare richieste o warning console.

52.5 UI v2 «Frame & Flow» (dietro `FLAG_UI_V2`)

Redesign avviato il 31/08/2026 sul branch `feature/ui-v2-frame-flow` ; dossier vincolante in `docs/design/ruffino-flow-ui-v2.md` (+ token, motion, responsive, matrice pagine, gate anti-slop). Contratto:

- `FLAG_UI_V2` è un interruttore fail-closed del registro `server/platform/interruttori.ts` : acceso di default solo in development/test, spento in produzione finché la variabile non viene impostata. Governa esclusivamente skin e shell del client: nessun percorso server, nessuna query, nessuna mutation dipende dal suo valore.
- Il client applica `data-ui-v2` alla radice leggendo `platform.interruttori` ; senza attributo la resa resta la v1. I token vivono in quattro quadranti espliciti (`:root` , `.dark` , `[data-ui-v2]` , `[data-ui-v2].dark`) e ogni coppia testo/sfondo è verificata WCAG (tabella in `docs/design/ruffino-flow-tokens.md`).
- Identità v2: canvas caldo, inchiostro, giallo Ruffino `#F2B705` come accento (il «giallo saturo» di §52.1, finora mai implementato), petrolio strutturale, warning ambra distinto dal brand, famiglie di stato a 7 gruppi con etichette invariate, gradienti decorativi assenti.
- Rollback: rimuovere la variabile e ridistribuire; nessuna migrazione dati, nessun cambio di comportamento.

53. Piattaforma operativa

53.1 Eventi e notifiche

Le modifiche business rilevanti producono eventi sede-scoped con chiave di deduplica. Ogni consumer mantiene stato indipendente, retry limitato, dead-letter e recupero dei lease stale. Le assegnazioni devono notificare il nuovo responsabile con link all'entità; presa in carico o completamento risolvono il gruppo canonico invece di generare nuovi avvisi.

Le notifiche realtime usano SSE con replay da `Last-Event-ID` ; il polling resta fallback. L'attivazione avviene per sede nell'ordine eventi shadow, notifiche shadow, notifiche active, SSE e infine Web Push.

53.2 Predisposizioni per il futuro agente (spente)

I flag `contextEngineMode`, `plannerMode` e `semanticSearchMode` restano nel modello dei feature flag (default `off`) ma **non hanno più alcun consumer**: il codice di contesto, planner e ricerca ibrida è stato rimosso il 28/08/2026 insieme all'agente. Il server continua a rifiutare `active` su questi flag e a degradare a `shadow` eventuali valori storici al bootstrap (fail-closed). `autonomyCapabilities` resta una whitelist vuota senza consumer.

Limite noto (dal 28/08/2026): non esiste alcuna API per modificare i flag di piattaforma. L'unico endpoint di scrittura era `tars.config.setPlatformFlags`, rimosso con l'agente; `platform.flags` è di sola lettura. I flag restano congelati ai valori salvati per sede finché non verrà reintrodotta un endpoint direzione con motivazione e audit (§31.2).

53.3 Ricerca ibrida — rimossa

Il codice di indicizzazione e ricerca ibrida è stato rimosso con l'agente. Non esiste alcun indice semantico né lessicale trasversale: la ricerca del CRM è quella delle singole pagine, su testo e metadati, sempre sede-scoped. I requisiti di un eventuale indice futuro (ACL prima e dopo il ranking, chunk versionati, evidence ref) restano descritti in §54.

53.4 Apprendimento e autonomia — rimossi

Nessun meccanismo di learning o autonomia esiste nel prodotto. I principi che regolavano la qualifica di autonomia (default negato, soglie di accuratezza, decisione della direzione, undo, kill switch) restano il punto di partenza del progetto futuro (§54).

53.5 Diagnostica

`diagnostica.snapshot` è accessibile solo alla direzione. Mostra code eventi per consumer con relative dead-letter, notifiche pendenti e connessioni SSE della sede. Non espone prompt, corpi di comunicazioni, numeri, email, token, user id o entity id come label metrica.

54. Tars v2 — runtime corrente e potenziamento in corso

RIALLINEAMENTO TO 31/08/2026: Tars v2 è nel checkout `main` con orchestratore unico, strumenti L0-L3 limitati, cache, memoria, briefing shadow e governor. I contratti vincolanti sono in `docs/tars/architettura-tars-v2.md` e lo stato verificato dominio→servizio→tool→gap in `docs/tars/matrici-azioni-tars.md`. In caso di divergenza, prevalgono questi due documenti sul testo storico. Non introdurre pezzi dell'agente nei router business.

Visione originaria approvata il 28/08/2026 (storia sotto). Il workstream partiva solo dopo la stabilizzazione della verità (documenti, invarianti, test, sicurezza) e la definizione dei contratti dati/eventi, e procede poi in parallelo all'evoluzione degli altri domini.

54.1 Missione

Non una chat, non un classificatore, non un generatore di proposte sparse: una rappresentazione coerente di ciò che accade in azienda, capace di rispondere a «cosa richiede attenzione adesso, perché, chi se ne occupa, entro quando». Il modello mentale è il **caso operativo** con ciclo di vita, evidenze (EvidenceRef verso fonti autorevoli), fingerprint anti-stale e una sola next best action; nessuna_azione e chiedi_chiarimento sono esiti validi.

54.2 Tre livelli

1. **Realtà deterministica** — tutto ciò che il server sa esattamente resta deterministico: state machine, gate, somme, scadenze, permessi, matching con identificativi certi. Mai un LLM per aritmetica o enforcement.
2. **Contesto operativo** — snapshot e dossier per entità che collegano cliente, commessa, documenti, ordini, produzione, calendario, pagamenti, comunicazioni e post-vendita nello stesso caso.
3. **Ragionamento** — solo qui il modello: interpreta il non strutturato, collega fatti, riconosce conflitti, sceglie strumenti, spiega perché.

54.3 Orchestrare, non replicare

Principio fissato dalla direzione: Ruffino Flow governa il lavoro del rivenditore, non sostituisce i sistemi autorevoli altrui.

- **Configurazioni tecniche, listini e compatibilità** restano nei software dei produttori: il CRM li **importa e collega** alla commessa tramite adapter (PDF, Excel, XML, CSV, API), senza ricalcolarli con motori propri meno affidabili.
- **Fatture in Cloud resta la fonte fiscale autorevole**: il CRM governa fatture, incassi, residui e marginalità attraverso l'integrazione, senza diventare un secondo software fiscale.
- Il modello tecnico apertura → configurazione → commessa → ordine → posa → garanzia accoglie i dati autorevoli dei produttori, non ne replica i configuratori.
- L'agente stesso segue la regola: legge fatti dalle fonti autorevoli, non ne inventa (docs/source-of-truth-matrix.md).

54.4 Sicurezza e governo

- La policy tipizzata decide il rischio: letture e preparazione non richiedono conferma; un L1 esplicito personale può agire direttamente; un'azione condivisa/esterna usa al massimo una anteprima immutabile e una conferma. State machine, fonte autorevole, importi e approvazioni restano sempre server-side. Nessuna proposta concede al modello il potere di approvare o aggirare i gate.
- Enforcement server-side, mai nel prompt: sedeId , ACL, NOT_FOUND cross-sede, budget, timeout, idempotenza, audit per run.
- Email, WhatsApp, PDF e allegati sono dati non fidati: un prompt injection nel contenuto non può cambiare policy né eseguire azioni.
- Comandi di dominio tipizzati: mai force , mai tRPC invocato dal modello, nessun accesso SQL, executeSql , updateRecord o mutation generica. Il provider reale passa esclusivamente dal governor.
- Rollout in shadow per sede, reversibile, con eval end-to-end prima di ogni autonomia; costi e budget osservabili per sede.

54.5 Prerequisiti e materiale

Read-API tipizzate e autorizzate, eventi affidabili, evidence/fingerprint, casi persistenti, dataset di eval. I residui conservati apposta (colonne `tars_*`, capability `tars.*`, flag §53.2) si riusano o si migrano solo con una matrice campo→consumer e una decisione registrata. Le domande aperte e la storia della rimozione: `docs/tars-rimosso-2026-08-28.md`; la ricognizione: `docs/discovery-dossier-2026-08-28.md`.

54.5-bis Accettazione T0 vincolante

La regressione Maccari non è una demo: il comando «Analizza l'allegato dell'ultima email di Maccari. Se appartiene alla commessa, archivalo nel fascicolo e, se non trovi problemi, passa la commessa a misure esecutive» deve arrivare a evidenze, audit e Undo solo con match certo, gate valido e transizione adiacente consentita; con ambiguità chiede una sola scelta, con incoerenza non scrive criticamente, con gate invalido non transita e senza capability non rivela dati. Il promemoria «fra un'ora: finanziamento Maccari» deve essere unico, diretto e idempotente. La catena completa è ancora un gap registrato nella matrice: nessuna frase di questo PRD la dichiara esistente.

La tranche T3 del 31/08/2026 chiude il solo tratto finale della catena: preview `verifica_transizione_commissa` e passaggio adiacente `transizione_adiacente_commissa` usano lo stesso servizio deterministico del router. Il comando esplicito è diretto, senza conferma ridondante; richiede le due capability, sede, gate e versione correnti, non accetta `force` e produce audit + Undo monouso. Allegato, analisi/classificazione e archivio Maccari restano il gap della tranche successiva: la catena completa non è dichiarata conclusa.

I tre livelli proattivi sono tutti criteri di completamento: osservazione singola con evidence/fingerprint/stato; pattern aziendali descritti come correlazioni; proposte strutturate di miglioramento da un SafeProductCatalog senza accesso a repository o segreti. Nel codice corrente solo due detector L1 lavorano in shadow; L2 e L3 non sono implementati.

54.6 Document Intelligence — comprensione dei documenti

Stato: la **prima slice è implementata** — analisi deterministica delle conferme d'ordine PDF con testo nativo, comportamento corrente documentato in §19.4. Tutto il resto di questa sezione (OCR, visione, altri formati, collegamento assistito, azioni proposte, dataset di valutazione) resta da costruire secondo il piano in `docs/reports/d7-document-intelligence-piano.md`.

Decisione della direzione del 28/08/2026 (dossier §13, D7): la comprensione dei documenti non è una funzione accessoria ma una **fondazione del futuro agente**, da progettare e implementare dopo le fondamenta di sicurezza e autorizzazione (slice 2 authz) e **prima delle capacità operative avanzate**. Senza comprensione documentale verificabile, l'agente non può essere considerato il cervello operativo dell'azienda. Come tutto il §54: NON IMPLEMENTATA.

Priorità assoluta: le conferme d'ordine dei fornitori in PDF, digitali e scansionate. L'estrattore dedicato deve saper riconoscere, quando presenti: fornitore; numero e data della conferma; riferimento al nostro ordine e a cliente/commessa; righe e posizioni con codici articolo, descrizioni, quantità, misure, colori e finiture; prezzi, sconti e totali; data o settimana di consegna promessa e consegne parziali; va-

riazioni richieste dal fornitore; note, esclusioni e condizioni; pagina e sezione di provenienza di ogni informazione.

Architettura: un registro di parser estendibile, senza promettere un supporto universale non verificato. Priorità dei formati: PDF nativi con testo e tabelle, PDF scansionati, fotografie e scansioni; poi DOCX e Office, XLSX/CSV e listini, XML/JSON, EML con allegati, archivi ZIP, formati tecnici o proprietari dei fornitori tramite parser dedicati. File cifrati, corrotti, illeggibili o non supportati producono uno stato esplicito che richiede assistenza umana: mai fallimenti silenziosi.

Pipeline con stati osservabili ricevuto → validato → estratto → classificato → collegato/ambiguo → revisionato → applicato. Per ogni documento: originale conservato immutato; impronta univoca per rilevare i duplicati; provenienza, autore, data di ricezione, nome, formato e allegati registrati; tipo reale, dimensione, sicurezza e integrità verificati; PDF digitale distinto dalla scansione; estrazione che combina testo nativo, analisi delle tabelle, rendering delle pagine, OCR e visione; classificazione del tipo; estrazione dei campi; tentativo di collegamento a commessa, ordine e fornitore; confronto con i dati già nel CRM; anomalie, proposte ed evidenze; approvazione obbligatoria prima di toccare dati autorevoli.

Confronto con l'ordine originario: articoli mancanti o aggiunti, differenze di quantità, prezzo, misura, colore o configurazione, data di consegna modificata o incompatibile con produzione e posa, riferimenti commessa assenti o incoerenti, duplicati, righe non riconosciute, condizioni commerciali cambiate — classificati per gravità e impatto operativo.

Evidenze e affidabilità: ogni valore estratto porta documento sorgente, numero di pagina, area della pagina quando disponibile, frammento di testo di supporto, metodo di estrazione, livello di confidenza ed eventuali interpretazioni alternative. Distinzione obbligatoria fra dato esplicito nel documento, dato calcolato, collegamento inferito, ipotesi, conflitto e informazione mancante. Un'informazione priva di evidenza non viene presentata come certa.

Collegamento alle commesse: prima i riferimenti deterministici (numero ordine, codice commessa, fornitore, identificativi esterni). Un solo collegamento affidabile si propone; più commesse compatibili si presentano come candidati senza scegliere. Il collegamento manuale diventa un dato verificato e riutilizzabile, senza modificare retroattivamente il documento originale.

Azioni operative: la lettura può produrre aggiornamenti proposti, attività, scadenze, anomalie, richieste di verifica, notifiche, casi operativi e suggerimenti di ripianificazione. Nessuna estrazione AI modifica automaticamente date, importi, quantità, stato della commessa, ordini o appuntamenti: la proposta passa dall'approval gateway (§54.4) mostrando conseguenze ed evidenze. Esempio canonico: la conferma sposta la consegna dal 3 al 10 settembre e la posa è prevista il 12 — l'agente collega il PDF all'ordine corretto, mostra la frase e la pagina che provano la nuova data, valuta il rischio e propone verifica o ripianificazione; non sposta la posa da solo.

Idempotenza e tracciabilità: lo stesso file non crea due volte attività o aggiornamenti. Ogni elaborazione registra impronta del documento, versione del parser e del modello, data, risultati, correzioni umane e azioni approvate o rifiutate; una rielaborazione con versioni nuove non perde i risultati precedenti.

Sicurezza: i contenuti dei file sono input non attendibili — protezione da malware, prompt injection incorporata nei documenti, contenuti ingannevoli e istruzioni rivolte all'AI presenti nel testo. Le configurazioni tecniche del produttore e i relativi documenti restano fonti autorevoli (§54.3): l'agente non le inventa, non le reinterpreta arbitrariamente e non le sostituisce.

Valutazione obbligatoria su un dataset anonimizzato di documenti reali, con almeno: PDF digitale, PDF scansionato, documento ruotato, scansione di bassa qualità, tabelle su più pagine, conferme multilingua, più ordini nello stesso file, riferimenti commessa ambigui, variazioni di quantità/prezzo/ consegna, documento duplicato, file cifrato o corrotto, annotazioni manoscritte, testo con tentativi di prompt injection. Metriche: precisione per singolo campo, qualità del collegamento alla commessa, rilevamento delle differenze, numero di falsi aggiornamenti. Requisito inderogabile: **nessuna modifica critica non autorizzata**.

I nomi dei componenti (ingestion, registro parser, run di estrazione, evidenze, estrattore conferme, candidati di match, confronto, caso operativo) si decideranno studiando il modello dati esistente — documenti §8, allegati comunicazioni §51, ordini fornitore §19 — senza creare una seconda fonte di verità.

55. Contratto strutturato e computo dei limiti di spesa (piano 1, 03-04/09/2026)

Primo dei tre piani chiusi fra il 3 e il 5 settembre 2026 (piano 1: docs/superpowers/plans/2026-09-03-contratto-e-computo-limiti.md , 16 task). Il «contratto» smette di essere un elenco libero di prodotti desiderati e diventa un documento strutturato con righe misurate e prezzate; da quelle righe il CRM ricalcola i **limiti di spesa** ammessi dalla detrazione, gli stessi che l'ufficio calcolava a mano sul foglio «CALCOLO NUOVI LIMITI». Tutto vive dietro l'interruttore `limiti` (§55.8): a flag spento la commessa si comporta esattamente come prima.

Fonti: docs/superpowers/specs/2026-09-03-limiti-e-fatturazione-design.md (§1-§13) e docs/superpowers/specs/2026-09-03-limiti-analisi-fogli-reali.md , che **prevale sulla prima dove divergono** — è la specifica scritta sui fogli reali di tre commesse chiuse nel 2026 e verificata al centesimo.

55.1 Modello dati

- `commessa_contratti` — testata: `pattuito ( pattuitoCent + pattuitoTipo imponibile|lordo )`, tipo detrazione, comune e indirizzo di cantiere, zona climatica (derivata, con override registrato), posa inclusa e `posaCent`, `rate`, `opzioniComputo`, `estrazioneId` (§57).
- `commessa_righe` — una riga per voce venduta: categoria, `tipologia` = **codice del prodotto DEI del catalogo** (non più l'enum `TIPOLOGIE_SERRAMENTO`), quantità, larghezza/altezza in millimetri, mq esatto ($L \times H \times q / 10^6$, `NUMERIC(12,6)`, nessun arrotondamento), accessori `{codice, quantità}`, oscurante integrato con la sua tipologia DEI, prezzo di riga.
- `computi + computo_voci` — l'esito di un calcolo: voce per voce (`inclusa`, `inCheck2`, dettaglio di base e accessori), `CHECK1`, `CHECK2`, limite vincolante, esito, più gli hash di righe e parametri che dicono se il computo è ancora valido.
- Codice: `server/contratti/{repository,hash,servizio}.ts`, `server/computo/{motore,aggregati,zone,tariffe,repository,servizio}.ts`, tipi condivisi in `shared/limiti/tipi.ts` e `shared/euroCent.ts`. Senza `DATABASE_URL` i due repository ricadono in memoria: un test locale non dimostra lo stato dei dati su Railway.
- Router: `contratti ( get , salva , catalogo )`, `computo ( ultimo , esegui )`, `tariffe ( catalogo in sola lettura )`.

55.2 Catalogo DEI e zone climatiche

- `shared/limiti/tariffe-seed.json` : **342 prodotti, 74 accessori, 22 controtelai, 19 opere**, più massimali per gruppo e zona, coefficienti, aliquote di detrazione e `beneSignificativoDefault`. Si rigenera con `scripts/estrai-tariffe-limiti.py`; il foglio sorgente del listino **non entra mai nel repository**.
- `shared/limiti/comuni-zona.json` : **8.104 comuni** con la zona climatica della Tabella A del DPR 412/93, import statico letto da `server/computo/zone.ts` (nessun caricamento manuale al deploy). Le sigle di provincia sono quelle del 1993 (LO/MB/PU/FM/BT/BI non aggiornate): la provincia disambigua solo gli omonimi, non cambia la zona.
- Il seed è a **versione unica** e il pannello Tariffe è in sola lettura: si aggiorna rigenerando il file, non dalla UI (decisione D10 della spec, ancora aperta).

55.3 Motore (`server/computo/motore.ts`)

Funzione pura, nessun I/O, riprodotta al centesimo sui casi d'oro (§55.7).

Passo	Regola
Aggregati	quantità, mq e larghezza totale per i gruppi del foglio (serramenti, serramenti+tapparella/persiana/scuro, cassonetti, <code>cassonettiB</code> , oscuranti soli, schermature, tende, pergole, porte blindate, portoncini, legno e legno-alluminio)
Tempi	ore di tiro al piano e ore di posa dai coefficienti; giornate = <code>ROUNDUP(orePosa / 8)</code>
CHECK1	massimali A/B/C (€/mq per zona × mq del blocco) + controtelai + opere incluse + eventuali richiesti (+ spese professionali se incluse)
CHECK2	Σ dei prezzi DEI ricalcolati riga per riga (prodotto scelto + accessori) + controtelai + opere incluse tranne sviluppo ordine, trasporto e posa
Limite	<code>min(CHECK1, CHECK2)</code> ; se una riga non ha voce DEI, CHECK2 è <code>null</code> , il limite è CHECK1 e l'esito è «incompleto»

Stranezze del foglio riprodotte per scelta, perché sono fatti contabili già accettati (cambiarle è una decisione di direzione, non un fix):

- **minimo 1 mq** applicato al totale della riga, non al pezzo, e solo per serramenti in PVC e alluminio; legno e persiane senza minimo;
- **maggiorazione dell'avvolgibile**: `prezzo × max(1,8; mq + 0,25 × (L + 0,05) + 0,05 × (H + 0,25))`, una volta per riga — il cassonetto aggiunge 25 cm di telo su tutta la **larghezza**, le guide 5 cm su tutta l'**altezza** (correzione H1 del 05/09: i due coefficienti erano invertiti e ora si chiamano `avvolgibileExtraLarghezza / avvolgibileExtraAltezza`);
- **cassonettiB**: il cassonetto venduto insieme al serramento pesa nel massimale **B**, non in A (correzione H2 del 05/09); ovunque conti il prodotto — rilievo, rimozione tapparelle, smaltimento, tiro, posa — i cassonetti dei due blocchi si sommano, e la tapparella che ospita non si posa una seconda volta;
- precedenza degli operatori dello smaltimento identica al foglio; ribalta a 70 € per pezzo; incollaggio a 120 € per anta; soglia del portoncino una volta per riga; cardini cappotto × 2;
- `OpzioniComputo`: rilievo **foro oppure pezzo** (mai entrambi), spese professionali dentro o fuori dal totale, eventuali richiesti in cantiere.

55.4 Interfaccia

- Tab «**Contratto**» al posto di «Prodotti» quando il flag è acceso (`client/src/components/contratto/ContrattoTab.tsx` , `RigaContrattoEditor.tsx`): righe con misura, codice DEI dal catalogo filtrato per zona, accessori ed eventuale oscurante abbinato.
- Tab «**Limiti**» (`client/src/components/computo/LimitiTab.tsx`): «Calcola i limiti» (richiede `computo.run`), esito voce per voce con CHECK1, CHECK2 e limite vincolante.
- `ContrattoStatoBanner.tsx` : riga di stato che porta l'operatore sulla tab giusta senza fargli cercare la linguetta.
- Badge «da contratto · {pattuitoTipo}» accanto al pattuito nella card Pagamenti della scheda commessa.
- Impostazioni → «Limiti di spesa»: `TariffeLimitiPanel.tsx` , catalogo in sola lettura dietro `tariffe.manage` .

Salvando il contratto, `applicaPattuitoDaContratto` (`server/routers/commesse.ts`) allinea pattuito e piano rate della commessa alle righe nuove, **senza toccare le rate già incassate**.

55.5 Gate sulla transizione

`richiedeComputo` (`server/commesse/transizioni.ts`) blocca **solo** il passo `aggiornamento_contratto` → `fatture_pagamento` . Se il contratto manca, o è stato modificato dopo l'ultimo computo (hash di righe o parametri diversi), compare lo stesso dialog «Procedi comunque» dei gate documentali; lo scavalco usa lo stesso `bypassGateDocumentale` e resta scritto nel registro come `gateScavalcato: "documentale" | "computo"` (`null` quando non c'era nulla da scavalcare). Il gate documentale, se manca anche il file, ha la precedenza.

Tars vede lo stesso gate, mai una scorciatoia: `verifica_transizione_commessa` restituisce `gate.-computo` (`richiesto / valido`) e l'anteprima dichiara il blocco prima di muovere qualcosa; `transizione_adiacente_commessa` lo rivaluta a ogni tappa e senza scavalco si ferma dicendo che manca il **computo**, non un file.

55.6 Permessi e flag

- `contratto.read` è condivisa da tutti i ruoli (le misure servono a chi rileva e a chi posa); `contratto.manage` e `computo.run` sono di amministrazione, commerciale e direzione; `tariffe.manage` solo direzione (§4.4).
- Il client non duplica queste stringhe: legge il proprio set effettivo da `trpc.permessi.mie` in `client/src/contexts/OperationalContext.tsx` .
- Interruttore `limiti` (`FLAG_LIMITI`) in `server/platform/interruttori.ts` , **fail-closed**: attivo di default solo con `NODE_ENV` `development` o `test` . Ogni endpoint verifica il proprio interruttore; con il flag spento le mutation rispondono `PRECONDITION_FAILED` e la UI non mostra le tab. Questo documento non attesta lo stato del flag in un ambiente esterno.

55.7 Fixture d'oro dai fogli reali

`server/computo/__fixtures__/casi-reali.json` contiene **20 casi**: i 3 storici del 03/09 più 17 ricavati il 05/09 dai fogli «CALCOLO NUOVI LIMITI» compilati a mano nel 2026. **13 sono verdi al centesi-**

mo; 7 sono saltati con il motivo scritto nel campo `salta` di ciascun caso — divergenze capite e dichiarate, non tolleranza allargata.

Un caso si rigenera con `python3 scripts/harvest-fixture-limiti.py <foglio.xlsm> --nome <nome> --detrazione ecobonus|ristrutturazione`: lo script stampa il caso JSON su stdout e i dubbi su stderr, legge solo misure, codici DEI, prezzi di riga e totali, e **non legge** le celle con nominativo, indirizzo e comune. I fogli non entrano mai nel repository e il caso si chiama come dice `--nome`.

Divergenze parcheggiate, in attesa di una decisione di direzione o del commercialista — finché non arrivano, i casi che le toccano restano `salta`:

	Divergenza
H3	le veneziane del blocco D sono contate a pezzo nel foglio, a mq nel seed
H4	cinque fogli vengono da un'edizione precedente del listino DEI (44 prezzi diversi dal seed, che è a versione unica)
H5	nei totali il foglio somma solo le opere davvero fatturate (colonna «Da fattura»), il motore un insieme fisso: <code>OpzioniComputo</code> non sa escludere una singola opera
H6	lo stesso foglio prezza l'avvolgibile PVC standard a due prezzi diversi (111,11 €/mq nel primo blocco, 110,63 dal secondo in poi)

Un settimo caso resta fuori senza decisioni da prendere: una riga dichiarata «serramento + persiana» senza prodotto persiana scelto rende CHECK2 non calcolabile — fail-closed per progetto. Resta da raccogliere un foglio reale con serramenti **in legno**: nessuno dei 19 ne usava.

55.8 Fuori taglio e debito

- Tariffe modificabili con validità dalla UI (D10): il pannello è in sola lettura.
- I test di servizio di `server/contratti` e `server/computo` non possono forzare il repository in memoria quando `DATABASE_URL` è impostata (`getContrattiRepository` / `getComputiRepository` scelgono il driver da un singleton legato all'env al primo uso).
- L'INSERT in blocco di `commessa_righe` / `computo_voci` ha il tetto PostgreSQL di 65 535 parametri (~2 900 righe, ~4 600 voci per statement).
- `immobile` null viene letto come «altro»; i CHECK constraint delle tabelle non sono additivi (una categoria nuova richiede una migrazione).
- Nel `RigaContrattoEditor` la quantità degli accessori non segue quella della riga; `TariffeLimitiPanel` restituisce `null` su errore invece di dirlo.
- Aliquote di detrazione 2025/2027 nel seed: da confermare col commercialista.

56. Fatturazione dal contratto (piano 2, 04/09/2026)

Secondo piano (`docs/superpowers/plans/2026-09-04-fatturazione-dal-contratto.md` , 18 task). Dal contratto strutturato e dal computo dei limiti (§55) nasce la **bozza di fattura**, che il CRM emette su Fatture in Cloud con invio allo SdI (in prova finché la direzione non lo spegne), archivia in PDF e XML e segue nei suoi stati; la nota di credito, totale o parziale, passa dalla stessa pipeline. Tutto dietro l'interruttore `fatturazione`, che richiede anche `limiti` sullo stesso ambiente.

56.1 Modello dati (6 tabelle)

Tabella	Contenuto
fatturazione_config	per sede: IBAN, banca, intestatario, metodo di pagamento (default MP05), numerazione FiC, conto e id delle aliquote IVA letti da FiC, dicitura di piè di pagina, spese di documentazione, esito dell'ultima verifica dello scope di scrittura
fatture	sede, commessa, computo, hash righe, tipo (fattura / nota_credito), stato, id/numero/data FiC, snapshot cliente congelato all'emissione, imponibile/IVA/totale e scarto sul pattuito, diciture, chiavi di storage di PDF e XML con sha256, ultimo ei_status ed ei_errore , scavalco dei limiti e motivo, revisione
fattura_righe	intestazione , bene , servizio , markup , storno_bs , riaddebito_bs , nota : descrizione, quantità, prezzo, aliquota 22/10, collegamento alla voce di computo o alla riga di contratto, limite della voce
fattura_riepilogo_iva	imponibile e imposta per aliquota
fattura_scadenze	numero, quota %, data, importo, ficPaymentId , stato
fattura_eventi	append-only: creazione, modifiche, emissione_avviata , cliente_fic , crea_fic , xml_ok/xml_errore , inviata , stato_sdi , scarto , nota_credito , pdf_archiviato

server/fatture/repository.ts (memoria senza DATABASE_URL , PostgreSQL altrimenti, ensure-Schema memoizzato con ALTER additivi). Dallo stato in_emissione in poi il servizio rifiuta ogni modifica a righe, riepilogo e scadenze: la correzione è una nota di credito, mai una riscrittura.

56.2 Risolutore (server/fatture/risolutore.ts)

Simboli: **G** pattuito, **B** beni significativi, **N** altri beni, **S** servizi, **M** markup, **P = N + S + M** (prestazione).

- Regola dei beni significativi: se $B > P \rightarrow 10\%$ su $2P$ e 22% su $B - P$; se $B \leq P \rightarrow$ tutto al 10% .
- Pattuito imponibile: $M = G - B - N - S$.
- Pattuito lordo: ipotesi $B > P \rightarrow P = (G - 1,22 \cdot B) / 0,98$; se $P \geq B$ l'ipotesi cade $\rightarrow P = G / 1,10 - B$. Poi $M = P - N - S$.
- M è sempre derivato**, mai un input. Con $M < 0$ la bozza resta salvabile ma non emettibile (markup_negativo): la prassi della commercialista è **abbassare i beni**, non tagliare i servizi, e il pulsante «Riequilibra i beni» scala le righe dei beni significativi in proporzione fino al markup desiderato (default 0) — arrotondamento cumulativo, somma sempre esatta al target, righe mai negative, scarto ≤ 1 centesimo a riga.
- Tutto in centesimi; imposta per aliquota con arrotondamento half-up. Se il totale non coincide con G si cercano $P \pm 1...3$ centesimi, altrimenti resta uno scarto dichiarato che l'operatore accetta.

56.3 Generatore (server/fatture/generatore.ts)

Dalla commessa con computo valido, funzione pura:

- riga intestazione («Fattura per la prossima fornitura e posa di:» + categorie dal contratto);
- una riga **bene** per riga di contratto, aliquota 22% ;

- una riga **servizio** per ogni voce di computo con limite > 0, importo proposto = limite arrotondato all'euro **mai per eccesso**, aliquota 10 %, con il limite scritto sulla riga;
- riga **markup** «MarkUp servizi di vendita» al 10 %, dal risolutore;
- coppia **storno/riaddebito** dei beni significativi (−Q al 22 %, +Q al 10 %);
- riga **spese di documentazione**: è un **bene al 22 %** (non un servizio), configurabile per sede (default 150,00 €) ed esclusa sia dal blocco prodotti sia dal blocco servizi dei limiti;
- righe **nota**: dicitura dell'intervento, «manutenzione straordinaria» e template della pratica edilizia (con avviso quando CILA/SCIA sono dichiarate ma restano segnaposto da compilare), indirizzo di cantiere, bonifico parlante secondo il tipo di detrazione;
- scadenze dal contratto, default **50/40/10** a **0/60/75/90 giorni** con il resto sull'ultima.

In bozza si aggiungono e si tolgono righe manuali (massimo 20 per operazione, 300 caratteri di descrizione) e si scelgono le diciture di piè di pagina; le diciture che il generatore stampa già come testo di riga non compaiono nell'elenco selezionabile.

56.4 Limiti verificati per blocco

Tre confronti **separati**, mai un totale unico:

Blocco	Confronto
<code>limite_prodotti</code>	beni senza voce di computo (righe di contratto e manuali) più il markup , contro la Σ dei massimali
<code>limite_servizi</code>	Σ dei servizi (manuali compresi, spese di documentazione escluse) contro la Σ delle opere ed eventuali proposte
<code>limite_totale</code>	imponibile contro il minore fra CHECK1 e CHECK2

Le righe derivate (markup a parte, storno e riaddebito) non entrano mai in queste somme. Se il termine di paragone di un blocco è zero, l'esito è un avviso `limiti_non_verificati` — mai un «ok» di comodo e mai un errore; senza computo l'avviso copre tutti e tre i blocchi. L'eccedenza su una singola voce resta un indicatore: cambia la detrazione stimata del cliente, non ammette o vieta la fattura.

Un blocco oltre il limite blocca l'emissione. «Procedi comunque» richiede una **seconda autorizzazione** dentro `fatture.aggiornaBozza` — tracciata come endpoint `fatture.scavalcoLimiti`, capability `fattura.emit` — e un motivo non vuoto, controllato anche nel servizio perché valga fuori dal router; **spegnere** lo scavalco resta un'operazione da `fattura.draft`. Lo scavalco è registrato sulla fattura e dichiarato nel fascicolo. `rigeneraBozza` azzera scavalco e motivo, tornando alla proposta di sistema.

56.5 Emissione (`server/fatture/emissione.ts`)

Prerequisito una tantum per sede: scope OAuth FiC di **scrittura** su clienti, fatture e note di credito (§27.8), verificato con una chiamata di lettura a `/issued_documents/info`.

Passi, idempotenti uno per uno — se `ficDocumentId` esiste non si ricrea nulla, e **mai** una cancellazione automatica su FiC:

1. validazione (cliente completo, CF con checksum per i privati, P.IVA per le aziende, codice destinatario o PEC, requisiti della detrazione, computo valido o scavalco registrato, scadenze che sommano al totale, configurazione di sede completa);

2. cliente su FiC (ricerca per CF/P.IVA, altrimenti creazione con fattura elettronica attiva; un privato con un nome di una sola parola nasce come `company`, perché FiC rifiuta una `person` senza nome proprio);
3. contesto FiC (`/issued_documents/info` : id delle aliquote 22/10, conti, numerazioni), messo in cache nella configurazione di sede;
4. creazione del documento con righe, scadenze, IBAN e metodo di pagamento, codice commessa nell'oggetto visibile e diciture nelle note; **confronto dei totali** con i nostri: se differiscono ci si ferma prima dell'invio;
5. verifica dell'XML, ripetuta a ogni ripresa finché la fattura non è `inviata` ;
6. invio allo Sdl, con `dry_run` governato da `FATTURAZIONE_SDI_DRY_RUN` ;
7. archivio: XML e PDF nello storage con sha256, e il PDF **registrato come documento «fattura» della commessa** (soddisfa il gate documentale esistente);
8. eventi e timeline.

Lease. A impedire due documenti FiC non è il confronto di revisione — che si fa solo alla partenza, quando la fattura è ancora `bozza` — ma un compare-and-swap su stato e revisione all'inizio di ogni giro: due «Emetti» sovrapposti sulla stessa bozza danno `CONFLITTO` al secondo **prima** di toccare `Fatture in Cloud`. Mai due numeri quando le due run hanno letto la stessa revisione. Da `emessa` / `inviata` il lease non riporta indietro lo stato: serializza soltanto.

Dry-run. `FATTURAZIONE_SDI_DRY_RUN` è una variabile di **tutto il deployment**, non un campo per sede, ed è accesa finché non vale esplicitamente `off`. Con il dry-run l'invio è simulato: lo stato resta `emessa` (mai `inviata`) con l'etichetta «Emessa (prova Sdl)». FiC però **numera davvero** il documento: non ha bozze.

Sonda. `startSondaFattureWorker` gira ogni **15 minuti** in un solo processo sulle fatture inviate (e su quelle emesse in dry-run): legge `ei_status` e lo mappa negli stati del CRM, recupera l'archivio mancante e riappaia le scadenze rimaste scollegate — a **ogni** giro, non solo durante l'emissione. Non ritenta mai l'invio. Un pulsante «Aggiorna stato» fa lo stesso a richiesta. `eiErrore` è riscritto in fondo a ogni passaggio, mai lasciato appiccicato da un giro precedente già risolto.

56.6 Nota di credito

Da una fattura `emessa` o successiva (`emessa`, `inviata`, `consegnata`, `rifiutata`, `mancata_consegna`): per il totale le righe sono uno **specchio esatto** dell'origine, segno compreso; per la parziale solo le righe scelte, con storno e riaddebito ricalcolati sul sottoinsieme. Nessun risolutore: gli importi sono già decisi. Prima riga: un'intestazione «Accredito su ns. fattura n. X del Y», col motivo quando c'è. La nota salta i controlli di computo, limiti e forma della detrazione (storna, non propone prestazioni nuove) ma mantiene cliente, configurazione FiC e scadenze. Una nota di credito non si rigenera e non si storna con un'altra nota.

56.7 Sincronizzazione con Fatture in Cloud

Un documento FiC il cui id corrisponde a `fatture.ficDocumentId` nasce già collegato alla commessa (`commessaMatch`: `"crm"`): non passa dal match automatico, non rigenera il PDF (il sync degli allegati lo salta, il file è già nel fascicolo dall'emissione) e alimenta pattuito, rate e incassi come oggi (§40). La mappa CRM↔FiC si costruisce sugli id del giro, senza tetti. Un avviso compare quando il totale della fattura si discosta dal pattuito del contratto di oltre 1 €. La mutation manuale di collegamento **rifiuta** di

spostare una riga `commessaMatch: "crm"` su un'altra commessa: si corregge solo con una nota di credito.

56.8 Interfaccia

- Tab «**Fattura**» della commessa: bozza modificabile, riequilibrio dei beni, scadenze, emissione con conferma, vista della fattura emessa con le azioni ammesse dal suo stato, download di PDF e XML, dialog della nota di credito (`client/src/components/fattura/` , presentazione pura in `client/src/lib/fatturaView.ts`).
- Impostazioni → **Contabilità** (direzione, come tutta la sezione): pannello «Fatturazione» con IBAN, banca, intestatario, metodo, numerazione FiC, conto, spese di documentazione, «Verifica permessi» (che carica aliquote, numerazioni, conti e metodi da FiC) e la riga «Permessi di scrittura fatture» con la ri-autorizzazione OAuth. Dopo la verifica il conto auto-assegnato dal server entra nel modulo **solo se il campo era vuoto**: un conto scelto a mano non si perde. Scollegare l'OAuth FiC azzera anche l'esito della verifica.
- `/pagamenti` (Cassa): sezione «Fatture emesse dal CRM».

56.9 Tars

Nessuno strumento nuovo. Col flag acceso, `leggi_fascicolo_commessa` mostra una riga per fattura o nota — **mai un importo**, perché il fascicolo vive dietro `commessa.read` e non dietro le capability economiche: bozza → Fattura: bozza `#<id>` (con l'eventuale scavalco dei limiti attivo); emessa e oltre → Fattura n. X del Y: `<stato leggibile>` più, se serve, « · prova Sdl » e la frase fissa « · avviso: esito Sdl/FiC da verificare nella tab Fattura » — mai il testo di `eiErrore` , che porta importi. Il fascicolo si invalida a ogni scrittura sulla fattura e a ogni cambio del flag a runtime.

56.10 Permessi e flag

- `fattura.read` : amministrazione, commerciale, direzione. `fattura.draft` , `fattura.emit` , `fattura.credit_note` : amministrazione e direzione (§4.4).
- Interruttore `fatturazione` (`FLAG_FATTURAZIONE`) in middleware sui due router (`fatture` , `fatturazioneConfig`) e `limiti` verificato per handler: la fatturazione non esiste senza il contratto strutturato. Con `limiti` spento ogni mutation risponde `PRECONDITION_FAILED` . Entrambi fail-closed; questo documento non attesta lo stato dei flag in un ambiente esterno.

56.11 Runbook della prima fattura reale

Procedura completa in `handoff.md` §11-vicies quaterdecies. In sintesi:

1. sede o ambiente di prova, `FLAG_LIMITI` e `FLAG_FATTURAZIONE` accesi lì soltanto, `FATTURAZIONE_SDI_DRY_RUN` al suo default (acceso);
2. una commessa già fatturata a mano nel 2026: contratto e computo → «Genera bozza dai limiti» → confronto **riga per riga** col PDF della fattura reale (beni, servizi, markup, storno, riepilogo IVA, scadenze, spese di documentazione, diciture) e «Riequilibra i beni» fino ai valori tenuti dalla commercialista;
3. **prima di «Emetti»**, confermare la numerazione FiC col commercialista: se resta «Numerazione predefinita», FiC numera con la propria serie e non lo segnala come errore;

4. «Emetti» in dry-run con un solo operatore e una sola scheda, senza retry finché la chiamata non risponde;
5. XML scaricato dalla tab Fattura e verificato dal commercialista;
6. solo dopo quella conferma, `FATTURAZIONE_SDI_DRY_RUN=off` su quell'ambiente.

Alla prima nota di credito reale va verificato sul PDF di FiC il **segno** del totale: il CRM manda righe positive speculari all'origine, le note reali in mano alla commercialista stampano il totale in negativo.

56.12 Fuori taglio

- Fatture libere e acconti (non nati da un contratto strutturato).
- IVA al 4 % e clienti B2B senza contratto.
- PEC, codice destinatario e `ficEntityId` del cliente restano campi server (`clienti.update`) senza UI nel form cliente.
- Da confermare col commercialista: il segno della nota di credito e la company FiC di prova per la prima emissione reale (la numerazione è reale, non simulata).

57. Lettura del contratto PDF (piano 3, 04-05/09/2026)

Terzo piano (`docs/superpowers/plans/2026-09-04-lettura-contratto.md` , 9 task). Il modello legge il PDF del contratto firmato e **propone** righe, pattuito, posa, rate e cantiere; la proposta si rivede campo per campo e solo allora tocca il contratto strutturato del §55. Non salva mai da sola.

57.1 Flusso

1. Si parte da un documento di tipo `contratto` già nel fascicolo della commessa (PDF: nessun altro formato).
2. `disponibilitaEstrazione()` decide **prima di leggere qualunque cosa**: flag `contrattoEstrazione` e `limiti` accesi, più un provider Tars reale utilizzabile. Manca anche una sola condizione e la UI mostra solo «Compila a mano», con il motivo esatto — mai un pulsante che poi fallisce in silenzio.
3. Riuso: si controlla se esiste già un'estrazione per quel documento e quella versione di prompt **prima** di estrarre il testo, perché OCR e lettura visiva costano; la chiave di riuso porta l'impronta della configurazione OCR corrente.
4. Testo: `estraiTestoDocumento` chiamato **senza** l'opzione di lettura visiva — solo testo nativo e OCR locale. Una scansione che l'OCR non legge è un errore esplicito, mai una spesa aggiuntiva silenziosa.
5. Chiamata al modello (classe di costo `document_intelligence` , modello `TARS_MODEL_ESTRAZIONE_CONTRATTO`) con schema JSON **strict**: pagine intere fra marcatori, troncamento dichiarato mai a metà pagina, e **un solo tentativo** quando la risposta non è valida (stesso `runId` , un errore di altro tipo non si ritenta). Su fallimento non si salva nulla.
6. Mappatura deterministica al catalogo DEI, proposta salvata, revisione umana nel dialog, applicazione.

Il contenuto del PDF è **input non fidato**: nessuna istruzione al suo interno ha effetto (test di prompt injection incluso), e il testo di pagina neutralizza i marcatori di pagina prima di comporre l'input, così un documento non può fingere una pagina che non esiste.

57.2 Mappatura deterministica (estrazione/mappa.ts)

Il modello descrive, il codice decide. Ogni valore proposto porta {valore, evidenza, daVerificare, nota} e l'evidenza è **verificata sul testo vero** del PDF: una citazione che non si ritrova nasce «da verificare», mai spacciata per letta.

- **Codici DEI solo dal catalogo tariffe**, mai inventati dal modello: senza un candidato univoco la riga resta senza codice. Il tie-break fra candidati è dichiarato (prima la variante senza «> 1,3», poi la famiglia coerente col materiale, poi il codice).
- **Natura scorrevole/alzante/complanare**: vale per il sostantivo più **vicino** che la precede, non per il primo della descrizione; una parola di apertura esplicita del serramento (battente, ribalta, oscillobattente, vasistas) prevale sullo scorrimento e lo dichiara come avvertenza. La portafinestra scorrevole è un prodotto ordinario e non genera avvertenza di contrasto.
- **Materiale**: con più materiali citati vince il **primo nominato**, con avvertenza «più materiali citati» e campo da verificare; il materiale dell'oscurante si legge solo nel segmento di testo che segue la sua parola, e lì vince la posizione, non una precedenza fissa.
- **Oscuranti abbinati**: tapparelle, persiane e scuri autonomi si fondono nella riga del serramento solo se trovano un serramento con le **stesse misure** (± 10 mm) e i pezzi bastano per l'intera riga; la riga del serramento nasce «da verificare» con la nota di quanto comprende. Fuori da quelle condizioni restano righe a sé, con l'avvertenza che lo dice. I cassonetti citati come righe autonome restano righe autonome.
- **Accessori** riconosciuti solo da etichette note: un'etichetta libera che non trova corrispondenza resta in nota come «da verificare», mai un codice a caso.
- **Posa**: le parole chiave assorbono solo righe **senza misure**; `salvaContratto` forza `posaCent = null` quando la posa non è inclusa.
- **Controlli derivati** (`righe_vs_pattuito` , cliente, zona, aliquota) ricalcolati dopo ogni arricchimento; un pattuito lordo scorpora l'imponibile solo se l'IVA è a un'unica aliquota, altrimenti il controllo è saltato con avviso — mai un numero del contratto inventato.
- **Arricchimento layout WnD** (`layoutWnd.ts`), facoltativo: quando il testo porta le etichette esatte del configuratore, i suoi numeri riscrivono misure, quantità, prezzi, pattuito e rate con evidenza certa; su ogni altro contratto la proposta del modello resta intatta. È un solo arricchimento deterministico, non un parser per fornitore.

57.3 Persistenza, router e interfaccia

- `contratto_estrazioni` (`estrazione/repository.ts`): memoria senza `DATABASE_URL` , PostgreSQL altrimenti; idempotente per documento e versione del prompt.
- `estrazione/servizio.ts` : `eseguiEstrazioneContratto` (fail-closed **da solo**, non si fida del router), `applicaEstrazione` — che scrive **soltanto** attraverso `salvaContratto` , unico percorso, e mette stato e timeline in try/catch così che un loro errore diventi un'avvertenza e mai un contratto scomparso —, `scartaEstrazione` , `ultimaEstrazione` .
- Router `estrazioniContratto` (`stato` , `esegui` , `applica` , `scarta`): interruttore `contrattoEstrazione` in middleware, `limiti` per handler, `commessaId` verificato in sede su `applica` e su `scarta` (estrazione di un'altra commessa → `NOT_FOUND`). Nessuna capability nuova: riusa `contratto.read` / `contratto.manage` .
- UI: `client/src/components/contratto/LeggiContrattoDialog.tsx` — proposta con evidenza e nota per ogni campo, note del lettore (di riga e di testata), revisione inline riga per riga, applicazione verso il contratto strutturato; «Compila a mano» resta sempre disponibile quando la lettura non è

configurata. Il badge della zona e il filtro del catalogo usano la zona del contratto **salvato** solo se il comune proposto coincide con quello salvato; altrimenti lo dichiarano.

57.4 Valutazione

`pnpm eval:contratti ( server/contratti/eval/ )` gira sulle tre fixture sintetiche — layout WnD, documento Word, scansione — usando lo stesso estrattore di testo della produzione e poi la mappatura su un esito **finto**: nessuna rete, deterministico, coperto anche da `pnpm test`. Report Markdown con accuratezza per campo in `docs/reports/`. La chiamata reale è possibile **solo** con `EVAL_CONTRATTI_REALE=on` e un provider realmente disponibile: doppia condizione, difesa in profondità.

`server/contratti/eval/casi-reali/` è in `.gitignore` ed è **vuota**: nessun contratto reale anonimizzato è ancora stato raccolto, quindi oggi l'eval misura la tenuta di parser e mappatura su fixture controllate, non l'accuratezza reale del modello.

57.5 Attivazione

`FLAG_CONTRATTO_ESTRAZIONE` e `FLAG_LIMITI` accesi sullo stesso ambiente, più le condizioni del provider governato di Tars (provider reale configurato, modello con tariffa attiva, budget, ledger PostgreSQL autorevole) — le stesse verificate da `statoProvider`. Entrambi i flag sono fail-closed; questo documento non attesta lo stato di flag o provider in un ambiente esterno.

Runbook della prima lettura reale (dettagli in `handoff.md §11-vicies quindicies`): sede di prova → una chiamata di prova con `EVAL_CONTRATTI_REALE=on` **prima** di accendere il flag altrove, perché il pattern nullable dello schema strict non è mai stato esercitato dal vivo contro l'API reale da questo codebase → un contratto già inserito a mano, confrontato riga per riga con la proposta, prima di fidarsi su una commessa dove il contratto manca. Il costo del run si legge dal ledger di Tars per `runId`, classe `document_intelligence` (prima consumatrice reale di questa classe): non c'è un costo mostrato nella UI dell'estrazione.

57.6 Fuori taglio e debito

- Non applica nulla da sola, non legge contratti che non siano PDF, non inventa codici DEI, non ha strumenti Tars: la lettura resta un'azione umana dal dialog.
 - Residui dichiarati della disambiguazione: un sostantivo di accessorio ancora fuori dalla lista chiusa, su una portafinestra senza una parola di apertura esplicita, si prende ancora lo scorrimento in silenzio; una menzione composta («legno-alluminio») è dedotta come materiale unico senza avvertenza; l'intestazione del prompt (commessa e cliente CRM) non passa dalla neutralizzazione dei marcatori, che copre solo le pagine del documento.
 - L'editor delle rate è duplicato fra il dialog e la tab Contratto.
 - `posaCent` della proposta non ha ancora un consumatore a valle oltre al contratto salvato: lo consumerà la fatturazione, o va tolto quando quella decisione sarà presa.
 - Nel controllo delle misure il ramo che scarta una misura fuori intervallo non è raggiungibile dallo schema del modello: resta come difesa in profondità, non come comportamento provato dai test.
-

58. Fatturazione guidata (piano 4, PIANIFICATA il 05/09/2026)

Non implementata. Spec docs/superpowers/specs/2026-09-05-fatturazione-guidata-design.-md , piano docs/superpowers/plans/2026-09-05-fatturazione-guidata.md , approvati dalla direzione il 05/09/2026. Oggi contratto, limiti e fattura vivono in tre tab dense della pagina commessa, senza un ordine evidente e senza un posto dove vedere «cosa manca»: il piano 4 dà un ingresso unico.

Decisioni registrate:

- **Elenco** /fatturazione (gruppo Economia): tutte le commesse della sede negli stati `aggiornamenti_contratto` e `fatture_pagamento` che non hanno una fattura — né una fattura FiC collegata né una fattura CRM `emessa` o successiva. Una bozza non conta: la commessa resta in elenco con «Continua».
- **Percorso** /fatturazione/:commessaId : quattro passi in sequenza — Documenti, Contratto, Limiti, Fattura — con avanzamento visibile, interrompibile e riprendibile; «Avanti» attivo solo a passo fatto, indietro sempre possibile; chi non ha la capability di un passo lo vede in sola lettura.
- **Tab della pagina commessa** (Contratto, Limiti, Fattura): restano ma in **sola lettura**, con riassunto e pulsante «Apri fatturazione». Si lavora in un posto solo.
- **Card**: cliente, codice, stato e giorni nello stato, numero di documenti, i quattro passi come pallini, pattuito e importo previsto **solo con** `economia.read`.
- **Server**: un router `fatturazioneGuidata` di sole query (`daFare` , `passi`) dietro l'interruttore `limiti` , con lo stato dei passi calcolato da una funzione pura testata da sola; **nessuna mutation nuova** — i passi usano le procedure esistenti di contratto, computo, estrazione e fatture.
- **Fuori ambito**: transizioni di stato automatiche, nuove regole di dominio su contratto/limiti/fattura, ri-disegno dei componenti interni delle tab, notifiche.